



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

GENERACIÓN DE MALLAS POLIGONALES A PARTIR DE CAVIDADES

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

NICOLÁS ESCOBAR ZARZAR

PROFESORA GUÍA:
NANCY HITSCHFELD KAHLER

PROFESOR CO-GUÍA:
SERGIO SALINAS FERNÁNDEZ

MIEMBROS DE LA COMISIÓN:
ÉRIC TANTER
MAURICIO CERDA VILLABLANCA

Este trabajo ha sido parcialmente financiado por Fondecyt 1241596

SANTIAGO DE CHILE
2025

GENERACIÓN DE MALLAS POLIGONALES A PARTIR DE CAVIDADES

La generación de mallas poligonales de alta calidad es un problema fundamental en geometría computacional, con aplicaciones directas en simulación numérica, gráficos por computador y métodos numéricos como el *Virtual Element Method* (VEM). En particular, la construcción de mallas poligonales a partir de triangulaciones de Delaunay es de particular interés, ya que permite combinar buenas propiedades geométricas con una mayor flexibilidad en la forma de los elementos.

En este trabajo de memoria se propone y analiza una estrategia alternativa para la generación de mallas poligonales bidimensionales basada en el concepto de cavidad: a partir de una triangulación de Delaunay inicial, el algoritmo selecciona triángulos según criterios definidos por el usuario, calcula sus circuncentros y construye cavidades como el conjunto de triángulos cuyos circuncírculos contienen dichos puntos. La unión de las aristas de borde de cada cavidad define nuevos polígonos, dando origen a una malla poligonal compuesta por elementos de geometría arbitraria.

La solución propuesta se implementa en C++20 haciendo uso de estructuras *half edge* y un diseño modular basado en *templates* y *concepts*, lo que permite desacoplar la representación de la malla de los algoritmos de generación. Como parte del trabajo, se reimplementa el generador de mallas Polylla en este nuevo marco de diseño, facilitando su comparación directa con el algoritmo basado en cavidades, y otorgándole mejor legibilidad y capacidad de extensión al código.

Los resultados experimentales muestran que el enfoque propuesto es capaz de generar mallas poligonales válidas y de buena calidad geométrica, teniendo una concentración mayor de polígonos con una cantidad de aristas entre 4 a 7; comparables a las obtenidas por Polylla, en términos de cantidad de polígonos, convexidad y distribución de ángulos, pero con tiempos de ejecución y uso de memoria mayores. Finalmente, se discuten aspectos a mejorar y posibles extensiones del trabajo realizado, destacando su potencial como alternativa para la generación de mallas aptas para el VEM.

A mi familia, amigos y especialmente a mi madre por apoyarme siempre.

Tabla de Contenido

1. Introducción	1
1.1. Introducción y Motivación	1
1.2. Objetivos	4
1.2.1. Objetivo General	4
1.2.2. Objetivos Específicos	4
1.3. Contenido de la memoria	4
2. Marco Teórico	6
2.1. Conceptos relevantes	6
2.1.1. Polígono, polígono simple y no simple	6
2.1.2. Mallas poligonales	7
2.1.3. Cavidad	8
2.1.4. Mallas poligonales de polígonos arbitrarios	8
2.1.5. <i>Finite Element Method</i> (FEM) y <i>Virtual Element Method</i> (VEM)	10
2.1.6. Estructura <i>Half-Edge</i>	11
2.1.7. C++20	12
2.2. Estado del arte: herramientas para la generación de mallas	14
2.2.1. Triangle	14
2.2.2. Detri2: generador y visualizador de triangulaciones	18
2.2.3. Polylla	18
2.2.4. CGAL	21
3. Problema	22
3.1. Polylla-Mesh-DCEL[1]	22
3.2. Diseño propuesto	27
4. Implementación de algoritmo basado en cavidades	43
4.1. Algoritmo de generación de mallas basado en cavidades	43
4.1.1. Selección y ordenamiento de triángulos según criterios	43

4.1.2. Cómputo de circuncentro y cavidades	45
4.1.3. Inserción de Cavidades	50
4.1.4. Postprocesado	52
4.2. Reescritura de Polylla	56
5. Resultados	57
5.1. Parámetros utilizados y características de la máquina	57
5.2. Muestra de mallas generadas	57
5.3. Tablas comparativas de cualidades geométricas, tiempo y memoria	61
5.4. Distribución de polígonos según cantidad de aristas	65
5.5. Análisis y discusión	72
5.5.1. Eficiencia en tiempo y memoria	72
5.5.2. Calidad geométrica de las mallas	72
5.5.3. Impacto del postprocesado	73
5.5.4. Discusión	73
6. Conclusión	74
Bibliografía	75
A. Repositorio del proyecto	80
B. Formatos de archivo para mallas geométricas	81
2.1. Formato .off	81
2.2. Formatos .node, .ele y .neigh	81
2.2.1. Formato .node	81
2.2.2. Formato .ele	82
2.2.3. Formato .neigh	82
2.3. Formato .ale	82
2.4. Formato .mat de MATLAB	83
C. CLI11	85
D. PlantUML y Clang-UML	86
E. Diagrama completo de clases	87

Índice de Tablas

Tabla 1	Tabla comparativa de cantidad promedio de polígonos según cantidad de vértices para cada generador	61
Tabla 2	Tabla comparativa de tiempo de ejecución total en milisegundos	62
Tabla 3	Tabla comparativa de uso de memoria total en Kilobytes, truncado a la unidad	62
Tabla 4	Tabla comparativa de porcentaje de polígonos convexos	62
Tabla 5	Tabla comparativa de ángulo mínimo y máximo en grados	62
Tabla 6	Tabla de tiempos promedio en milisegundos detallados para el generador basado en cavidades según cantidad de vértices, truncado a 3 decimales	63
Tabla 7	Tabla de uso promedio de memoria detallado en KB para el generador basado en cavidades según cantidad de vértices	63
Tabla 8	Tabla de tiempos promedio en milisegundos detallados para Polylla nuevo según cantidad de vértices, truncado a 3 decimales	64
Tabla 9	Tabla de uso promedio de memoria detallados en KB para Polylla nuevo según cantidad de vértices	64
Tabla 10	Tabla de tiempos promedio en milisegundos detallados para Polylla original según cantidad de vértices, truncado a 3 decimales	65
Tabla 11	Tabla de uso promedio de memoria detallados en KB para Polylla original según cantidad de vértices	65
Tabla 12	Iconografía de PlantUML	86

Índice de Ilustraciones

Figura 1	Triangulación de Delaunay con circuncírculos compuesta por 10 vértices. . . .	2
Figura 2	Selección de triángulos para formar un polígono a partir de una cavidad. . . .	3
Figura 3	Polígonos resultantes de las cavidades marcados en azul con los lados discontinuos borrados	3
Figura 4	Ejemplos de polígono no simple	7
Figura 5	Ejemplo de diagrama de Voronoi. Fuente: Software Detri2[2]	9
Figura 6	a) Triangulación de Delaunay y Diagrama de Voronoi superpuesto, b) Triangulación de Delaunay restringida, c) Diagrama de Voronoi restringido [3]	10
Figura 7	Triangulación de Delaunay (en negro) junto a su diagrama de voronoi (en rojo). Fuente: Wikimedia[4]	10
Figura 8	Ejemplo de estructura <i>half-edge</i> [5]	11
Figura 9	Segundo ejemplo de estructura <i>half-edge</i> [6]	12
Figura 10	PSLG de entrada una guitarra electrica con un número de vértices y segmentos no definidos como se ilustra en Triangle [7]	15
Figura 11	Triangulación de la cerradura convexa del PSLG de la Figura 10 con segmentos originales faltantes [7]	15
Figura 12	Triangulación restringida del PSLG de la Figura 10 [7]	16
Figura 13	Triangulación restringida del PSLG de la Figura 10 con triángulos extra removidos [7]	16
Figura 14	Segmentos divididos según su círculo diametral [7]	17
Figura 15	Triángulo separado según su circuncentro [7]	17
Figura 16	Malla poligonal final resultante de aplicar el algoritmo Triangle [7]	18
Figura 17	Triangulación de Delaunay y su Diagrama de Voronoi dual en el software Detri2	18
Figura 18	Región de arista terminal. a) $Lepp(t_0)$ donde la arista roja es la arista terminal. b) Cuatro Lepps con la misma arista terminal: $Lepp(t_a)$, $Lepp(t_b)$, $Lepp(t_c)$, $Lepp(t_d)$. c) Región de arista terminal generada por la unión de los Lepp de b) [3]	19
Figura 19	a) Colección de vértices aleatorios, b) Triangulación de Delaunay donde las líneas sólidas son aristas frontera, las líneas punteadas negras son aristas	

	internas y las aristas punteadas rojas son aristas terminales. c) Partición a partir de regiones de arista terminal [3]	20
Figura 20	Triangulación de Delaunay y su malla Polylla respectiva [1]	20
Figura 21	Representación UML de estructuras <code>vertex</code> y <code>halfEdge</code> de Polylla-Mesh-DCEL	23
Figura 22	Representación UML de la clase <code>Triangulation</code> de Polylla-Mesh-DCEL[1] .	24
Figura 23	Representación UML de la clase <code>Polylla</code> de Polylla-Mesh-DCEL[1]	26
Figura 24	Representación UML de la clase <code>PolygonalMesh</code>	28
Figura 25	Representación UML de la clase <code>HalfEdgeMesh</code>	30
Figura 26	Representación UML de las clases <code>Vertex</code> , <code>HEVertex</code> y <code>HalfEdge</code>	31
Figura 27	Representación UML de la clase abstracta <code>MeshReader</code>	32
Figura 28	Representación UML de la clase abstracta <code>MeshWriter</code>	33
Figura 29	Representación UML de las clases <code>NodeEleReader</code> y <code>OffReader</code>	33
Figura 30	Representación UML de las clases <code>OffWriter</code> y <code>AleWriter</code>	33
Figura 31	Representación UML de la clase <code>MeshGenerator</code>	34
Figura 32	Representación UML de la clase <code>PolyllaGenerator</code>	35
Figura 33	Representación UML de la clase <code>MeshGeneratorData</code>	36
Figura 34	Representación UML de enumeraciones para estadísticas	36
Figura 35	Representación UML de la clase <code>PolyllaData</code>	37
Figura 36	Representación UML de la clase <code>MeshHelper</code> de Polylla	38
Figura 37	Representación UML de la clase <code>DelaunayCavityGenerator</code>	39
Figura 38	Representación UML de la estructura <code>Cavity</code>	41
Figura 39	Representación UML de la clase <code>DelaunayCavityData</code>	41
Figura 40	Representación UML de la clase <code>MeshHelper</code> del algoritmo basado en Cavidades	41
Figura 41	Algoritmo de selección de triangulos	45
Figura 42	Algoritmo de cómputo de cavidades	49
Figura 43	Algoritmo de inserción de cavidades	51
Figura 44	Pasos de la inserción de una cavidad	52
Figura 45	Algoritmo de fusión de polígonos	55
Figura 46	Malla poligonal generada por Triangle[8] de 10 vértices y 10 triángulos . . .	58
Figura 47	Malla poligonal generada por Polylla original a partir de la Figura 46 con 4 polígonos	58
Figura 48	Malla poligonal generada desde cavidades a partir de la Figura 46 antes de postprocesar con 5 polígonos	59
Figura 49	Malla de la Figura 48 después de postprocesar con 3 polígonos	59
Figura 50	Malla poligonal generada por Triangle[8] de 100 vértices y 268 triángulos . .	59
Figura 51	Malla poligonal generada por Polylla original a partir de la Figura 50 con 37 polígonos	59
Figura 52	Malla poligonal generada desde cavidades a partir de la Figura 50 antes de postprocesar con 62 polígonos	60
Figura 53	Malla de la Figura 52 después de postprocesar con 48 polígonos	60

Figura 54	Malla de la Figura 47 con la nueva implementación de Polylla con 4 polígonos	60
Figura 55	Malla de la Figura 51 con la nueva implementación de Polylla con 37 polígonos	61
Figura 56	Distribución promedio de polígonos según cantidad de aristas con 10 vértices	66
Figura 57	Distribución promedio de polígonos según cantidad de aristas con 100 vértices	67
Figura 58	Distribución promedio de polígonos según cantidad de aristas con 1000 vértices	68
Figura 59	Distribución promedio de polígonos según cantidad de aristas con 10000 vértices	69
Figura 60	Distribución promedio de polígonos según cantidad de aristas con 100000 vértices	70
Figura 61	Distribución promedio de polígonos según cantidad de aristas con 1000000 de vértices	71
Figura 62	Diagrama UML completo de la implementación de Polylla original	88

Capítulo 1

Introducción

1.1. Introducción y Motivación

Las mallas poligonales son estructuras fundamentales en el modelado geométrico y la simulación computacional. Se definen como colecciones de vértices, aristas y caras que, en conjunto, representan la superficie de un objeto, generalmente mediante una subdivisión en triángulos. Esta representación es ampliamente utilizada en áreas como el diseño asistido por computador (CAD), gráficos por computadora, ingeniería estructural, biomecánica y videojuegos. La calidad de las mallas generadas influye directamente en la precisión de las simulaciones y en el rendimiento computacional de los sistemas que las utilizan.

Uno de los problemas recurrentes en este ámbito es la generación automática de mallas poligonales a partir de un conjunto discreto de puntos en el plano, también conocido como problema de triangulación. Si bien existen algoritmos eficientes disponibles en herramientas como Triangle [7] o Detri2 [2] para formar una triangulación de Delaunay [9], la construcción de formas poligonales a partir de dichas triangulaciones sigue siendo un área de investigación activa. En particular, existe un interés creciente por encontrar métodos que generen mallas con mayor fidelidad geométrica, adaptabilidad local, y que mantengan cualidades deseables como ángulos adecuados y buena distribución de los elementos.

Una triangulación de Delaunay se define como tal si cumple con la propiedad de que para todos los triángulos de la triangulación, se cumple que el circuncírculo del triángulo definido por sus vértices no contiene ningún otro en su interior (Figura 1). Estas son de particular importancia porque maximizan el tamaño del ángulo más pequeño de la triangulación. Esta propiedad previene problemas de precisión que ocurren al tener ángulos muy agudos los cuales propician “casos degenerados” donde los puntos de un triángulo son interpretados como colineales. Programas que trabajan con ESTAS mallas resultantes, que no sean lo suficientemente robustos, puedan fallar por completo en el peor de los casos.

Este trabajo de memoria se enfoca en el desarrollo y análisis de una estrategia alternativa de generación de mallas en dos dimensiones, basada en el concepto de *cavidad* o *concavity*

como se describe en el artículo Triangle [7]. La idea central es que, a partir de una triangulación de Delaunay (como la de la Figura 1), se seleccionan ciertos triángulos de la malla en un orden particular utilizando un criterio definido por el usuario. Luego, se calculan los circuncentros p_i de estos triángulos, y se identifican los conjuntos de triángulos de la malla cuyo circuncírculo contiene alguno de estos puntos por separado. La unión de las aristas del borde de estos triángulos vecinos para un punto p forma una *cavidad*, la cual define un nuevo polígono. Este procedimiento permite generar regiones poligonales que pueden servir como base para construir un nuevo tipo de mallas poligonales.

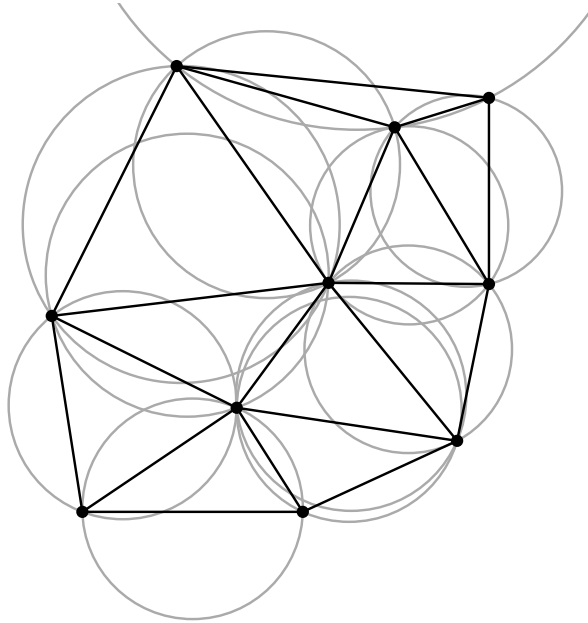


Figura 1: Triangulación de Delaunay con circuncírculos compuesta por 10 vértices.

A modo ilustrativo, en la Figura 2 se presenta una selección de triángulos a partir de la triangulación de la Figura 1.

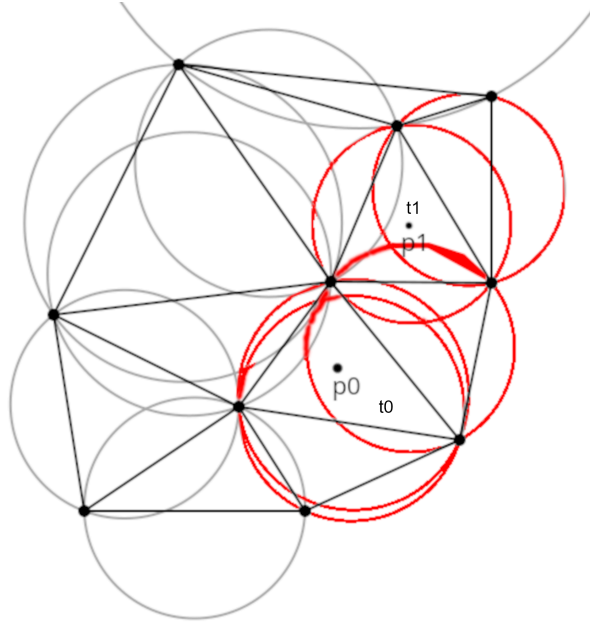


Figura 2: Selección de triángulos para formar un polígono a partir de una cavidad.

En este ejemplo, se seleccionaron dos triángulos t_0 y t_1 , cuyos respectivos circuncentros están representados por los puntos p_0 y p_1 . Estos puntos están contenidos en los circuncentros de varios triángulos vecinos, incluyendo aquellos que los generaron. La unión de las aristas de borde de los triángulos que contienen a p_0 y la unión de las aristas de borde de los triángulos que contienen a p_1 forman cada uno una cavidad, las cuales se pueden ver en la Figura 3.

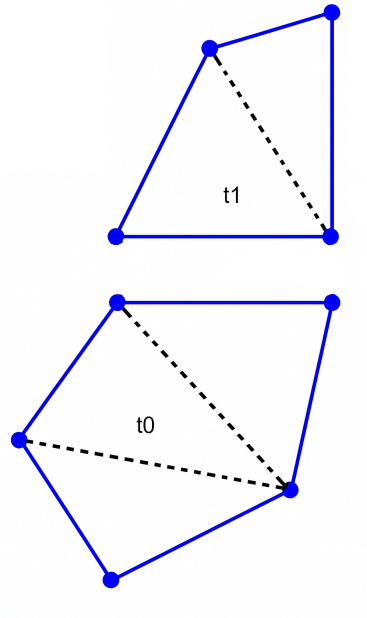


Figura 3: Polígonos resultantes de las cavidades marcados en azul con los lados discontinuos borrados

Este enfoque mediante cavidades no solo busca explorar una nueva forma de construir mallas, sino también comparar su rendimiento con métodos existentes. En particular, se contrastará esta técnica con el generador de mallas *Polylla* [3], un sistema moderno que emplea estructuras de datos avanzadas para lograr eficiencia y calidad en la generación de polígonos.

La necesidad de desarrollar nuevas estrategias para la generación de mallas responde a múltiples factores: mejorar la eficiencia de los algoritmos existentes, generar elementos con mejores propiedades geométricas, y adaptar las mallas a dominios complejos sin intervención manual.

1.2. Objetivos

1.2.1. Objetivo General

Diseñar e implementar un algoritmo que permita generar polígonos a partir de una *cavidad* desde una triangulación de Delaunay y comparar su desempeño con el generador de mallas Polylla en cuanto a tiempo, memoria, calidad de las mallas, número de vértices, aptitud para el VEM, entre otros criterios.

1.2.2. Objetivos Específicos

1. Investigar como funciona el algoritmo de mallas Polylla, sus estructuras de datos y métricas apropiadas para el VEM.
2. Definir escenarios de prueba para que el conjunto de polígonos que conforman la cavidad computada sean los correctos y generar un nuevo polígono a partir de ellos.
3. Comparar el desempeño del algoritmo basado en cavidades con el algoritmo de Polylla basado en regiones terminales en cuanto a eficiencia en tiempo y memoria.
4. Diseñar e implementar estrategias para seleccionar triángulos cuyo circuncentro se usará para definir cavidades.
5. Comparar la calidad de las mallas resultantes de ambos algoritmos bajo diversas métricas como calidad de la malla, número de aristas, cantidad de polígonos, etc.

1.3. Contenido de la memoria

Los capítulos siguientes de esta memoria abordan lo siguiente:

En el capítulo 2 se describen los conceptos necesarios para comprender este trabajo de memoria, junto con un resumen del estado del arte listando herramientas existentes. En el capítulo 3 se plantea el problema a resolver, se explica el funcionamiento del algoritmo a adaptar (Polylla[3]), su diseño, un breve análisis de aspectos a mejorar y el diseño del algoritmo basado en cavidades junto con una explicación del mismo.

En el capítulo 4 se muestra la implementación de la solución, su estructura a nivel de código, algoritmos concretos y validaciones. En el capítulo 5 se presentan los resultados (mallas poligonales) de la ejecución del algoritmo en diversas configuraciones, detallando

estadísticas obtenidas y tablas comparativas de calidad, tiempo y memoria tanto del algoritmo original, su reimplementación y el algoritmo de la solución.

Finalmente en el capítulo 6 se analizan los resultados obtenidos y un posible trabajo futuro.

Capítulo 2

Marco Teórico

En este capítulo se explican los conceptos relevantes utilizados a lo largo del trabajo memoria, relacionados con geometría computacional y el lenguaje de programación C++, adicionalmente, se hará una breve mención del estado del arte en cuanto a generación de mallas con su algoritmo o software pertinente.

2.1. Conceptos relevantes

2.1.1. Polígono, polígono simple y no simple

Un polígono es un caso particular de figura geométrica en dos dimensiones, compuesto por un conjunto de segmentos de recta conectados que forman una región en el plano (Figura 3). Un polígono se define como simple, si sus aristas forman un ciclo cerrado, es decir, con una única división entre su interior y exterior. Por otro lado, un polígono no simple, es aquel que posee aristas internas o externas que rompen el ciclo (ver Figura 4). Estos últimos son problemáticos en prácticamente cualquier caso de uso, ya que no permiten “recorrer” los polígonos una *mallla poligonal* de manera regular y se tratan de eliminar de esta de alguna manera en caso de que estén presentes.

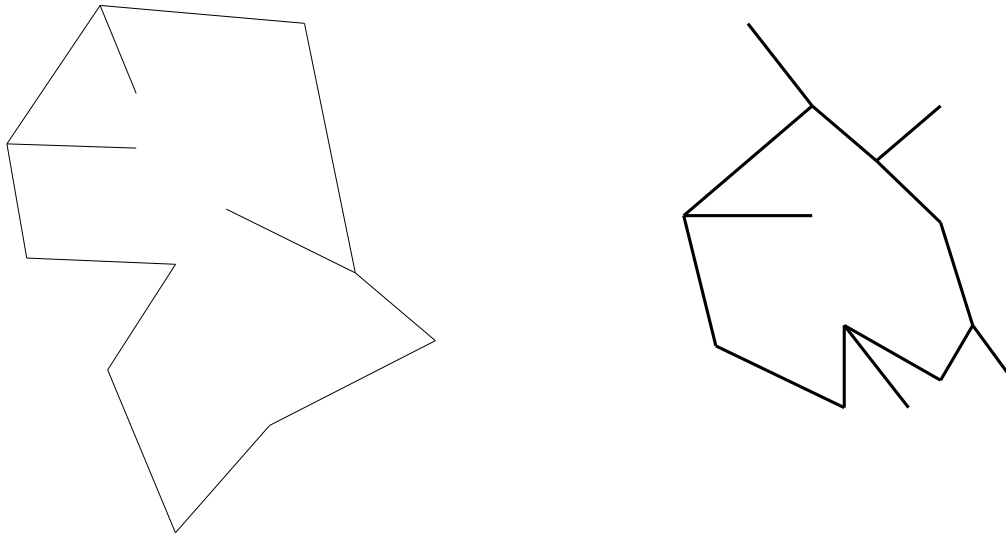


Figura 4: Ejemplos de polígono no simple

2.1.1.1. Planar Straight Line Graph (PLSG)

Un conjunto de vértices y segmentos que describen la geometría de un objeto como el de la Figura 10. Los segmentos de un PLSG describen una forma o borde concreto que puede no ser convexa como la del ejemplo. Un PLSG se puede utilizar como base para crear una *mallá geométrica*.

2.1.2. Mallas poligonales

Una malla poligonal, o malla geométrica, es una forma de describir un objeto o un espacio como una colección de polígonos adyacentes. Estos polígonos se denotan según sus vértices y aristas que unen dichos vértices para formarlos (sus caras). Dichos polígonos pueden existir en un espacio en dos, tres o incluso más dimensiones dependiendo del caso de uso. Este trabajo de memoria solo se centrará en aplicaciones a mallas geométricas en 2D.

Una malla se puede representar de varias formas, siendo la más común una basada en caras, en que se guarda la información de los vértices que componen la malla y qué vértices forman cada cara, siendo las aristas guardadas de manera implícita en las caras. En la solución propuesta se hace uso de esta representación al recibir una malla inicial como entrada contenida en uno o más archivos de texto, ya sea en formato `.node`, `.ele` y opcionalmente `.neigh` que guardan vértices, aristas (en forma de caras) e información de adyacencia respectivamente o en formato `.off` que guarda información de vértices y aristas, pero no de adyacencia. También se escribirán las mallas resultantes en formato `.off` o en el formato `.ale`, que también posee una representación basada en caras, el cual puede convertirse al formato binario `.mat` utilizado principalmente por MATLAB u otro software científico para usar la malla en una solución numérica de una ecuación diferencial en derivadas parciales. Estos formatos se describen más a fondo en el Anexo B.

Además de su representación, una malla poligonal también se puede clasificar en distintos tipos según los polígonos que la componen y las características que estos cumplen.

La representación más común es mediante una malla compuesta por triángulos, también llamada triangulación. Una triangulación divide el objeto u espacio a representar mediante el uso de triángulos, insertando puntos en su interior para formarlos de ser necesario. Dado que el triángulo es el polígono con menos aristas, este permite hacer una discretización, es decir, una división en partes concretas, del objeto o espacio a modelar con mayor precisión, y además, estos tienen la ventaja de ser siempre convexos y sus vértices viven en el mismo plano en el caso de mallas en tres dimensiones (3D). Las mallas basadas en triángulos son particularmente útiles para describir objetos en sistemas CAD usadas como esquema de diseño para algún tipo de objeto concreto en 3D afecto a fenómenos físicos como la distribución de fuerzas, cuya simulación es más fiel a la realidad en un objeto descrito con el mayor nivel de detalle que sea razonable utilizar. Estas mallas también son ampliamente utilizadas en videojuegos por las mismas razones.

Un caso particular de triangulación, es la llamada triangulación de Delaunay, esta triangulación cumple la propiedad de que para todo triángulo que conforma la triangulación, su *circuncírculo*, es decir, el círculo único cuya circunferencia pasa por sus 3 vértices, no contiene ningún punto, como en la Figura 1. Como se mencionó anteriormente, las triangulaciones de Delaunay son particularmente útiles debido a que maximizan el ángulo más pequeño de la triangulación, cuya utilidad será explicada más adelante. Cuando una triangulación de Delaunay se utiliza para subdividir un objeto con un borde concreto, como el de uno descrito por un PLSG, se dice que la triangulación de Delaunay es restringida, ya que debe incluir dichas aristas y no tener aristas fuera del borde. Esta distinción se hace porque típicamente una triangulación de Delaunay suele triangular conjuntos de puntos sin un borde definido inicialmente, el cual termina siendo la cápsula convexa del conjunto, donde su cápsula convexa es el menor polígono convexo (en cuanto a superficie o volumen) que contiene todos los puntos en su interior.

2.1.3. Cavidad

En la explicación de la solución se dará a entender como cavidad el polígono resultante de aplicar el algoritmo propuesto, es decir, dado un triángulo “semilla” t_i con circuncentro p_i , la cavidad será el polígono formado por la unión de las aristas de borde del conjunto de triángulos t cuyo circuncírculo contiene al circuncentro p_i del triángulo “semilla” en su interior. Ver Figura 3.

2.1.4. Mallas poligonales de polígonos arbitrarios

Otro uso de las mallas poligonales es en el cálculo de una solución numérica en la resolución de ecuaciones diferenciales en derivadas parciales, que describen diversos fenómenos como transferencia de calor o sonido en un espacio u objeto, las cuales no son calculables de manera exacta con un procedimiento analítico. Esta solución numérica se puede aproximar mediante el uso del *Virtual Element Method*[10] (VEM). Para el VEM son de particular importancia las mallas poligonales basadas en polígonos arbitrarios que cumplen ciertas propiedades de calidad, como tener polígonos simples, mayormente convexos, con ángulos no muy grandes ni muy pequeños, entre otras. Si la malla a utilizar parte desde una

triangulación de Delaunay, algunas de estas propiedades son más fáciles de alcanzar. El uso de mallas basadas en polígonos arbitrarios permite un cálculo más rápido para el VEM y con un margen de error aceptable.

Un caso particular de malla poligonal de polígonos arbitrarios, es aquella basada en un *diagrama de Voronoi* cuando este está restringido. Un diagrama de Voronoi es una partición de un dominio P en regiones o “celdas” R_i , de las cuales cada una contiene un punto p_i llamado “semilla” y los puntos q_i contenidos en R_i cumplen que $\|q_i - p_i\| < \|q_i - p_j\| \forall p_i, p_j \in P, i \neq j$ donde p_j es la semilla de cualquier otra celda distinta a R_i . Las regiones de un diagrama de Voronoi normalmente se extienden “hacia el infinito” (Figura 5), pero al igual que las triangulaciones de Delaunay restringidas, también existe el diagrama de Voronoi restringido cuando este modela una geometría particular, usualmente basado en una misma triangulación de Delaunay restringida.

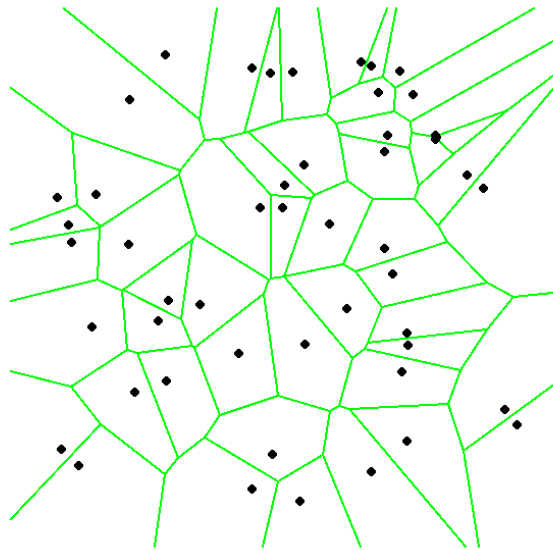


Figura 5: Ejemplo de diagrama de Voronoi. Fuente: Software Detri2[2]

El diagrama de Voronoi también se le conoce como el *dual* de una triangulación de Delaunay, esto se debe a que, dada una triangulación de Delaunay, se puede obtener su diagrama de Voronoi equivalente si los circuncentros de los triángulos se convierten en puntos para las regiones de Voronoi (el circuncentro es el punto al centro del circuncírculo de un triángulo). Al unir los circuncentros mediante aristas, se forman las regiones del diagrama de Voronoi. En la Figura 6 se puede ver un ejemplo de una triangulación de Delaunay restringida con su diagrama de Voronoi restringido respectivo superpuesto. También en la Figura 7 se puede ver una triangulación de Delaunay no restringida (segmentos en el borde corresponden a la cápsula convexa) junto con su diagrama de Voronoi superpuesto.

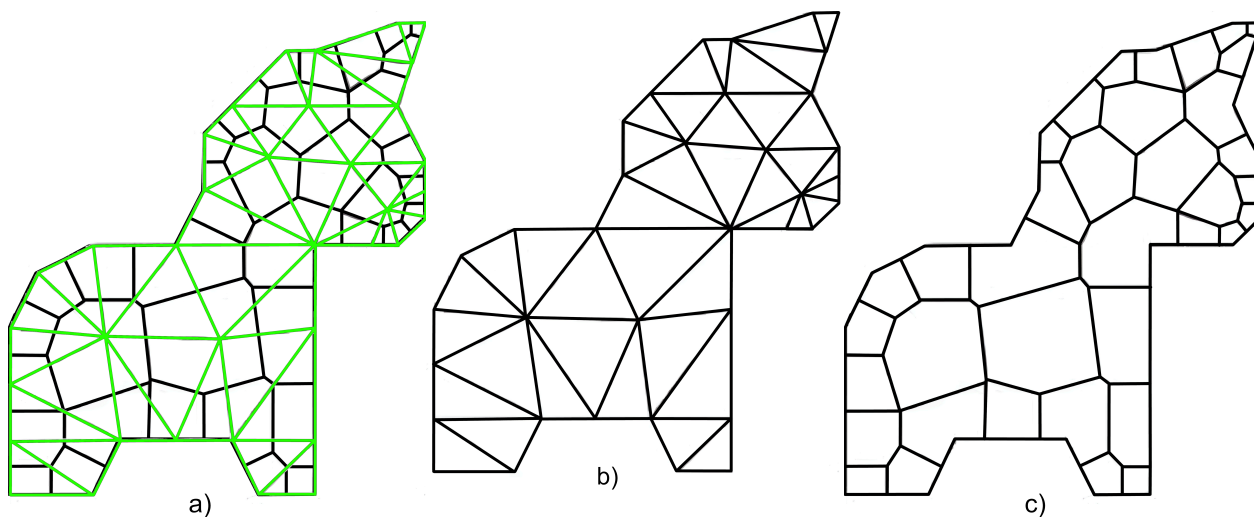


Figura 6: a) Triangulación de Delaunay y Diagrama de Voronoi superpuesto,
b) Triangulación de Delaunay restringida,
c) Diagrama de Voronoi restringido [3]

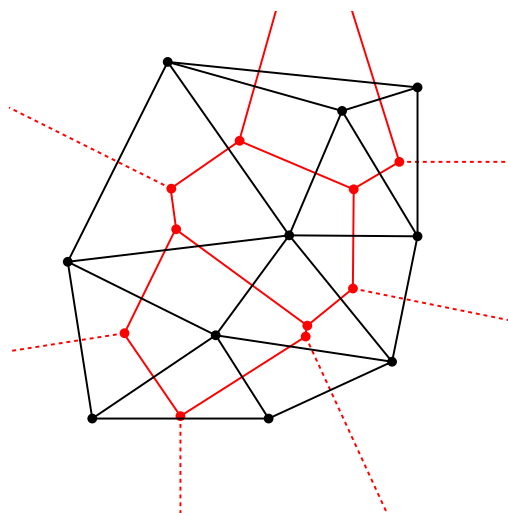


Figura 7: Triangulación de Delaunay (en negro) junto a su diagrama de voronoi (en rojo).
Fuente: Wikimedia[4]

2.1.5. *Finite Element Method (FEM) y Virtual Element Method (VEM)*

Como fue mencionado anteriormente, el VEM[10] es un método numérico para resolver ecuaciones diferenciales haciendo uso de las mallas poligonales arbitrarias, pero antes de que existiese el VEM, existía el FEM[11, 12], estos métodos numéricos tienen el mismo objetivo, pero se diferencian en la flexibilidad permitida de los datos de entrada, siendo el FEM mucho más rígido respecto a la malla de entrada, en particular, el FEM no permite polígonos no convexos y solo acepta polígonos de un solo tipo particular como entrada, ya sean triángulos, cuadriláteros, u otros, pero siendo todos del mismo tipo, lo que no lo hace viable para algoritmos como el desarrollado en este tema de memoria.

2.1.6. Estructura *Half-Edge*

Además de la representación basada en caras para mallas poligonales, se hará uso de la estructura *Half-Edge*[13, 14], la cual guarda los vértices y divide las aristas de cada polígono en dos, una arista en sentido horario (abreviado como *CW* por *clockwise* del inglés) y otra en sentido antihorario (abreviado como *CCW* por *counter clockwise*). Por convención, una arista en sentido antihorario, se considera como una arista interna a un polígono y un polígono cualquiera se representa como el ciclo completo que inicia desde una arista en sentido *CCW* y vuelve a la misma. Cada arista posee la siguiente información: un vértice de origen (*origin*), un vértice objetivo (*target*), su arista siguiente y anterior (según su orientación, llamadas *next* y *prev* respectivamente), y su arista “gemela” (*twin*), la cual representa la misma arista en el sentido contrario la cual es interna al polígono vecino si no está en el borde de la malla. En la Figura 8 se puede ver un ejemplo de esta estructura.

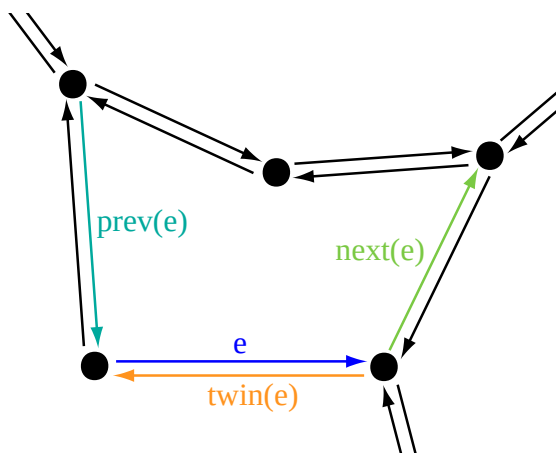


Figura 8: Ejemplo de estructura *half-edge*[5]

La estructura *Half-Edge* brinda una enorme versatilidad al momento de recorrer mallas, ya que, dada la orientación de sus aristas, es posible saber cuando se recorrió un polígono completo si se visitan las aristas *next* en un ciclo hasta volver a la arista original. Dentro de una malla también se definen otras operaciones de recorrido que se pueden derivar directamente de los atributos existentes, estas son: *CWEdgeToVertex* y *CCWEdgeToVertex*, las cuales permiten encontrar la arista “siguiente” de la actual en sentido horario y antihorario respectivamente. En la Figura 9 es posible notar que partiendo desde el vértice *vertex*, la arista siguiente en sentido horario de aquella marcada como *halfedge*, es el *next* de su *twin*, y la arista siguiente en sentido antihorario de *halfedge*, es el *twin* de su *prev*.

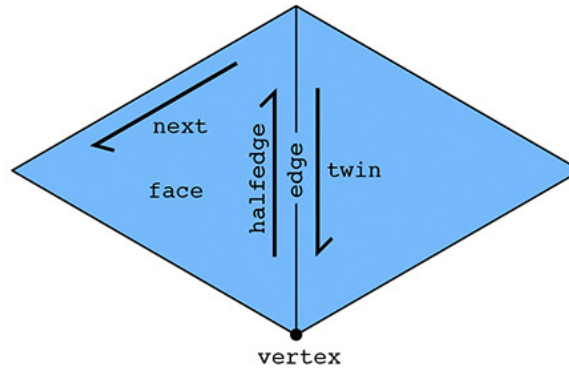


Figura 9: Segundo ejemplo de estructura *half-edge*[6]

Haciendo uso de las operaciones anteriores también es posible determinar el “grado” de un vértice, donde el “grado” (o *degree*) de un vértice se entiende como la cantidad de aristas cuyo origen es este vértice.

2.1.7. C++20

Para este trabajo de memoria se hace uso del lenguaje de programación C++ en su estándar C++20 (el estándar actual al momento de escribir este documento es C++23). C++ es un lenguaje que en sus orígenes, introdujo clases y programación orientada a objetos al lenguaje C, pero hoy en día es un lenguaje multi-paradigma que permite características propias de lenguajes funcionales, como el uso de funciones lambda.

Se escogió este lenguaje por su alto rendimiento y por el hecho de que la implementación de Polylla[1] en la que se basa este trabajo fue escrita en este lenguaje. La versión específica del estándar se escoge para asegurar mejor compatibilidad y por la introducción de *concepts*[15, 16] al lenguaje.

En C++ es posible declarar una “plantilla” (o *template*[17, 18]) de una clase “A” otorgándole un parámetro de tipo que en principio puede ser cualquier cosa, generalmente a este tipo genérico se le llama “T”, “U”, o similar; y dentro de la clase se puede operar con variables declaradas con un tipo “T” sin hacer ninguna verificación preliminar. Una vez que la clase se utiliza efectivamente dentro del código es cuando este tipo genérico T se debe declarar explícitamente como algún tipo concreto particular, y es solo entonces cuando el compilador genera el código máquina relevante para que la clase “A” donde “T” es dicho tipo concreto, exista. Es en este momento en que se verifica que la clase A<T> para ese tipo T concreto sea válida, es decir, verificar que toda variable de tipo T tenga todos los métodos que se solicitan de ella (si es que se utilizó alguno) y si T es utilizado dentro de métodos que reciben algo distinto de T, que T sea *convertible* a dicho tipo. Por ejemplo, si el método pide un tipo `int` y T es algún otro tipo numérico convertible a `int`, entonces A<T> es válida, pero si T es `std::string`, el código no compilará.

También es posible tener una “especialización” de una clase que tiene un parámetro *template*, esto significa declarar de manera explícita una clase A donde su tipo T es algún tipo concreto como `float`, en tal caso, la especialización tiene prioridad y sobrescribe a la clase genérica en el momento en que es instanciada, descartando cualquier método o miembro que esta tenga a favor de lo que la especialización indique, es decir, si `A<T>` tiene un método `foo`, pero existe una especialización `A<float>`, entonces `A<float>` debe declarar su propia versión del método `foo` y es esa implementación la que se invoca para una variable de tipo `A<float>`.

El uso de *templates* no está limitado a clases completas, sino que también es posible declarar métodos específicos dentro de una clase (e inclusive funciones libres fuera de una clase) con parámetros *template*, los cuales pueden ser distintos de los parámetros *template* de la clase (si es que la clase los posee), y también pueden especializarse cumpliendo una función similar a la sobrecarga de métodos (*method overloading*), en la que se puede tener varias implementaciones del mismo método con distinta cantidad de argumentos o argumentos de distinto tipo. Esto es de particular utilidad al momento de declarar una sola vez funciones que operan con interfaces comunes, pero poseen excepciones para tipos específicos.

Un ejemplo de uso de templates, es una función `max` o `min` que, en vez de recibir 1 o 2 tipos concretos definiendo una sobrecarga para cada par de tipos, le basta con recibir argumentos de tipos *template* T y U que tengan una relación de orden bien definida, usualmente a través de los métodos `operator<()` y `operator>()` que definen el comportamiento de utilizar los símbolos < (menor que) y > (mayor que), con variables de tipos T y U. Si se desea, también es posible declarar una especialización donde el tipo de T y el tipo U son explícitamente iguales, o una especialización donde uno de los tipos es fijo y tiene alguna lógica extra involucrada que solo le concierne a este y no al otro. Por ejemplo, en el caso de una función que determine un “orden” particular sobre strings donde un string “mínimo” se define como aquel que tiene menos caracteres y estos tienen menor valor en un orden lexicográfico.

Si bien los tipos *template*[17, 18] proveen una muy alta flexibilidad, también son muy propensos a errores, ya que una clase declarada con un tipo *template*, no es realmente una clase hasta el momento en que es instanciada (sin contar “especializaciones”, las cuales si son clases concretas), y si alguna condición para que la clase sea válida no se cumple, el mensaje de error del compilador suele ser muy largo y engorroso, y a menudo indicando errores en lugares no relacionados como funciones de la biblioteca estándar.

Para remediar esto, C++20 introduce los *concepts*[15, 16], similar a la función que cumple una *interface* en el lenguaje Java[19] para especificar que una clase debe implementar sus métodos para ser válida. Un *concept* permite indicar una serie de requerimientos o restricciones que un tipo *template* debe cumplir para siquiera ser un candidato a tipo dentro de una clase, y esto no solo está restringido a la implementación de métodos, sino que

también se pueden especificar atributos que el tipo o clase *T* debe tener, ya sean estáticos o no. Esto es de especial utilidad para este trabajo debido a que permite definir un *alias* para otro tipo dentro de *T* cuya existencia está asegurada por el *concept*, añadiendo una capa de abstracción que prescinde de detalles específicos relacionados con tipos concretos que un *T* pueda definir dentro de él, otorgando mayor flexibilidad a que cada *T* opere como desee con sus tipos alias indistintamente de quien sea la clase que utilice a *T* como tipo *template*.

Cuando un tipo *T* está restringido por un *concept* y este no se cumple, el mensaje que brinda el compilador es claro, y especifica qué es lo que no se cumple de manera directa, facilitando la tarea de depurar errores relacionados.

También es posible hacer uso de la instrucción `requires`[15, 16] de los *concept* al momento de declarar métodos dentro de una clase, otorgando la flexibilidad de, por ejemplo, que un método exista solo si el tipo *T* cumple algún *concept* específico proveyendo versiones alternativas si no lo cumple, o cuando cumple con un *concept* distinto. Esto permite un muy alto nivel de eficiencia y abstracción al momento de utilizar los métodos que una clase provee, puesto que sus métodos solo existen cuando la clase instanciada realmente los necesita y con la implementación que corresponda.

2.2. Estado del arte: herramientas para la generación de mallas

Existen muchos algoritmos para generación de mallas poligonales, siendo los más relevantes para este trabajo Triangle [7], Detri2 [2] y Polylla [3].

2.2.1. Triangle

El algoritmo generador de mallas poligonales implementado en Triangle [7] se basa en el algoritmo de Ruppert [20] y consta de 4 etapas, de las cuales las primeras 2 son exactamente las mismas que las de Ruppert. Estas involucran triangular la geometría original (Figura 10), considerando solo los vértices dentro de su cápsula convexa, tal como se muestra en la Figura 11 y posteriormente insertar los segmentos originales de la triangulación como se ve en la Figura 12.

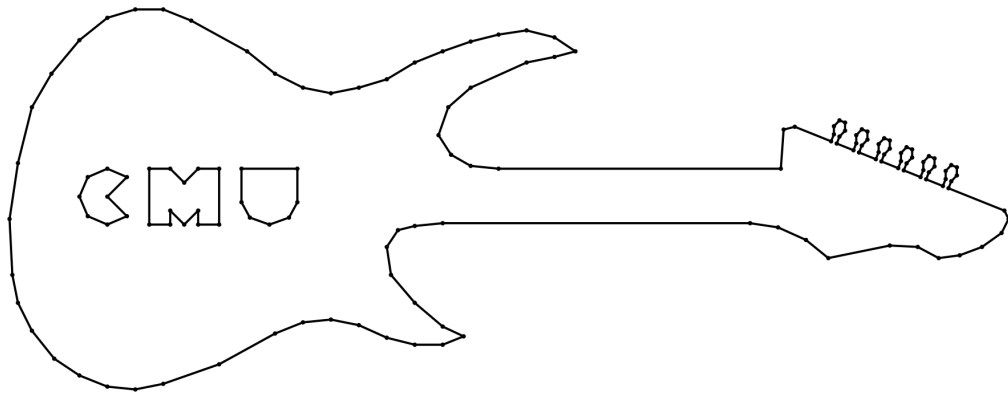


Figura 10: PSLG de entrada una guitarra electrica con un número de vértices y segmentos no definidos como se ilustra en Triangle [7]

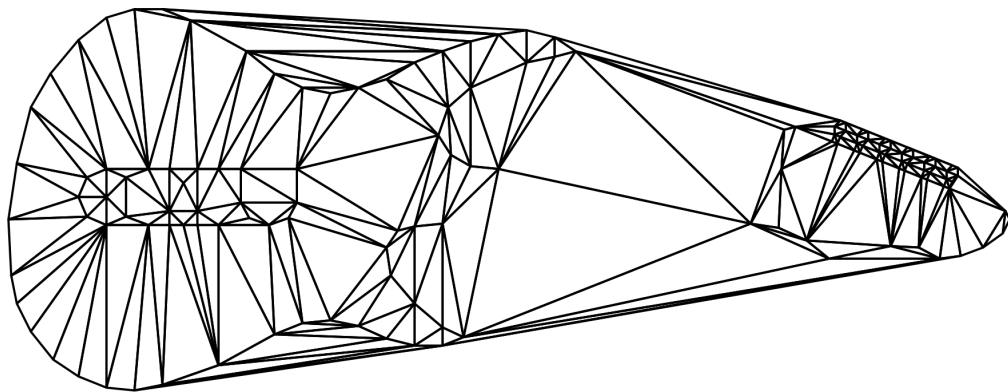


Figura 11: Triangulación de la cerradura convexa del PSLG de la Figura 10 con segmentos originales faltantes [7]

Dado que en la malla de la Figura 11 está incluida la cápsula convexa, esta posee aristas fuera de la geometría original. La segunda etapa consta de introducir los segmentos originales de la geometría; esto se puede hacer de 2 maneras posibles según preferencia del usuario. La primera es insertar un nuevo vértice que corresponda al punto medio de alguno de los segmentos que no aparezcan en la triangulación anterior y usar el algoritmo incremental de Lawson [21] para obtener una nueva triangulación de Delaunay basada en la anterior con el vértice adicional. Esto genera que el segmento original se divida en 2 y la nueva triangulación podría tener el segmento original formado por estos 2 sub segmentos. En caso de que los sub segmentos aún no formen un arco en la triangulación, el proceso de insertar un vértice del punto medio se repite recursivamente para los sub segmentos hasta que el segmento original exista como una secuencia de segmentos lineales. La manera alternativa de insertar los segmentos originales, y la que se utiliza por defecto, es convertir la triangulación a una triangulación de Delaunay restringida, en la cual los segmentos originales deben aparecer. Esto se logra eliminando los triángulos que intersequen el segmento que se desea agregar y luego re triangulando las regiones a cada lado del segmento insertado, resultando en la Figura 12.

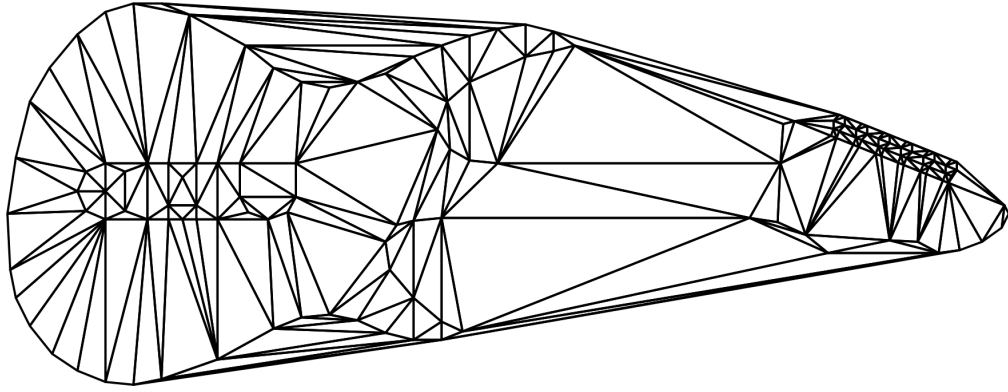


Figura 12: Triangulación restringida del PSLG de la Figura 10 [7]

La tercera etapa difiere del algoritmo de Ruppert y consiste en remover los triángulos extra que están fuera del borde definido por la geometría original, como aquellos presentes en zonas que originalmente eran no convexas, a las cuales Shewchuk llama concavidades, y los presentes en agujeros (como los del interior del cuerpo de la guitarra en el ejemplo). Esto se ilustra en la Figura 13.

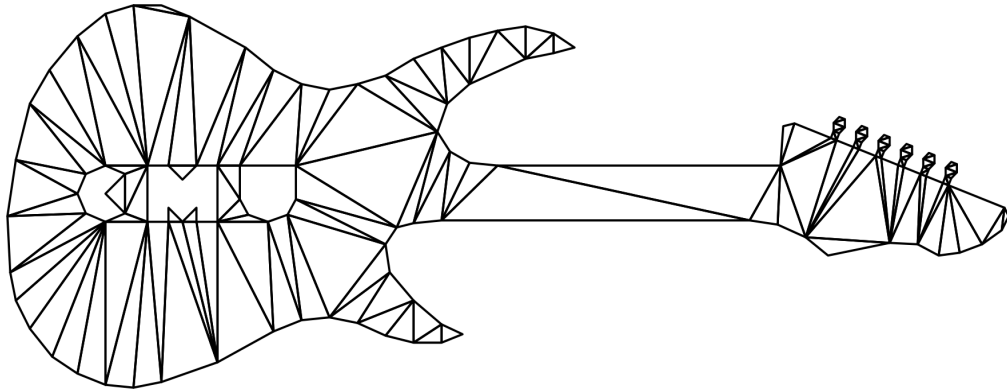


Figura 13: Triangulación restringida del PSLG de la Figura 10 con triángulos extra removidos [7]

La última etapa del algoritmo consiste en el refinamiento de la malla insertando vértices y re triangulando con el algoritmo incremental de Lawson [21] hasta que las restricciones de ángulo mínimo y área máxima de triángulo definidas por el usuario se cumplan. Esta inserción se hace siguiendo 2 reglas, la regla de *segmentos encerrados* y la de los *triángulos malos* dándole siempre prioridad a la primera:

- El *círculo diametral* de un segmento es el círculo único más pequeño que contiene el segmento como su diámetro. Un segmento se dice que está *encerrado* si un punto que no es un extremo del segmento está dentro de su círculo diametral. Cualquier segmento encerrado que aparezca se separa insertando un vértice en su punto medio. Los dos sub segmentos resultantes tienen círculos diametrales más pequeños y podrían estar o no estar encerrados. El proceso se repite hasta que no queden segmentos encerrados como se ve en la Figura 14.

- Un triángulo se dice que es *malo* dependiendo de algún criterio, por ejemplo si tiene un ángulo que es muy pequeño o un área que es muy grande para satisfacer las restricciones impuestas por el usuario. Un triángulo malo se destruye insertando un vértice en su circuncentro. Está asegurado que el triángulo malo será eliminado como se ve en la Figura 15 para mantener la propiedad de Delaunay. Si el vértice insertado encierra un segmento (como se define en la regla del círculo diametral), este será removido deshaciendo la inserción y los segmentos que encerraba se separarán según la regla del círculo diametral.

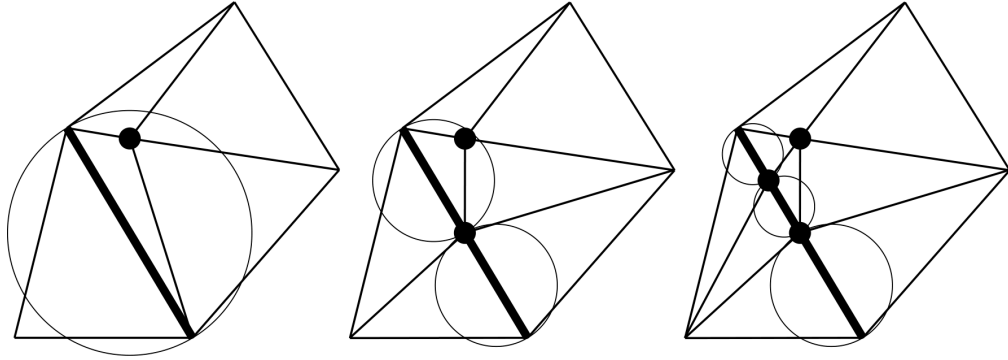


Figura 14: Segmentos divididos según su círculo diametral [7]

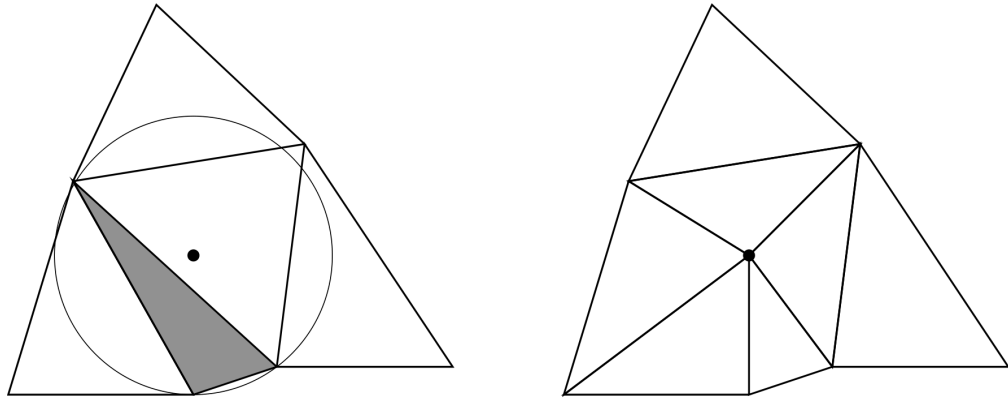


Figura 15: Triángulo separado según su circuncentro [7]

Para el ejemplo de la Figura 10, la malla resultante generada por Triangle es la que se ilustra en la Figura 16.

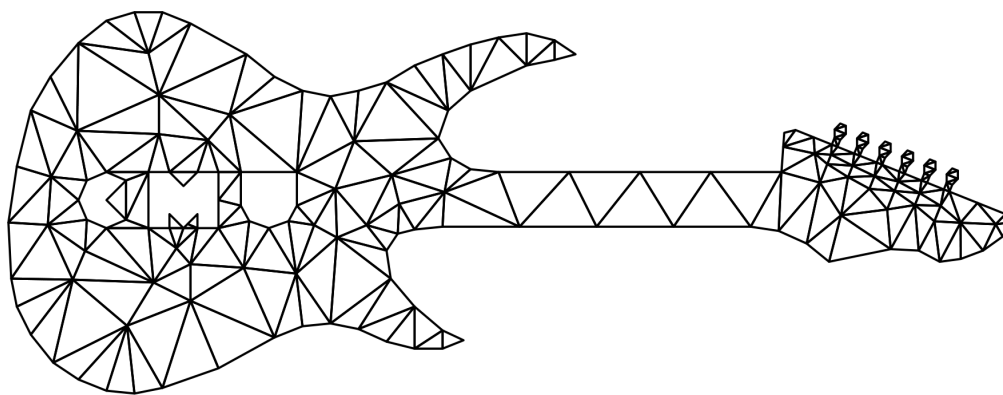


Figura 16: Malla poligonal final resultante de aplicar el algoritmo Triangle [7]

Esta última parte del algoritmo es de particular importancia, ya que el criterio del circuncírculo es análogo al de la cavidad, sin embargo, en este caso solo se utiliza para refinar la malla y retriangular las cavidades. Este trabajo de memoria busca explorar más a fondo este proceso y utilizar las cavidades para generar mallas de polígonos generales.

El algoritmo Triangle posee una implementación escrita en lenguaje C[8] de la cual se adaptó código, principalmente para detectar si un punto está contenido en un círculo.

2.2.2. Detri2: generador y visualizador de triangulaciones

El software Detri2[2] permite generar triangulaciones a partir de nubes de vértices aleatorios generados dentro de la misma aplicación, o a través de archivos `.node` (Anexo B), y utilizar distintos criterios para refinar la triangulación a través de una interfaz gráfica que permite una gran variedad de opciones y resulta muy útil para generar o verificar geometrías. Además de la triangulación de un conjunto de puntos, Detri2 también permite visualizar su diagrama de Voronoi. En la Figura 6 mostrada anteriormente y la Figura 17 se puede ver una triangulación de Delaunay, su diagrama de Voronoi equivalente, y ambos superpuestos. La Figura 17 muestra un ejemplo hecho en Detri2.

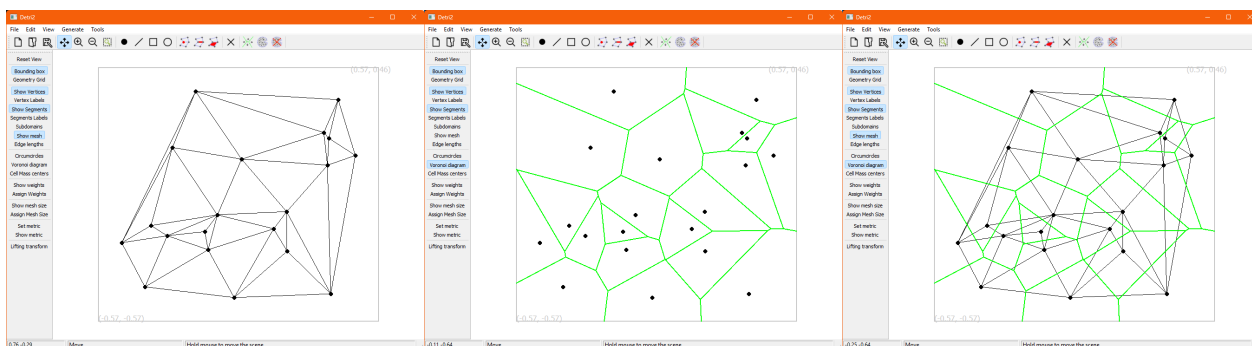


Figura 17: Triangulación de Delaunay y su Diagrama de Voronoi dual en el software Detri2

2.2.3. Polylla

Por otro lado, el algoritmo Polylla, busca generar una malla poligonal a partir de una triangulación arbitraria, usando lo que denomina como *Terminal-edge regions* o regiones de

arista terminal, definidas según el *longest edge propagation path* (camino de propagación de arista más larga o *Lepp* [22]) de los triángulos, las cuales utiliza para generar una partición de la triangulación que se asemeja a un diagrama de Voronoi [23].

El *Lepp* o camino de propagación de arista más larga de un triángulo se define de la siguiente manera: Por cada triángulo t_i en cualquier triangulación Ω , el $Lepp(t_i)$ es la lista ordenada de todos los triángulos $t_0, t_1, t_2, \dots, t_{l-1}, t_l$ con $l \in \mathbb{N}$, tal que t_i es el triángulo vecino de t_{i-1} a través de la arista más larga de t_{i-1} , para $i = 1, 2, \dots, l$. Si una arista más larga es compartida por t_{l-1} y t_l esta se define como una arista terminal donde termina el Lepp y t_{l-1} y t_l son triángulos terminales. Una región de arista terminal se define como la unión de los triángulos t tal que $Lepp(t)$ termina en la misma arista terminal. En la Figura 18 se puede ver un ejemplo de región de arista terminal.

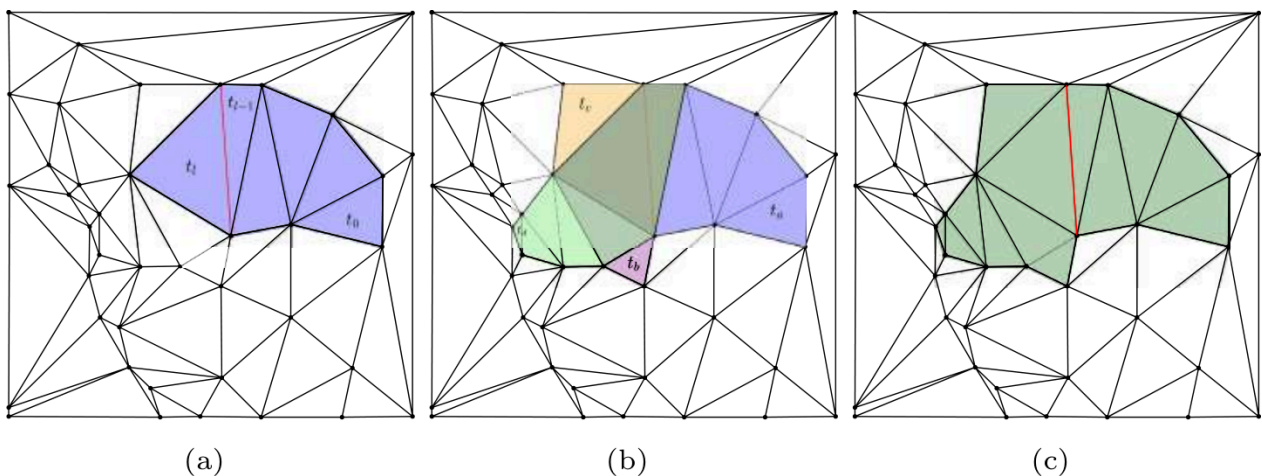


Figura 18: Región de arista terminal. a) $Lepp(t_0)$ donde la arista roja es la arista terminal. b) Cuatro Lepps con la misma arista terminal: $Lepp(t_a)$, $Lepp(t_b)$, $Lepp(t_c)$, $Lepp(t_d)$. c) Región de arista terminal generada por la unión de los Lepp de b) [3]

Además de las aristas terminales, Polylla [3] define los siguientes tipos de aristas. Dada una arista e y dos triángulos t_1 y t_2 que comparten e :

- *Frontier-edge* o Arista frontera: e no es la arista más larga ni de t_1 ni de t_2 .
- *Internal-edge* o Arista interna: e es la arista más larga de t_1 , pero no de t_2 o viceversa.
- *Boundary edge* o Arista de borde: e pertenece a un solo triángulo. Se manejan como aristas frontera.
- *Barrier edge* o Arista barrera: Arista frontera que queda adentro de una región terminal (y no es el borde).

Polylla consiste de tres fases: Primero etiqueta las aristas de la triangulación de entrada según las categorías anteriores para formar regiones terminales y además designa un *triángulo semilla* en cada región de arista terminal para construir las regiones. Luego, a partir de cada triángulo semilla, hace un recorrido en sentido antihorario o *counter clockwise* (CCW en inglés) de la región de arista terminal para encontrar aristas frontera las cuales formaran

la región. Algunas regiones pueden terminar como polígonos no simples (Sección 2.1.1), por lo que se hace una fase de reparación donde las aristas barrera se particionan en polígonos simples, formando ciclos cerrados que las contengan. En la Figura 19 se puede ver una partición de un polígono generada por regiones de arista terminal que presenta una región con un polígono no simple en verde.

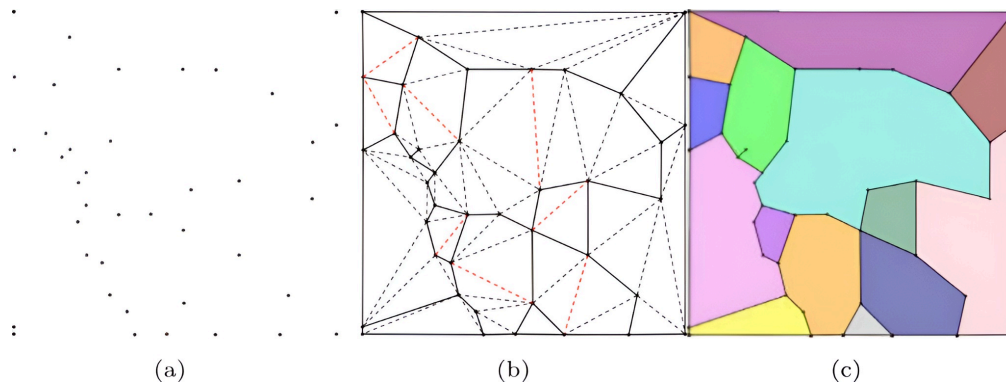


Figura 19: a) Colección de vértices aleatorios, b) Triangulación de Delaunay donde las líneas sólidas son aristas frontera, las líneas punteadas negras son aristas internas y las aristas punteadas rojas son aristas terminales. c) Partición a partir de regiones de arista terminal [3]

En la Figura 20 se puede ver una triangulación de Delaunay y la malla generada por Polylla a partir de ella.

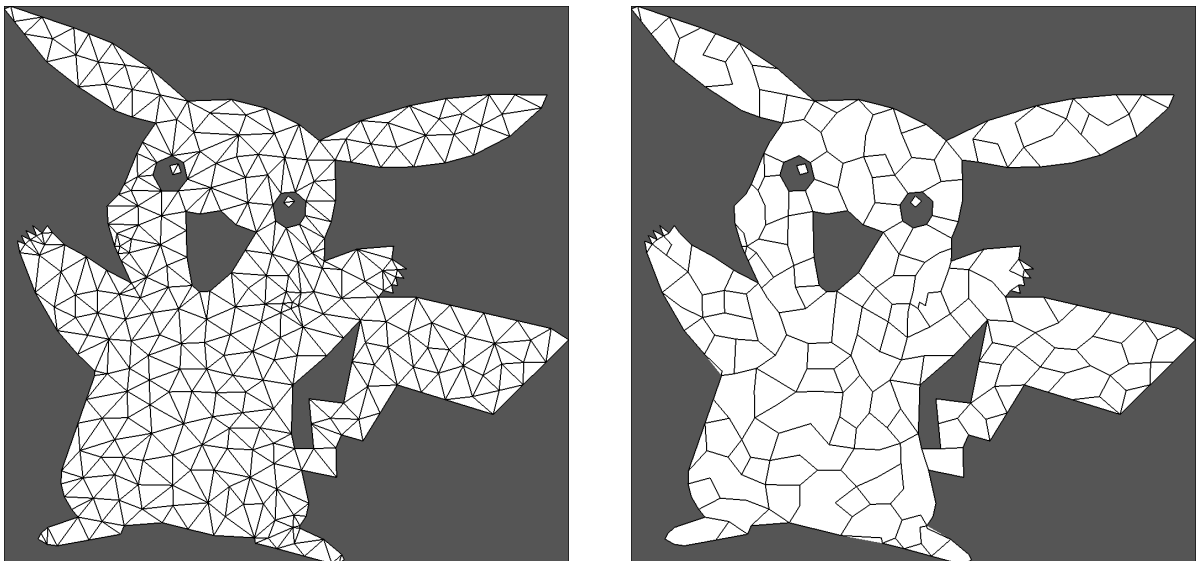


Figura 20: Triangulación de Delaunay y su malla Polylla respectiva [1]

El algoritmo Polylla destaca por sobre otros algoritmos debido a su gran simplicidad y eficiencia en comparación a algoritmos convencionales de construcción de diagramas de Voronoi restringidos, ya que toma bastante menos tiempo en construir con una cantidad de polígonos 3 veces menor y manteniendo la misma cantidad de vértices que el diagrama de

Voronoi hecho a partir de la misma triangulación[3]. A esto se le suma también la utilidad de las mallas Polylla en simulaciones de diversos fenómenos que hacen uso del VEM[10] mencionado anteriormente para encontrar una solución numérica.

En este trabajo se hará uso de la implementación del algoritmo Polylla presente en el repositorio *Polylla-Mesh-DCEL*[1], el cual hace uso de la estructura *half-edge*.

El algoritmo de construcción de mallas basado en cavidades se muestra como una alternativa a Polylla y mallas basadas en el diagrama de Voronoi que permite generar mallas de polígonos arbitrarios, las cuales pueden ser utilizadas para el VEM[10].

2.2.4. CGAL

La librería CGAL[24] (*Computational Geometry Algorithms Library*) es un proyecto de código abierto escrito en C++ que aloja una vasta colección de algoritmos geométricos y estructuras de datos eficientes y robustos. Su propósito es facilitar tareas geométricas complejas que aparecen en dominios tan variados como sistemas de información geográfica, diseño asistido por computador, biología molecular, imágenes médicas, gráficos por computador o robótica.

Dentro de sus módulos más utilizados destacan los dedicados a la generación de mallas y a las estructuras basadas en subdivisiones del plano. CGAL provee triangulaciones de Delaunay en 2D completamente funcionales, incluyendo variantes con restricciones y mecanismos de refinamiento, que facilitan producir mallas de alta calidad. A partir de estas mismas triangulaciones, la librería permite obtener de forma directa el diagrama de Voronoi correspondiente, aprovechando la relación dual entre ambas estructuras. Estas capacidades hacen de CGAL una base sólida para desarrollar, comparar o extender nuevos algoritmos de generación y procesamiento de mallas.

Si bien, en este trabajo no se hace uso de CGAL directamente debido a su complejidad y generalidad, parte del código original de Polylla está inspirado en ella, en particular, para el manejo de conectividad de *half-edges*. Dado que CGAL es una librería general enorme, su uso no aprovecharía una cantidad considerable de sus utilidades. Prescindir de CGAL también permite mayor control sobre la implementación.

Capítulo 3

Problema

Como se mencionó en la introducción, este trabajo de memoria busca implementar de manera eficiente un algoritmo que permita generar mallas poligonales basándose en el concepto de cavidad (véase Sección 2.1.3, Figura 1, Figura 2 y Figura 3), usando como inspiración las estructuras de datos presentes en Polylla[3], comparando las mallas resultantes de los dos algoritmos en términos de calidad, uso de memoria, aptitud para el VEM[10], entre otros. Adicionalmente, también se busca reescribir la implementación actual de Polylla basada en *half-edges*[1] (*Polylla-Mesh-DCEL*) para integrarla en un programa modular que pueda procesar triangulaciones de Delaunay, ya sea haciendo uso de Polylla, del algoritmo basado en cavidades o algún otro que se añada en el futuro.

A continuación se describirá a modo general como funciona esta implementación particular de Polylla y por qué existe la necesidad de reescribirla para integrarla en un programa más general.

3.1. Polylla-Mesh-DCEL[1]

Esta implementación de Polylla (Sección 2.2.3) escrita en C++, a diferencia de la implementación original basada en caras[3], usa *half-edges* (Sección 2.1.6). Polylla-Mesh-DCEL hace uso de 2 estructuras de datos y 2 clases esenciales, siendo estas: `vertex`, `halfEdge`, `Triangulation` y `Polylla`. Estas son utilizadas dentro de una función `main` que recibe argumentos posicionales con distinto significado según su cantidad, para procesar archivos o determinar una ruta de salida.

A continuación se presentan figuras mostrando partes del diagrama UML de clases de la implementación presente en el Anexo E. La iconografía de estas figuras y otras representaciones UML está disponible en el Anexo D.

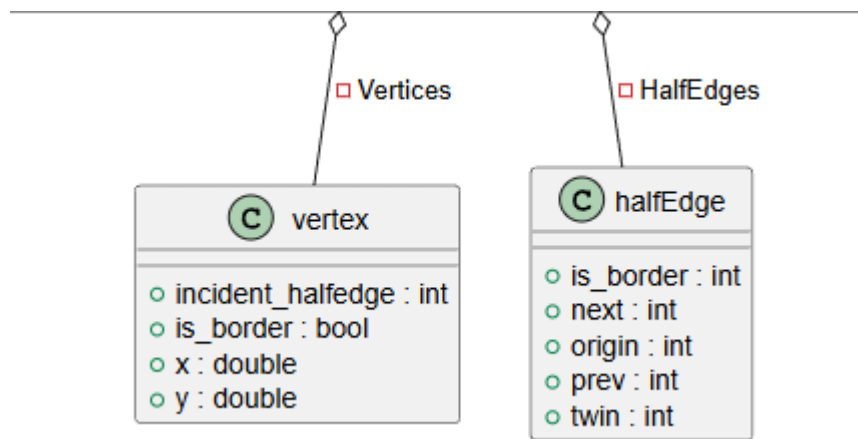


Figura 21: Representación UML de estructuras `vertex` y `halfEdge` de Polylla-Mesh-DCEL

La estructura `vertex` describe un punto, o vértice, de la malla conteniendo sus coordenadas x e y . También posee un booleano que indica si el vértice es parte del borde de la malla y un número entero que representa un índice hacia algún `halfEdge` que tiene a este vértice como origen dentro de un objeto de la clase `Triangulation` en un arreglo de tamaño dinámico (implementado con un objeto de clase `vector` de C++).

La estructura `halfEdge` contiene toda la información que compone a un *half-edge* como se mencionó en la Sección 2.1.6 (salvo por *target* que es implícito). Todos estos atributos son índices a vectores dentro de la clase `Triangulation` resumida a continuación.

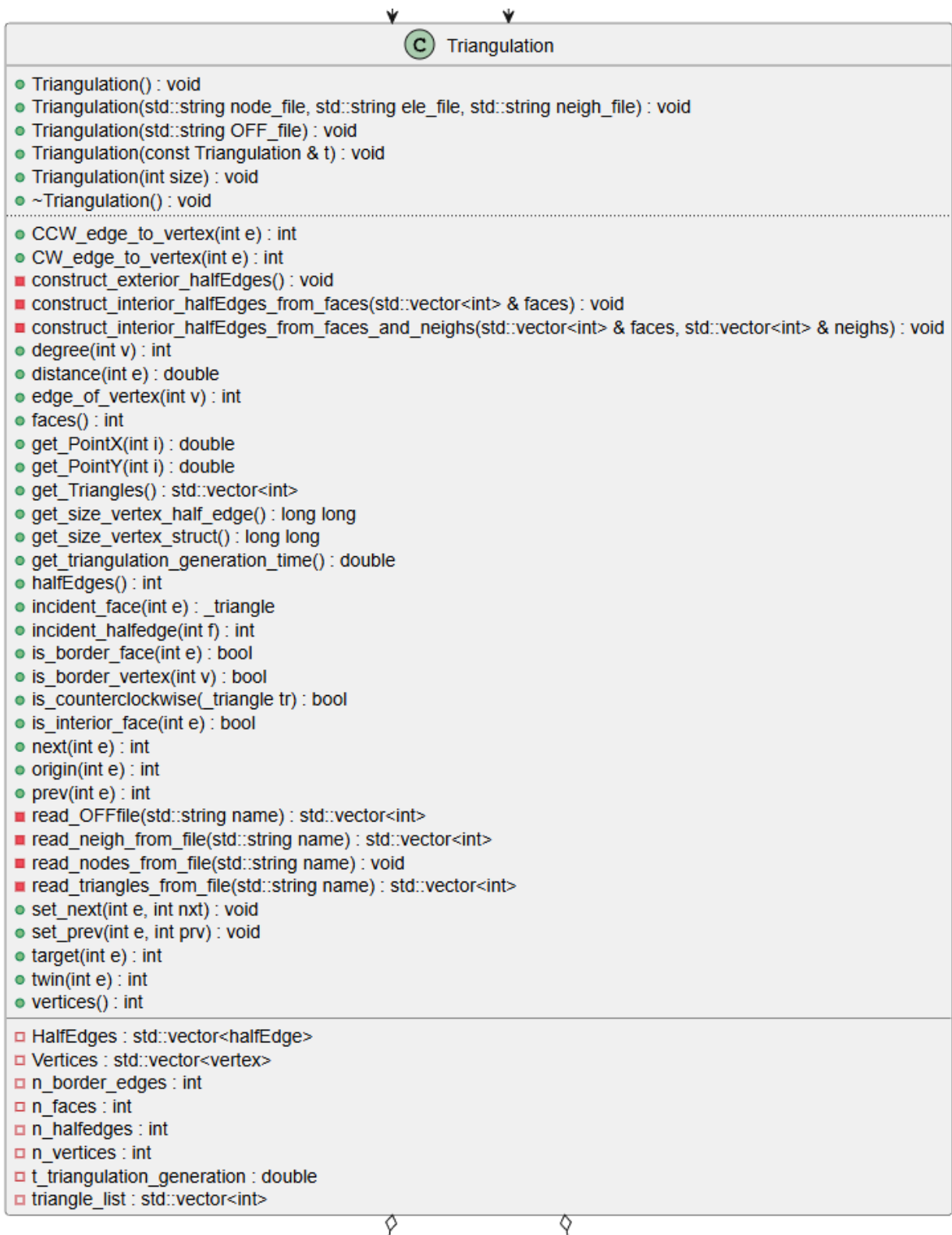


Figura 22: Representación UML de la clase `Triangulation` de Polylla-Mesh-DCEL[1]

La clase `Triangulation`, posee 2 miembros de tipo `vector` que contienen objetos `vertex` y objetos `halfEdge` respectivamente. Además, define los métodos necesarios para recorrer la malla haciendo uso de los índices definidos en `vertex` y `halfEdge`. Estos incluyen todas las

operaciones definidas en la Sección 2.1.6 como *next*, *prev*, *origin*, *target* y *twin*. Notar que no hay un atributo *target* en la estructura `halfEdge`, ya que este está guardado de forma implícita como el atributo `origin` del `halfEdge` apuntado por `twin`.

Uno de los constructores de la clase `Triangulation` lee archivos de texto con información geométrica para generar la malla, en particular, aquellos generados por Triangle[7], los cuales consisten en:

- `.node`: Contiene todos los vértices de la malla junto con sus coordenadas.
- `.ele` : Contiene todos los triángulos presentes en la malla como listas de vértices (representación basada en caras).
- `.neigh`: Contiene información sobre los vecinos de los triángulos. Esta es de gran utilidad al momento de construir la malla basada en *half-edges*.

También tiene un constructor para leer archivos en formato `.off`, extensamente utilizado en aplicaciones de computación gráfica, que también guarda una representación basada en caras.

La clase `Triangulation` es extensamente utilizada dentro de la clase `Polylla`:

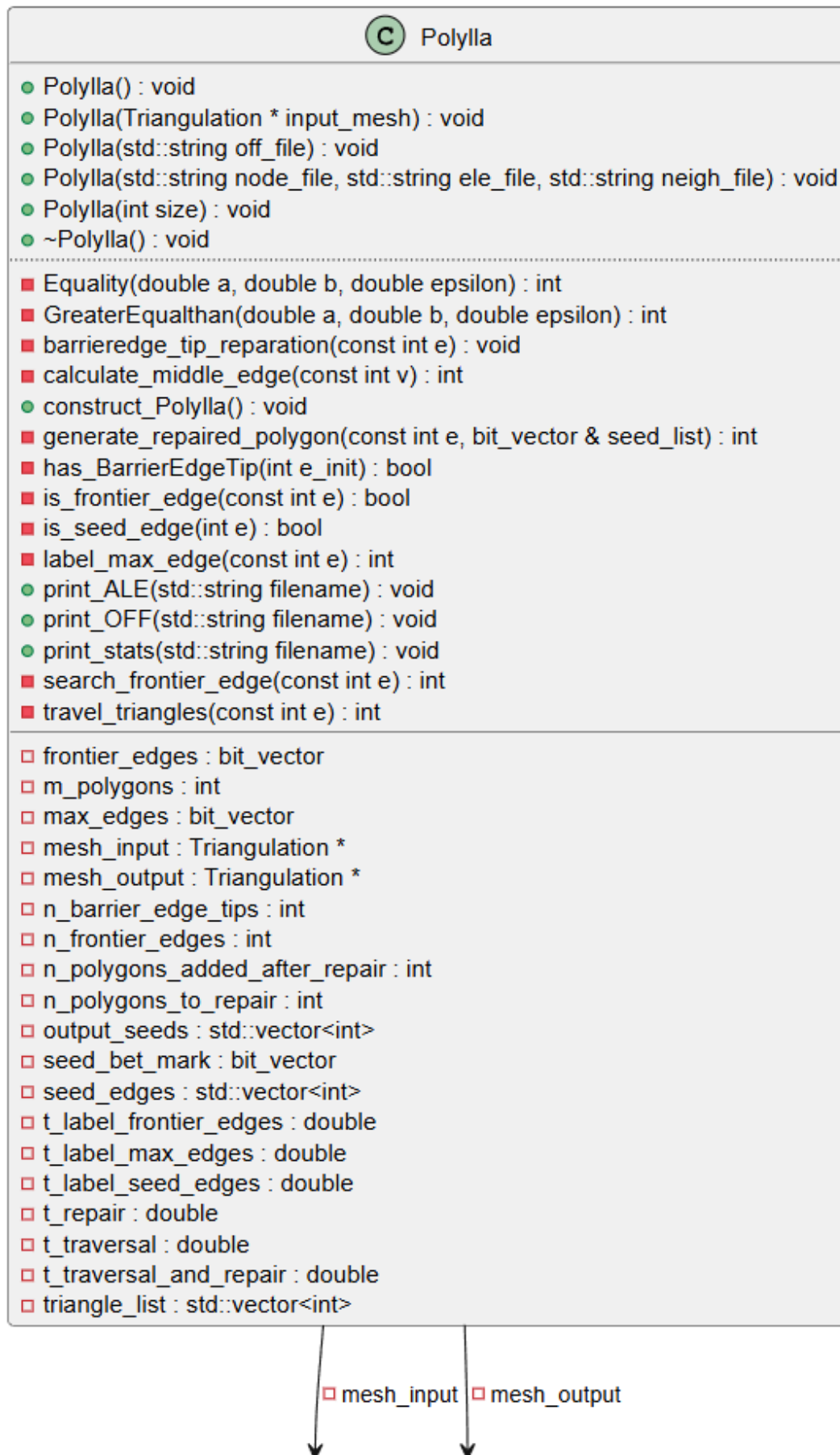


Figura 23: Representación UML de la clase Polylla de Polylla-Mesh-DCEL[1]

La clase `Polylla` implementa en su totalidad el algoritmo descrito en la Sección 2.2.3. Esta recibe como argumento en su constructor un objeto de la clase `Triangulation` o los argumentos necesarios para generar uno. Posterior a ello llama al método `construct_Polylla()` que realiza todas las etapas del algoritmo para generar la malla: etiquetado de aristas máximas (vector `max_edges`) y aristas frontera (vector `frontier_edges`) para posteriormente etiquetar aristas semilla (vector `seed_edges`) para formar las regiones terminales, recorrer dichas regiones terminales para corroborar si forman un polígono simple y si no, repararlo.

Si bien las clases de Polylla-Mesh-DCEL cumplen adecuadamente la implementación del algoritmo Polylla[3], estas poseen diversos problemas de diseño que las hacen difíciles de extender para otros usos y enormemente complejas de entender sin un estudio profundo del código debido al uso excesivo de tipos primitivos como `int`, que si bien son esencialmente índices, varios métodos reciben como argumento una variable de tipo `int`, pero el tipo de índice al que esta variable se refiere depende del método y la única forma de saber a cuál corresponde es tener conocimiento de como funciona la estructura *half-edge* y nombres de variables muy poco descriptivos tales como *e* o *v*. También su documentación es escasa y a veces poco clara. Esto hace muy propenso a errores cualquier modificación que se le realice al código.

Además, estas clases cumplen demasiadas funciones y están altamente restringidas a la configuración actual, principalmente:

- La clase `Triangulation` depende directamente de archivos con formatos específicos y opera con ellos cuando esto no debería ser su responsabilidad.
- La función de procesamiento de archivos está repartida entre `Triangulation` y `Polylla` de manera independiente cuando esto podría ser manejado por otras clases.
- Las clases `vertex` y `halfEdge` no están definidas en su propio archivo, sino que están definidas en el mismo *header* que la clase `Triangulation` (`triangulation.hpp[1]`).
- Hay una dependencia fuerte entre `Polylla` y `Triangulation`, cuando `Polylla` podría depender de una interfaz que implemente los métodos de `Triangulation`.

Esta implementación de Polylla fue hecha considerando la mayor eficiencia posible, pero es extremadamente rígida, por lo que se propone el siguiente esquema.

3.2. Diseño propuesto

Para reescribir Polylla-Mesh-DCEL brindándole más modularidad se propone un diseño basado en clases altamente genéricas utilizando tipos *template* con sus *concepts* asociados (véase Sección 2.1.7), disponible en <https://github.com/Tchy258/Delaunay-cavity>

La clase principal de este diseño es la clase “PolygonalMesh”, la cual hace uso extensivo del patrón de diseño *Strategy*[25] delegando las funciones de leer y escribir archivos geométricos, contener una malla y generar la malla a clases dedicadas, siendo el tipo de la malla un tipo *template* restringido por un *concept* utilizado como parámetro por todas ellas. Esta clase se muestra en la Figura 24. El diagrama completo está disponible en el Anexo E.

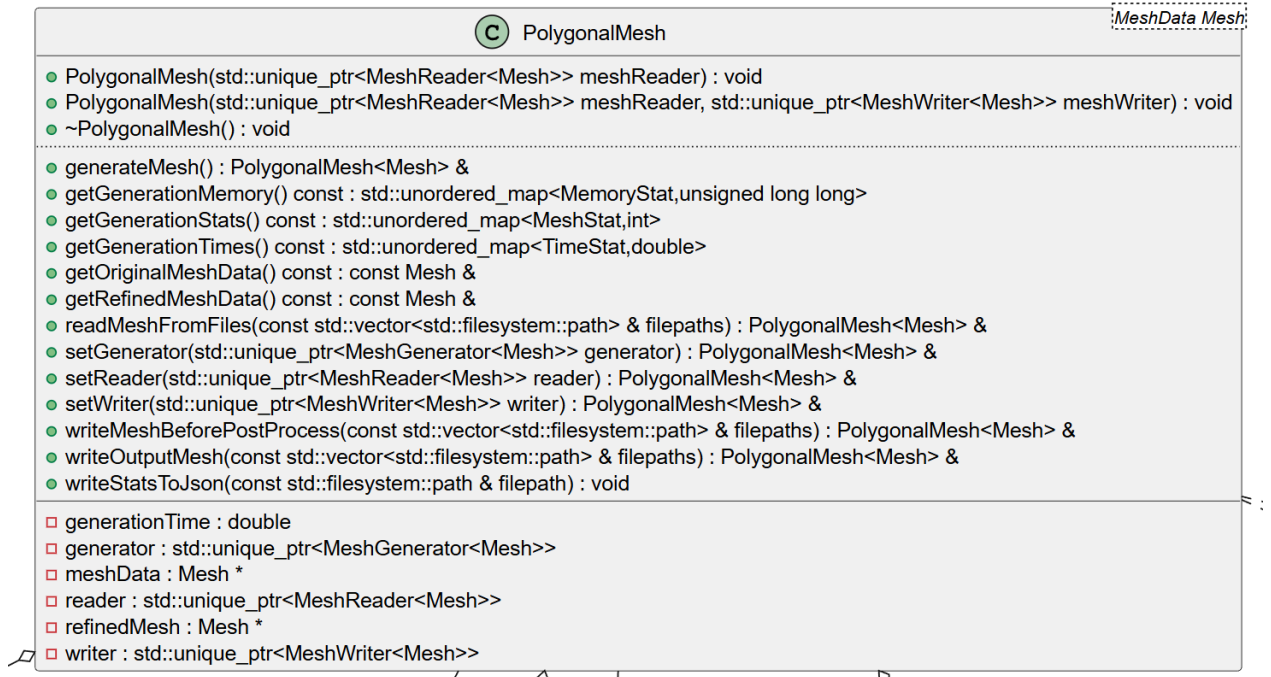


Figura 24: Representación UML de la clase **PolygonalMesh**

De manera similar a como la clase **Polylla** mantenía dos punteros a objetos **Triangulation**, la clase **PolygonalMesh** mantiene punteros a variables de tipo **Mesh**, el cual es un parámetro *template* de la clase, restringido por el *concept* **MeshData**. Esto desacopla el detalle de qué tipo de malla específica se desea utilizar sin el costo adicional que involucra tener una clase virtual de C++[26] independientemente de las optimizaciones que el compilador pueda hacer al respecto[27].

El *concept* **MeshData** a su vez está compuesto de 8 otros *concepts* específicos, cada uno definiendo distintas características que un tipo genérico debe respetar para ser considerado viable como una malla:

- **MeshAccessors**: Define *getters* que toda malla debiese tener, incluyendo *getters* para vértices, aristas, polígonos y datos relacionados como la cantidad de cada uno o la cantidad de aristas de un polígono.
- **MeshEdges** y **MeshVertices**: Declaran que toda malla debe definir un *alias* con la instrucción *using*[28] para sus vértices y aristas. Esto asegura correctitud al momento de llamar métodos de una malla que operan o retornan con vértices o aristas de la misma, puesto que gracias al alias, en tiempo de compilación es inequívoco con qué tipo de vértice o arista se está trabajando.
- **MeshIndices**: Declaran que toda malla debe definir un *alias* con la instrucción *using*[28] para “tipos índice”, esto con la finalidad de diferenciar cuándo se está trabajando con un índice que representa un vértice, una arista, una cara o “un índice de salida”, donde este último debe ser alguno de los anteriores. Este atributo es un detalle de la implementación concreta, ya que una “salida” es un índice de cara para una representación basada en caras, pero un índice de arista para una representación basada en *half-edges*. En la

práctica, todos estos índices pueden perfectamente ser `int` (y de hecho lo son) o cualquier otro tipo de entero que se pueda usar para indexar (haciendo uso de un *concept* auxiliar llamado `PrimitiveIntegral` que se asegura de ello), la ventaja, es que al momento de escribir o usar código que opere o retorne dichos índices, es evidente a qué se refieren, porque en vez de estar declarados como simplemente `int`, están declarados como el tipo de índice específico que la malla declara dentro de sí para el método, como `VertexIndex`, `EdgeIndex`, `FaceIndex` u `OutputIndex`. Adicionalmente, este *concept* también declara la existencia de un miembro estático constante llamado `invalidIndexValue`, este símbolo es de particular utilidad para invalidar índices de salida de forma clara, explícita y centralizada en un solo lugar en vez de usar un valor literal como `-1` en múltiples lugares para ese propósito.

- **MeshSetters:** Declara la existencia de *setters* generales para mutar el estado o características de la malla, incluyendo setters para el número efectivo de vértices, polígonos y aristas, junto con métodos para unir dos polígonos, deshacer una unión y un tipo interno declarado por la malla con un “respaldo de conectividad” (`ConnectivityBackupT`) para deshacer dicha unión de ser necesario, por ejemplo, si la unión entre dos polígonos no da un resultado deseado.
- **MeshTopology:** Declara métodos genéricos para consultar información topológica sobre la malla, por ejemplo: quienes son los vecinos de un polígono dado, cuáles son los vértices o aristas de un triángulo dado (asumiendo que sigue siendo triangular), si es que una arista es parte del borde de la malla, la arista o aristas compartidas entre 2 triángulos o 2 polígonos respectivamente, el largo al cuadrado de una arista, si es un polígono es convexo y por último si un polígono es simple (Sección 2.1.1).
- **MeshConstructible:** Declara constructores que una malla debe tener junto con sus argumentos, en particular, una malla debe ser capaz de recibir un **vector** de vértices, un **vector** de aristas y un **vector** de índices de cara como mínimo para ser válida. También debe tener un constructor de copia. Estos vértices, aristas e índices de cara son un detalle de la malla misma, definidos según los *concepts* anteriores. Este constructor se define con estos argumentos con la idea de que los datos de la malla vienen en una representación basada en caras, permitiendo que la implementación de malla particular reorganice estos datos como estime conveniente. A futuro se puede extender el *concept* para exigir constructores a partir de distintas representaciones.
- **MeshMemory:** *Concept* de utilidad que declara métodos para calcular el uso de memoria de una malla, usado para medir rendimiento.

Como se puede notar, estos *concepts* se basan muy fuertemente en la definición de malla que brindaba la clase `Triangulation`, pero delegan la tarea de leer los vértices hacia otra clase, eliminando la restricción de solo poder leer mallas desde archivos. La clase que se adhiere a estos *concepts* y es un reemplazo directo a `Triangulation` es la clase `HalfEdgeMesh` de la Figura 25.



Figura 25: Representación UML de la clase `HalfEdgeMesh`

Esta clase carece del constructor de `Triangulation` que recibía un número entero, puesto que su única función era generar una malla compuesta por una grilla cuadrada, la cual perfectamente puede ser provista desde fuera.

Esta clase implementa la estructura *half-edge* haciendo uso de los *structs* `HEVertex` y `HalfEdge` de la Figura 26, adaptados de Polylla-Mesh-DCEL. Notar que en C++, la única diferencia entre *struct* y *class* es que la visibilidad por defecto es distinta, siendo `public` en el primero y `private` en el segundo, pero en realidad ambos son clases capaces de definir atributos y métodos.

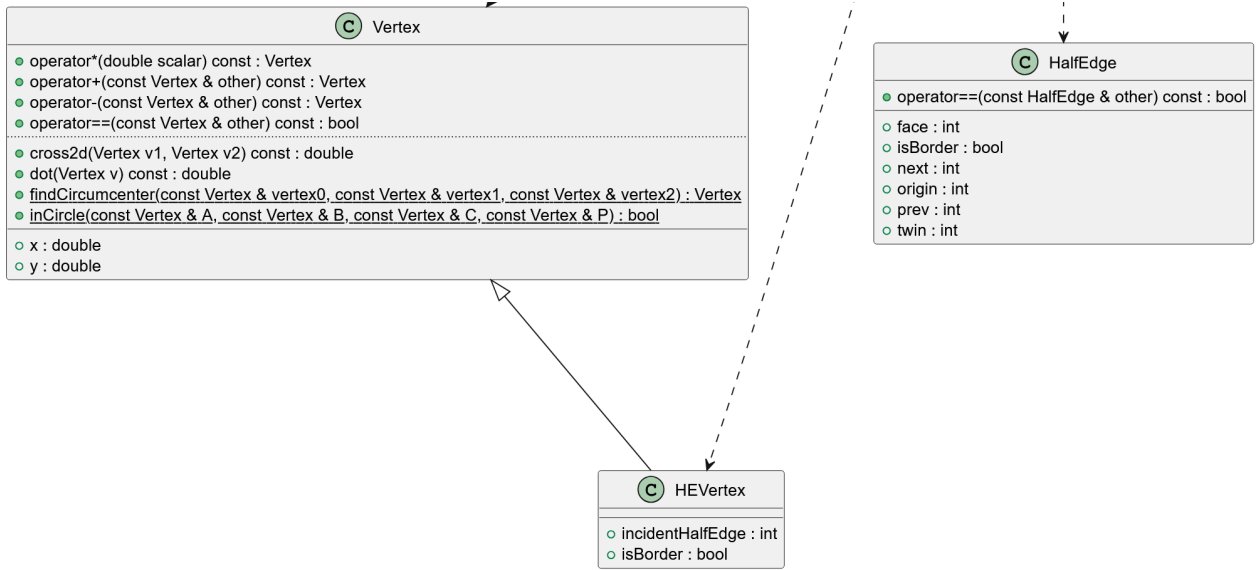


Figura 26: Representación UML de las clases `Vertex`, `HEVertex` y `HalfEdge`

Además de los atributos que poseían los *struct* `vertex` y `halfEdge` de Polylla-Mesh-DCEL, se hace una distinción entre un vértice genérico (`Vertex`) y un vértice especializado para *half-edges* (`HEVertex`), puesto que en la mayoría de los casos es suficiente operar con un vértice como si tuviese tipo `Vertex` con las operaciones que este define, son de particular utilidad sus métodos públicos:

- **operator*, +, -, ==:** Azúcar sintáctica que permite escribir en código operaciones como $v_1 + v_2$ que suma cada coordenada por separado, o $v_1 * s$ que multiplica las coordenadas de un punto v_1 por un valor escalar s , como se esperaría al operar con puntos en un plano en dos dimensiones, en vez de llamar métodos como `.plus` o `.multiply`.
- **cross2d:** Dados 3 vértices v_1 , v_2 , y v_3 , `v1.cross2d(v2,v3)` retorna el valor de la coordenada z al hacer un producto cruz entre los vectores formados por $v_2 - v_1$ y $v_3 - v_1$. Esta operación permite determinar la orientación en la que se encuentran los vértices, horario o antihorario, según su signo, donde un valor positivo representa una orientación en sentido antihorario, y un valor negativo, una orientación en sentido horario. Este valor también equivale a la mitad del área de un triángulo formado por estos 3 vértices, lo cual se puede extender para calcular el área de cualquier polígono arbitrario.
- **dot:** Operación de producto punto entre 2 vectores, útil al calcular el largo de una arista entre vértices v_1 y v_2 , ya que el producto punto de un vector consigo mismo equivale al cuadrado de su norma euclidiana. En este caso el “vector” es el `vertex` resultante de la resta entre v_1 y v_2 . Si bien es cierto que esta operación no es realmente una operación entre vertices, la resta entre ellos se puede interpretar como un vector, y un vértice por si solo se puede considerar como un vector que parte en el origen. La equivalencia del producto punto de un vector consigo mismo con el cuadrado de la norma euclidiana viene del hecho de que el producto punto se define como:

$$A \cdot B = |A| \times |B| \cos(\theta)$$

En este caso $A = B$ y $\theta = 0 \Rightarrow A \cdot A = |A|^2$

- **findCircumcenter** e **inCircle**: Adaptando la lógica de Triangle[8], estos métodos permiten encontrar el circuncírculo de un triángulo y determinar si un punto P está en el interior del círculo descrito por los puntos A , B y C usando un método basado en determinantes[29]. Estos métodos son cruciales para la formación de cavidades.

Continuando con otros miembros de la clase `PolygonalMesh` de la Figura 24, se tienen variables con clase `std::unique_ptr<MeshReader>` y `std::unique_ptr<MeshWriter>`, estos objetos son los llamados *smart pointers*[18] de C++, los cuales, a diferencia de punteros estándar, saben como manejar su memoria en el *heap*, con un costo leve de rendimiento. Para estos dos miembros se prefiere el uso de *smart pointers* porque, como mucho, se necesitan una sola vez cada uno para leer y escribir la malla a archivos, siendo despreciable el costo de rendimiento asociado comparado a la función que estos objetos realizan (entrada y salida de archivos). Los objetos de tipo `Mesh`, en cambio, son usados múltiples veces a lo largo de distintas de clases, por lo que convertirlos en *smart pointers* resultaría en una disminución considerable de rendimiento.

`MeshReader` y `MeshWriter`, son clases virtuales (o abstractas) que definen la interfaz común que una clase que lee y escribe de mallas debe definir respectivamente. Estos miembros de `PolygonalMesh` deben ser virtuales y no *templates*, ya que le permiten polimorfismo de clases en tiempo de ejecución (recordar que los tipos *template* son fijos una vez declarados), otorgándole la capacidad de leer y escribir mallas en distintos formatos de archivo en una misma ejecución del programa si así se quisiera. Estas clases se pueden ver en la Figura 27 y la Figura 28.

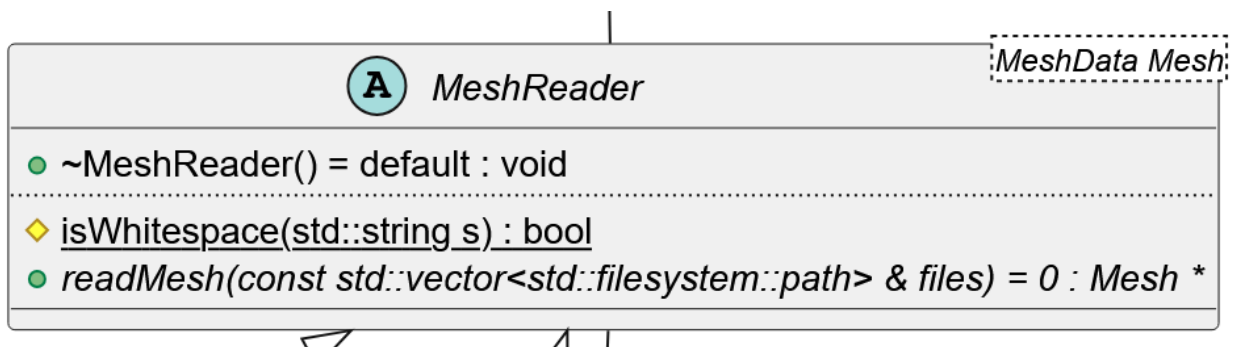


Figura 27: Representación UML de la clase abstracta `MeshReader`

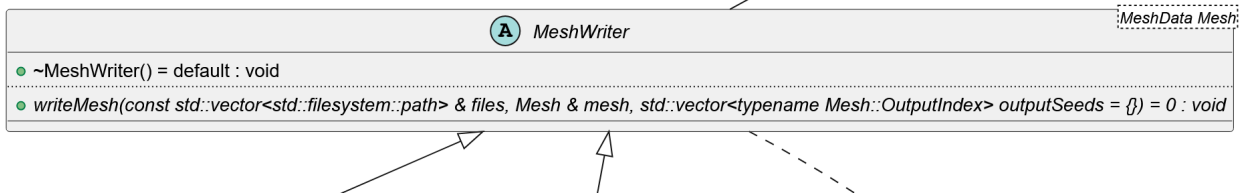


Figura 28: Representación UML de la clase abstracta MeshWriter

A diferencia de la clase `Triangulation` que recibe objetos de tipo `string` representando nombres de archivo, las clases que extienden `MeshReader` y `MeshWriter` utilizan un `vector` de objetos tipo `filesystem::path` de manera explícita, dando entender inmediatamente que estos parámetros hacen referencia a rutas en el sistema de archivos y no a cualquier `string`. Además, el hecho de que el argumento sea un `vector` le da más flexibilidad para que formatos que lean o escriban múltiples archivos tengan una interfaz uniforme. El método `isWhitespace` de `MeshReader` era una función libre declarada en el archivo `triangulation.hpp`[1], pero que no era parte de `Triangulation`, este método es usado para leer mallas correctamente omitiendo caracteres de espacios en blanco.

La clase `MeshReader` tiene 2 implementaciones concretas: `NodeEleReader` y `OffReader` que leen los formatos mencionados en la Sección 2.1.2, siendo estos `.node`, `.ele` y `.neigh` para el primero y `.off` para el segundo (Anexo B). Estas clases se pueden ver en la Figura 29

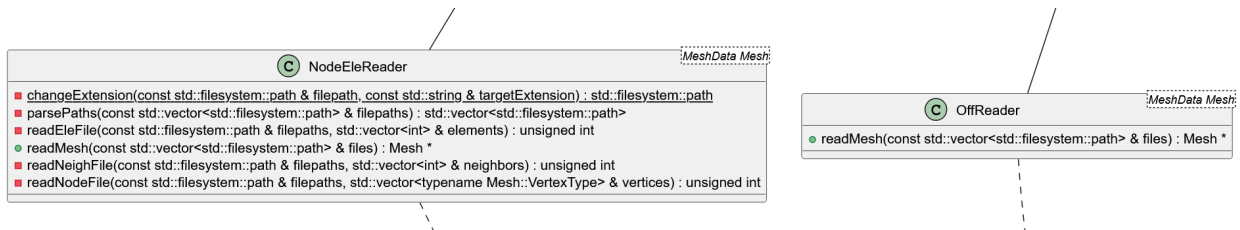


Figura 29: Representación UML de las clases NodeEleReader y OffReader

La clase `MeshWriter` también tiene 2 implementaciones concretas: `OffWriter` y `AleWriter`, que escriben en formato `.off` y `.ale` respectivamente (Anexo B). Su representación UML se puede ver en la Figura 30.

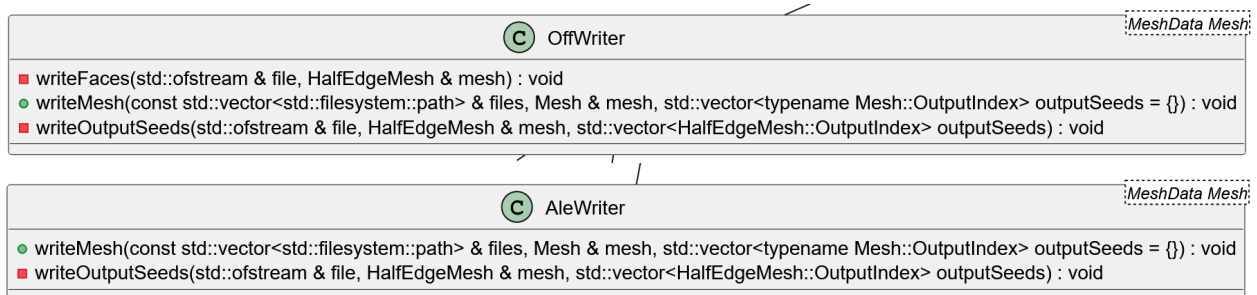


Figura 30: Representación UML de las clases OffWriter y AleWriter

Siguiendo con los miembros de `PolygonalMesh`, también se declara la clase `MeshGenerator`:

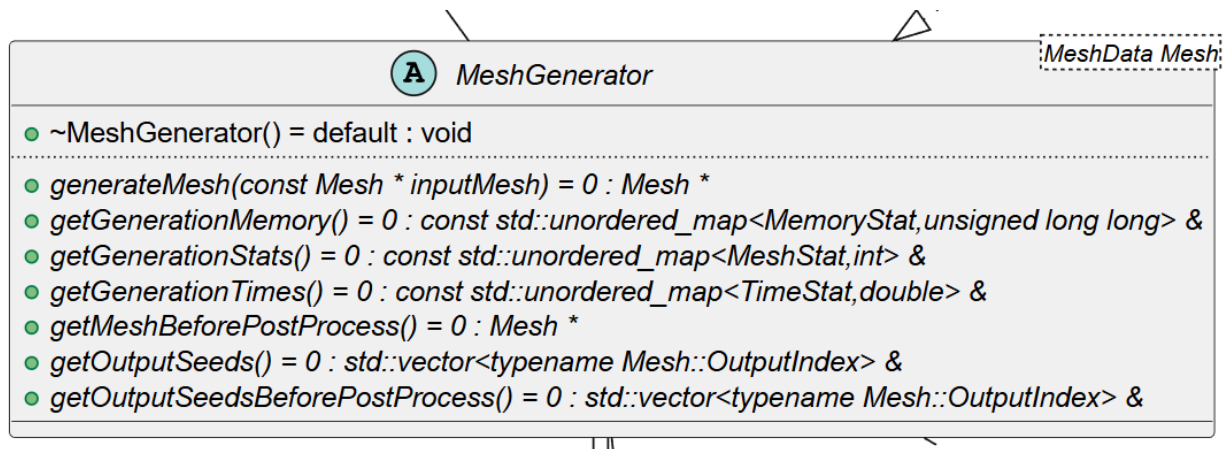


Figura 31: Representación UML de la clase `MeshGenerator`

Esta clase abstracta declara los métodos que un generador de mallas debe implementar, incluyendo aquellos que permiten recolectar estadísticas y obtener el conjunto correcto de índices de salida de la malla, puesto que, por temas de eficiencia de memoria, prácticamente nunca se eliminan elementos de la malla, estos se invalidan en la gran mayoría de los casos al omitirlos o marcarlos con el valor `invalidIndexValue`.

La clase `MeshGenerator` posee dos implementaciones concretas: `PolyllaGenerator` que encapsula la lógica del algoritmo Polylla[1] original y `DelaunayCavityGenerator` que representa el generador basado en cavidades.

La clase `PolyllaGenerator` implementa los métodos declarados en `MeshGenerator` y define el alias `BinaryVector` que corresponde a un `vector` de C++ que almacena valores de tipo `uint8_t` (al igual que la implementación original), es decir, enteros sin signo de 8 bits, que solo guardan valor 0 o 1. Esto se hace por razones de alineamiento de memoria y rendimiento, puesto que utilizar un `vector` de valores tipo `bool`, intenta comprimir la memoria de forma que el acceso y escritura pueden no ser directos y resultar en problemas de compatibilidad con otras funciones de la librería estándar de C++ según múltiples fuentes[30–36].

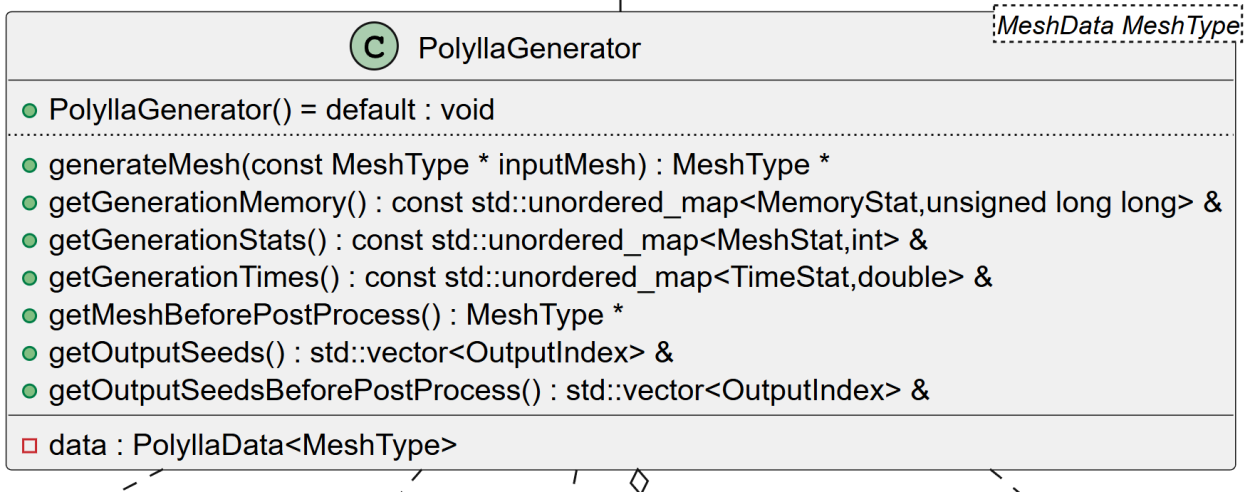


Figura 32: Representación UML de la clase `PolyllaGenerator`

Las implementaciones de `MeshGenerator` también tienen una implementación de la clase `MeshGeneratorData` (Figura 33), la cual provee contenedores dinámicos en forma de *hash maps* para almacenar estadísticas que se almacenan una sola vez, ya que escribir a un *hash map* es una operación con complejidad $O(1)$ amortizado[37] y puede afectar negativamente el rendimiento. Para mitigar esto se hacen 2 cosas: primero, las llaves de estos *hash map* son valores enteros predefinidos en 3 enumeraciones (*enum*) distintas que representan el grupo de estadística con un valor numérico asociado, que serán descritos más adelante, aprovechándose de que el *hash* de un valor `int` puede ser el mismo valor, y segundo, se escribe un 0 de manera temprana en todas las estadísticas que el generador efectivamente utilice logrando que cada estadística ya tenga una llave existente en el *hash map* para escrituras futuras. Estos 3 *enum* mencionados son los siguientes: `MeshStat` con enteros para estadísticas de la malla, como su cantidad de polígonos, cantidad de *barrier edge tips* reparados en caso de Polylla, entre otros; `TimeStat` con valores de tipo `double` para estadísticas de tiempo como el tiempo de generación de la malla, cuanto tiempo tardó una etapa en particular, entre otros; y `MemoryStat` con valores de tipo `unsigned long long` para estadísticas de memoria como la cantidad de bytes utilizados por cada arreglo.

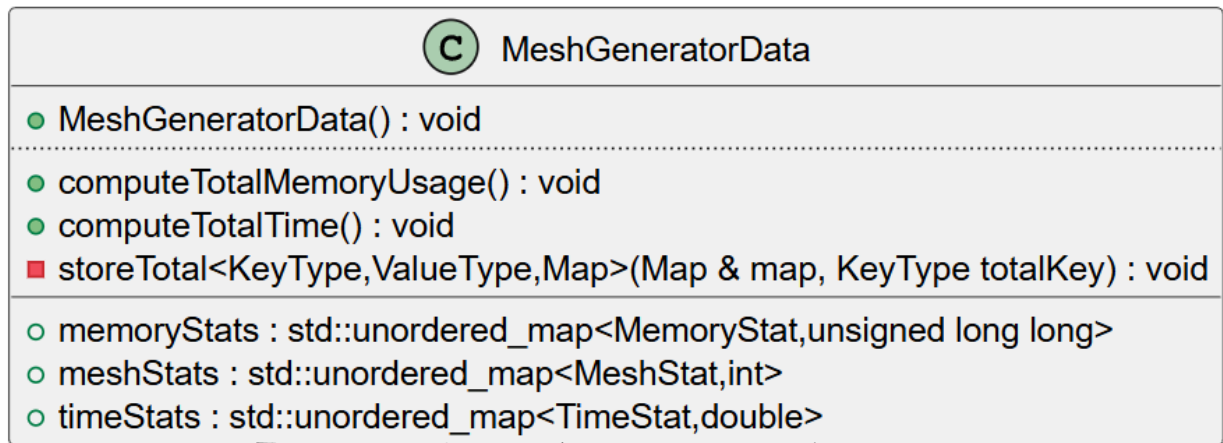


Figura 33: Representación UML de la clase MeshGeneratorData

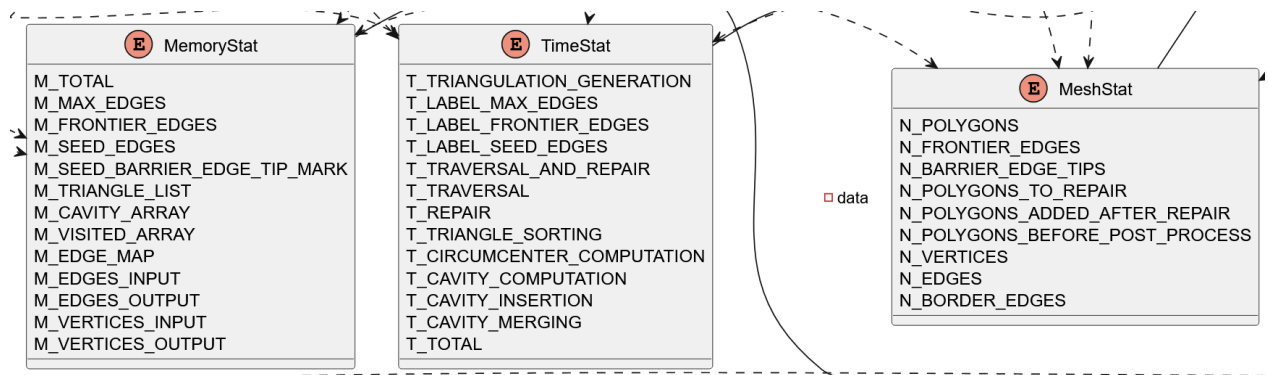


Figura 34: Representación UML de enumeraciones para estadísticas

Dentro de su archivo *header*, cada enumeración también define un arreglo estático y constante de *strings* con el “nombre” asociado a la estadística, esto facilita el trabajo de escribir estadísticas a un archivo *json* como lo hacía la implementación original de Polylla, con la ventaja de ser menos propenso a errores en caso de añadir una estadística nueva, puesto que se hará referencia a este arreglo de *strings* y no se escribirá el *string* directamente. Estos arreglos están marcados como *constexpr*[38], por lo que en tiempo de compilación sus valores se reemplazan literalmente en el lugar en que son utilizados. Esta configuración provee una manera altamente genérica de escribir estadísticas con el método *writeStatsToJson* de la clase *PolygonalMesh*, aprovechando la estructura de un *hash map* como contenedor de tipo llave-valor para hacer una escritura directa de su contenido sin importar cuál sea este, por lo que introducir estadísticas nuevas no requiere modificaciones al método.

La clase *MeshGeneratorData* define dentro de sí métodos altamente genéricos para calcular un valor “total” de algún grupo de estadística, particularmente, de tiempo y memoria.

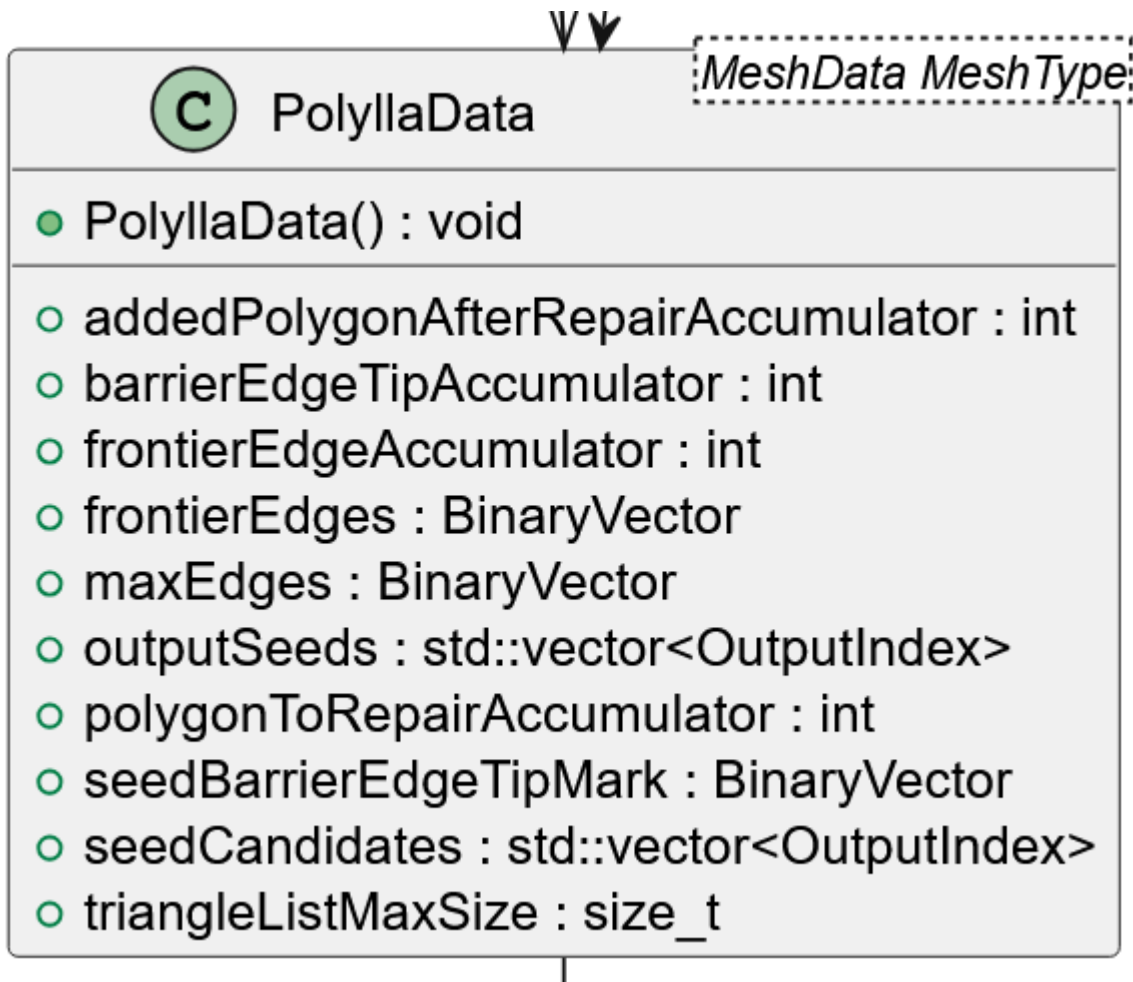


Figura 35: Representación UML de la clase `PolyllaData`

La clase `PolyllaData` utilizada por la clase `PolyllaGenerator` se encarga de almacenar los arreglos que poseía la clase `Polylla` original, esto se hace de esta forma dado que implementaciones de `Polylla` con distintos tipos de malla, podrían necesitar otras estructuras de datos o métodos especiales, los cuales se pueden declarar como una especialización concreta para la malla en cuestión sin necesidad de modificar la clase `PolyllaGenerator` original.

Además de las clases que implementan `MeshGeneratorData`, también existen clases aparte, denominadas `MeshHelper` en *namespaces* de C++ distintos, que engloban funciones estáticas que describen cada operación específica que un generador debe realizar en una malla con un tipo concreto, esto permite separar lo que es el algoritmo de generación de mallas como tal de detalles de implementación dependientes de la malla.

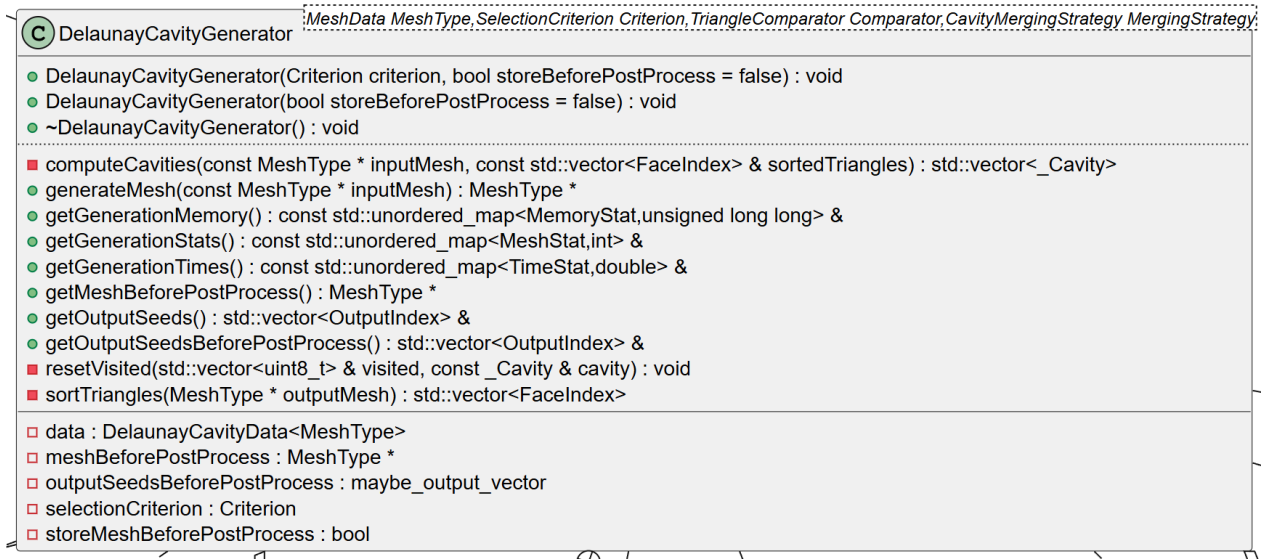


Figura 37: Representación UML de la clase `DelaunayCavityGenerator`

Estos parámetros *template* constan de lo siguiente:

- **MeshType**: El tipo de malla a utilizar, en la implementación actual solo es posible utilizar la malla basada en *half edges* de la clase `HalfEdgeMesh`.
- **Criterion**: Un objeto sujeto a la restricción del *concept* `SelectionCriterion`, este objeto se utiliza para determinar que triángulos son más “malos” según lo que determine el criterio en particular, que podría ser tener un ángulo mínimo menor a un cierto umbral, un área menor a un cierto umbral, entre otros. Estos triángulos “malos” son los primeros en ser utilizados como posibles “semillas” de una cavidad (Sección 2.1.3).
- **Comparator**: Objeto restringido por el *concept* `TriangleComparator`, el cual se encarga de ordenar los triángulos de la malla para determinar cuales se escogen primero como “semilla” después de que el objeto de `Criterion` haga una partición inicial.
- **MergingStrategy**: Objeto restringido por el *concept* `CavityMergingStrategy`, este se encarga de determinar reglas para unir los triángulos que forman una cavidad, ya sea durante la formación inicial o durante algún paso de postprocesado que este objeto defina si es que dicho paso existe.

Los *concepts* mencionados previamente se definen de la manera siguiente:

- **SelectionCriterion**: Declara que todo criterio de selección debe definir el método `operator()`, con dos argumentos: una malla de entrada y un índice de cara de la malla. Este método debe retornar algo convertible a `bool` (generalmente `bool`). Este valor de retorno indica si el triángulo dado debe ser seleccionado como triángulo “semilla” de forma prioritaria o no.
- **TriangleComparator**: Declara que todo comparador de triángulos debe tener un método estático `compare` que recibe 3 argumentos: una malla de entrada, y dos índices de cara, `t1` y `t2`. Este método `compare` se utiliza como parámetro en la función `std::sort` de C++, la cual le indica como se deben comparar dos triángulos al ordenarse. Debe retornar `true` si el triángulo representado por `t1` es “menor” que `t2`, es decir, que `t1` debe aparecer

antes en el arreglo tras ordenar, asumiendo un orden ascendente. Esto se invierte si el orden es descendiente.

- **CavityMergingStrategy**: Declara que toda estrategia de unión de cavidades debe definir al menos uno de los siguientes 4 métodos estáticos:
 1. **preAdd** (según información de cavidades): Método que recibe una malla de entrada, un índice de cara y un arreglo (**vector** de C++) de cavidades y retorna **true** si el triángulo es un candidato valido para formar parte de una cavidad. Este *concept* en la implementación actual no tiene uso real, ya que fue supercedido por una versión alternativa, pero se deja en caso de que otra implementación futura no pueda utilizar la versión alternativa y necesite más información respecto a la malla o las cavidades para determinar elegibilidad.
 2. **preAdd** (según presencia en cavidades): Método que recibe dos argumentos: un índice de cara y un arreglo (**vector**) de enteros sin signo utilizado exclusivamente con valores 0 o 1, similar al **BinaryVector** de **PolyllaGenerator**. Este método verifica elegibilidad basándose en el arreglo que indica si el triángulo dado ya forma parte de una cavidad o no.
 3. **postCompute**: Método que recibe la malla de entrada y el arreglo de cavidades, su rol es realizar cualquier operación necesaria que se estime conveniente después de computar que triángulos forman las cavidades, pero antes de proceder a la inserción efectiva de estas. Actualmente no tiene uso, pero se deja en caso de que una futura implementación lo necesite por algún motivo.
 4. **postInsertion**: Método que recibe la malla de entrada, la malla de salida con las cavidades ya insertadas y un objeto **DelaunayCavityData**, implementación de **MeshGeneratorData** para el generador basado en cavidades (Figura 39). Este método se encarga de realizar cualquier operación de postproceso necesaria una vez que las cavidades ya fueron insertadas según lo que determine la estrategia de unión en particular.

En este último *concept* se puede observar la gran flexibilidad que otorga un *concept* mezclado con el uso de `if constexpr`[38], permitiendo definir estrategias de unión que solo implementan aquellos métodos que realmente necesitan, eliminando por completo las líneas de código del binario final relacionadas al uso de métodos que esta no utilice.

La clase **DelaunayCavityGenerator** hace uso de la estructura **Cavity** para encapsular los componentes de una cavidad a medida que estas se van construyendo, esta estructura se puede ver en la Figura 38.

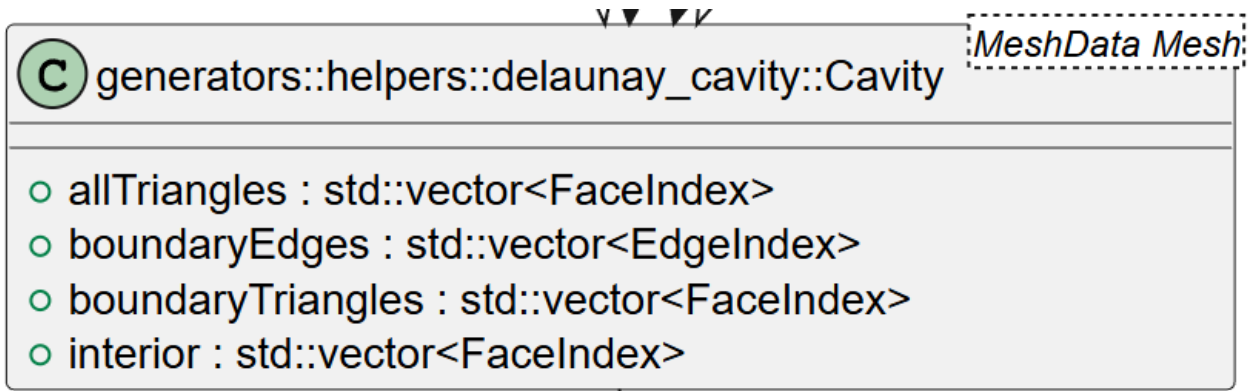


Figura 38: Representación UML de la estructura Cavity

La estructura de cavidad guarda los triángulos que componen la cavidad, sus aristas de borde, triángulos de borde y triángulos interiores.

Al igual que la clase PolyllaGenerator, la clase DelaunayCavityGenerator también posee su implementación de MeshGeneratorData y su MeshHelper respectivo, pero a diferencia de PolyllaGenerator, tiene más pasos que se pueden generalizar para todo tipo de malla y no dependen por completo en su MeshHelper.

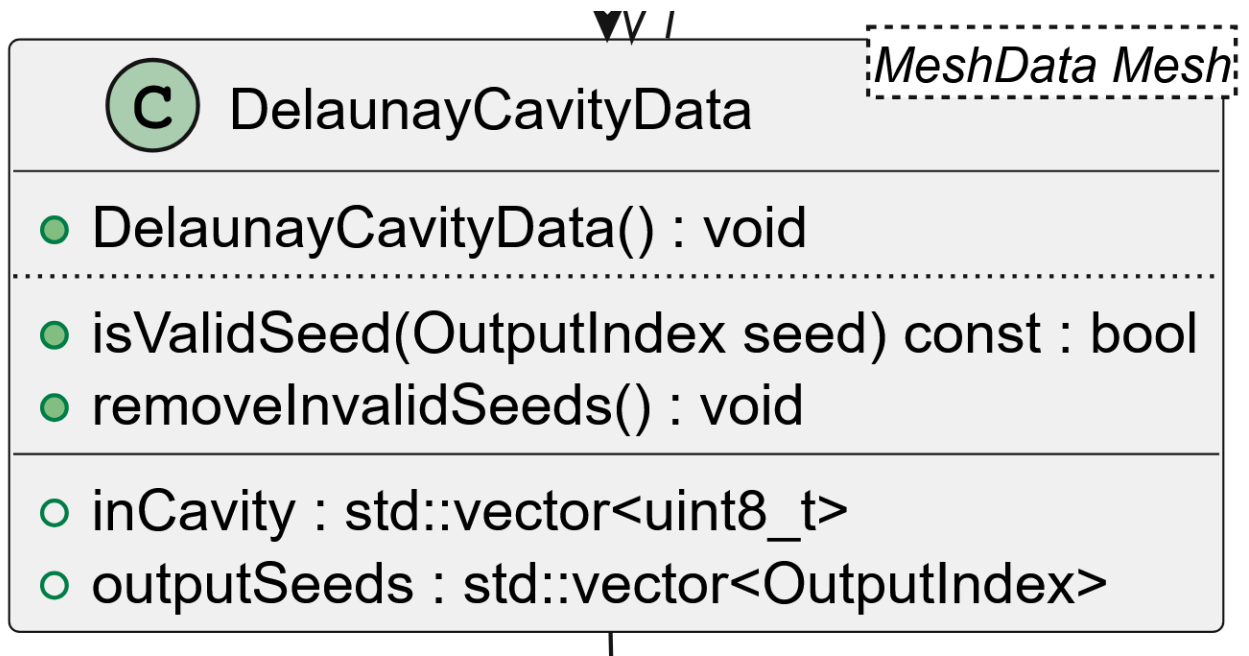


Figura 39: Representación UML de la clase DelaunayCavityData

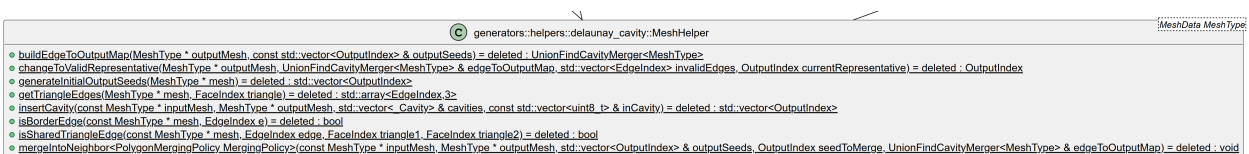


Figura 40: Representación UML de la clase MeshHelper del algoritmo basado en Cavidades

Las implementaciones concretas de estos parámetros y el funcionamiento del generador basado en cavidades serán descritos más a fondo en el capítulo siguiente.

Capítulo 4

Implementación de algoritmo basado en cavidades

A continuación se muestra la solución al problema: el algoritmo de generación de mallas basado en cavidades, detallando los pasos que lo componen, junto con pseudocódigo ilustrativo.

4.1. Algoritmo de generación de mallas basado en cavidades

El algoritmo de generación de mallas basado en cavidades consta de 4 etapas: Selección y ordenamiento de triángulos, computo de circuncentro y cavidades, inserción de cavidades y un paso opcional de postprocesado.

4.1.1. Selección y ordenamiento de triángulos según criterios

Además de la malla triangular de input, el algoritmo también necesita comparadores y criterios de selección, los cuales están descritos por los *concepts* `TriangleComparator` y `SelectionCriterion`, es según estos comparadores y criterios el cómo se decide el orden en que las cavidades serán calculadas y posteriormente insertadas.

Durante el semestre se desarrollaron los siguientes comparadores de triángulos, los cuales pueden ordenar de forma ascendente o descendente si es que aplica:

- `AngleComparator`: Ordena según ángulo mínimo o máximo.
- `AreaComparator`: Ordena según el valor del área o doble del área.
- `EdgeLengthComparator`: Ordena según largo mínimo o máximo de las aristas.
- `NullComparator`: No cambia el orden de los triángulos y este se mantiene según como vienen en la malla.
- `RandomComparator`: Revuelve los triángulos con un generador psuedoaleatorio *mersenne twister*[39] con un objeto del tipo `std::mt19937` de C++. La semilla es configurable por el usuario.

También se desarrollaron los siguientes criterios de selección que priorizan triángulos que cumplen con alguna característica deseada o indeseada, para que sean los primeros en considerarse como parte de una cavidad:

- **MinAngleCriterion:** Prioriza triángulos donde el coseno cuadrado del ángulo mínimo esté por debajo de un umbral. Se prefiere usar coseno cuadrado por su eficiencia, de la misma forma que se hace en el código de Triangle[8]. Su correctitud está asegurada mientras se trabaje con ángulos agudos.
- **MinAngleCriterionRobust:** Similar al anterior, pero calcula los ángulos reales mediante la función arcocoseno, computacionalmente caro.
- **MinAreaCriterion:** Prioriza triángulos donde el área esté por debajo de un umbral.
- **MinArea2Criterion:** Similar al anterior, pero ahorra una operación al utilizar el doble del área, resultado directo de un producto cruz en 2 dimensiones.
- **NullSelectionCriterion:** No prioriza ningún triángulo y permite al comparador ordenar la totalidad de ellos.

Inicialmente, se planteaba la capacidad de componer estos criterios de selección con pequeños funtores simulando álgebra booleana para tener criterios arbitrariamente complejos, los cuales existen, pero dada la cantidad infinita de formas en que estos pueden ser compuestos, no fueron utilizados durante pruebas:

- **NotCriterion:** Invierte un criterio, es decir, prioriza aquellos que no son priorizados por un criterio particular.
- **AndCriteria:** Dados dos criterios (incluyéndose a sí mismo como “un criterio”), solo prioriza un triángulo si este es priorizado por ambos criterios. Cumple la función de un *y* lógico.
- **OrCriteria:** Similar al anterior, pero le basta con que sea priorizado por un criterio o ambos. Cumple la función de un *o* lógico.

Esta parte del algoritmo sigue los pasos del siguiente pseudocódigo:

Etapa 1: Selección de triángulos
Entrada: Malla inicial M , Comparador O , Criterio de selección R
Salida: Conjunto de triángulos ordenados antes del cómputo de cavidades

```

1 L ← Lista de triángulos de M
2 if R y 0 no son nulos then
3     L ← Ordenar L ubicando los triángulos escogidos por R primero
4     P ← Índice del primer triángulo no escogido por R
5     L ← Ordenar L desde P en adelante usando el comparador 0
6 else if R es criterio nulo y 0 no then
7     L ← Ordenar L completo usando el comparador 0
8 else if 0 es comparador nulo y R no then
9     L ← Ordenar L ubicando los triángulos escogidos por R primero
10 end if
11 return L

```

Figura 41: Algoritmo de selección de triángulos

Notar que en la implementación actual, tanto el comparador como el criterio de selección preservan el orden relativo de los triángulos, haciendo uso de las funciones de la librería estándar de C++ `std::stable_sort` y `std::stable_partition`.

La implementación de estas funciones varía levemente según el compilador, siendo el algoritmo *merge sort*[40–44] la base común para `std::stable_sort` y alguna variación de *divide and conquer*[45–48] para `std::stable_partition`.

También cabe destacar que, dado que tanto el comparador como el criterio de selección son parámetros *template*, todos los `if` de este paso se resuelven en tiempo en compilación usando la instrucción `if constexpr`[38] introducida en C++17, la cual, de manera similar a una macro, permite descartar o incluir ramas completas del código máquina presentes en el archivo ejecutable final. Se diferencia de una macro en el hecho de que es capaz de usar características de reflexión intrínsecas del lenguaje (además de revisarse después de haber procesado macros), incluyendo validaciones de *concepts* para asegurar una correctitud más rigurosa que no compromete la eficiencia del programa final por no necesitar hacer validaciones en tiempo de ejecución.

4.1.2. Cómputo de circuncentro y cavidades

Para esta etapa del algoritmo, se recorre la malla poligonal usando el algoritmo *Breadth First Search*[49] (o *BFS*) utilizando la malla como un grafo considerando a cada triángulo individual como un nodo de este. El primer triángulo t_i presente en la lista de triángulos según el orden del paso 1, debe ser parte de la primera cavidad y se debe utilizar como el nodo de partida de un recorrido *BFS* manteniendo una cola de triángulos a visitar. Esta cola añade triángulos vecinos si su circuncírculo contiene al circuncentro del triángulo inicial, luego por cada vecino que se va quitando de la cola, se hace la misma verificación para sus vecinos, deteniéndose cuando la cola esté vacía.

En este paso se lleva a cabo el cálculo del circuncentro del triángulo semilla actual (el primero en quitarse de la cola). Esto se hace mediante el siguiente método basado en determinantes descrito en el libro de Shewchuk[29]:

Sean A , B y C los vértices de un triángulo en orientación CCW, primero, para simplificar cálculos y sin pérdida de generalidad, se aplica una traslación a estos vértices de modo que A , B o C quede en el origen, por simplicidad, se asumirá que A se traslada al origen y se definen los siguientes nuevos vértices:

$$A' = A - A = (0, 0)$$

$$B' = B - A$$

$$C' = C - A$$

Estos nuevos valores se utilizarán como vectores, para hacer uso del producto cruz o producto vectorial, cuya coordenada z equivale al doble del área de el triángulo formado por estos dos vectores si fuesen aristas más la arista que une las puntas de estos vectores. Se computará un valor D que corresponde al cuádruple del área del triángulo desplazado:

$$D = 2[(A' \times B')_z + (B' \times C')_z + (C' \times A')_z]$$

$$D = 2(B' \times C')_z$$

$$D = 2(B'_x C'_y - B'_y C'_x)$$

Luego las coordenadas del circuncentro desplazado U' serán

$$U'_x = \frac{1}{D} [C'_y (B'^2_x + B'^2_y) - B'_y (C'^2_x + C'^2_y)]$$

$$U'_y = \frac{1}{D} [B'_x (C'^2_x + C'^2_y) - C'_x (B'^2_x + B'^2_y)]$$

Se debe tener precaución al calcular D , ya que este podría ser cero, indicando la presencia de un “caso degenerado”, pero estos triángulos también serán eliminados como parte de la cavidad. Esto podría ocurrir por errores de precisión en los datos.

Finalmente, las coordenadas del circuncentro real estarán ubicadas en:

$$U = U' + A$$

Con esta información, esta etapa del algoritmo se describe de la manera siguiente:

Etapa 2.1: Cálculo de circuncentros
Entrada: Triángulo inicial t_i
Salida: Circuncentro de t_i

```

1   $V_1, V_2, V_3 \leftarrow$  Vértices de  $t_i$ 
2   $V_0 \leftarrow$  Vértice auxiliar que representa el origen (0,0)
3   $V_2', V_3' \leftarrow$  Vértices  $V_2$  y  $V_3$  desplazados al origen según  $V_1$ 
4   $D \leftarrow$  Determinante basado en producto cruz del origen con  $V_2'$  y  $V_3'$ 
5   $c' \leftarrow$  Circuncentro de  $t_i$  desplazado en  $V_1$ 
6   $c \leftarrow$  Circuncentro de  $t_i$ 
7  return  $c$ 

```

Código 1: Algoritmo de computo de circuncentros

Al remover un triángulo de la cola, este se marca como parte de la cavidad, posteriormente, se revisa si es un triángulo de borde de la malla, ya que de ser el caso, una de sus aristas debe preservarse en la cavidad. Luego se revisa cada uno de sus vecinos verificando tres condiciones en orden:

1. Si el triángulo vecino ya fue visitado en este recorrido, se procede al vecino siguiente en la cola *BFS* sin hacer nada más.
2. Si no fue visitado, se marca como visitado, luego se verifica si es que la estrategia de unión de polígonos considera a este triángulo vecino como un candidato “válido” para formar parte de la cavidad. En la implementación actual, un triángulo vecino es un candidato válido si no pertenece a otra cavidad previamente calculada, pero es posible extender esto a futuro para imponer cualquier otra restricción arbitraria.
3. Si el triángulo vecino es un candidato válido, se debe verificar que su circuncírculo contiene el circuncentro de t_i , en caso de cumplirse, entonces este triángulo se añade a la cola.

Finalmente, si el triángulo vecino no fue previamente visitado y no es válido o su circuncírculo no contiene al circuncentro del triángulo inicial, significa que la arista compartida entre este triángulo vecino y el triángulo actual es un borde de la cavidad y debe preservarse. Estas aristas de borde se guardan por separado, ya que serán las únicas aristas presentes en el output descartando todas las demás.

Una vez que se hace el recorrido completo de un triángulo según el orden del paso 1, se inicia un nuevo recorrido *BFS* tomando como punto de partida el triángulo siguiente si y solo si este no forma parte de una cavidad previamente computada, de lo contrario,

el recorrido no se realiza desde este triángulo pasando al siguiente hasta haber intentado iniciar un recorrido por todos los triángulos de la malla.

A continuación se presentan estos mismos pasos en forma de pseudocódigo:

Etapa 2.2: Cómputo de cavidades
Entrada: Malla inicial M , Conjunto de triángulos ordenados C
Salida: Conjunto de cavidades D

```

1 D ← {∅} Arreglo vacío para guardar cavidades
2 V ← Arreglo de booleanos para marcar triángulos
  visitados
3 I ← Arreglo auxiliar para marcar triángulos que ya
  forman parte de una cavidad
4
5 for cada  $t_i \in C$  do
6   if I[ $t_i$ ] = True then
7     continue
8   end if
9    $c_i \leftarrow$  Circuncentro de  $t_i$ 
10  Q ← Cola de triángulos vecinos para BFS
11  d ← Objeto de cavidad vacío
12   $T_i \leftarrow$  Triángulos interiores de d
13   $T_o \leftarrow$  Triángulos de borde de d
14  T ← Triángulos totales de d
15   $E_r \leftarrow$  Aristas de borde de d
16  Q.push( $t_i$ )
17  V[ $t_i$ ] ← True
18  T ← T ∪ { $t_i$ }
19  while not Q.empty() do
20     $t_n \leftarrow$  Q.pop()
21    I[ $t_n$ ] ← True
22    N ← Triángulos vecinos de  $t_n$  en M
23    E ← Aristas de  $t_n$  en M
24    b ← Marcador de triángulo de borde, False por
      defecto
25    if cantidadDeVecinos( $t_n$ ) < 3 then
26      for cada arista e ∈ E do
27        if e ∈ borde de M then
28          b ← True si  $t_n \neq t_i$ 
29           $E_r \leftarrow E_r \cup \{e\}$ 
30        end if
31      end for
32    end if
33    for cada vecino n ∈ N do
34      if V[n] = True then
35        continue
36      end if
37
38      if candidatoValido(n) and  $c_i \in$  circuncirculo
        de n then
39        V[n] ← True
40        Q.push(n)
41        T ← T ∪ { $t_i$ }
42      else if  $t_n = t_i$  then
43        for cada arista e ∈ E do
44          if e ∈  $t_n$  and e ∈ n then
45             $E_r \leftarrow E_r \cup \{e\}$ 
46          end if
47        end for
48      else then
49        b ← True
50        s ← Arista compartida entre  $t_n$  y n
51         $E_r \leftarrow E_r \cup \{s\}$ 
52      end if
53    end for
54
55    if b = True then
56       $T_o \leftarrow T_o \cup \{t_n\}$ 
57    else then
58       $T_i \leftarrow T_i \cup \{t_n\}$ 
59    end if
60  end while
61
62  Reiniciar V a False
63  D ← D ∪ {d}
64 end for
65
66 return D

```

Figura 42: Algoritmo de cómputo de cavidades

4.1.3. Inserción de Cavidades

Con la información del paso anterior es posible hacer efectiva la mutación de la malla para convertir los conjuntos de triángulos que forman las cavidades en polígonos arbitrarios formados por sus aristas de borde.

Actualmente, este paso es el único que no fue generalizado para cualquier tipo de malla, ya que el cómo se unen polígonos y qué representa a cada uno es un detalle de implementación de la malla. En este caso particular, la inserción de cada cavidad se hace asumiendo que la malla usa una representación basada en *half edges*, haciendo uso de la clase auxiliar **MeshHelper**, la cual es un esqueleto que define operaciones que el generador desea hacer sobre la malla, pero que la malla misma no necesita implementar por separado, ya que se logra mediante combinaciones de operaciones existentes en la malla que el generador precisa realizar para formar las cavidades. De no ser este el caso, cada malla tendría que definir específicamente las combinaciones de operaciones para un generador concreto, lo que genera un acoplamiento fuerte entre malla y generador, que es justamente lo que se desea evitar. Esta implementación se detalla a continuación:

1. Se marca qué aristas de la totalidad de la malla pertenecen al borde de la cavidad actual, esto para permitir una búsqueda inmediata al momento de formar el polígono final.
2. Se toma una arista de borde, en este caso la primera que aparezca en el objeto **Cavity** (puede ser cualquiera) y se calcula su arista **next**, luego, utilizando el método **CCWEdgeToVertex** se hace un barrido en sentido antihorario desde esta arista **next** buscando la siguiente arista que si es parte del borde de la cavidad. Cuando esta es encontrada, se reconecta con la arista anterior y esta pasa a ser la siguiente arista a reconectar con otra arista de borde. Este proceso sigue hasta volver a la primera arista de borde tomada.

Hecho esto, se actualiza también el conteo de aristas y polígonos que la malla reporta para que estos sean coherentes con la malla de salida, detalle importante al momento de escribir la malla a un archivo.

A continuación se presenta este proceso en forma de pseudocódigo:

Paso 4: Inserción de cavidades
Entrada: Malla inicial M , Conjunto de cavidades D
Salida: Malla mutada M' y arreglo de polígonos de salida P

```

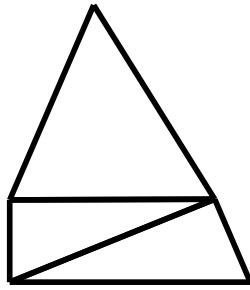
1 P ← {∅}
2 M' ← Copia de M
3 p ← Cantidad de polígonos de M'
4 a ← Cantidad de aristas de M'
5 for cada cavidad di ∈ D do
6     B ← Arreglo de booleanos para aristas de borde en la malla
7     for cada arista e ∈ di.aristasDeBorde do
8         B[e] ← True
9     end for
10    p ← p - size(di.triangulosTotales) - 1
11    a ← a - size(di.triangulosTotales) * 3 + size(di.aristasDeBorde)
12    f ← Primera arista en di.aristasDeBorde
13    P ← P ∪ {f}
14    h ← f
15    do while h != f
16        c ← M.CCWEdgeToVertex(h)
17        while B[c] = False do
18            c ← M.CCWEdgeToVertex(c)
19        end while
20        M'.setNext(h,c)
21        M'.setPrev(c,h)
22        h ← c
23    end while
24 end for
25 return M', P

```

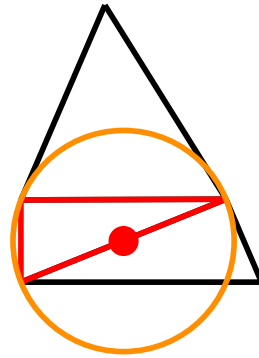
Figura 43: Algoritmo de inserción de cavidades

Notar que el arreglo P guarda aristas, ya que en la representación basada en *half edges*, los polígonos se identifican con un solo *half edge* de su interior. Aquí es cuando cobra particular importancia la distinción de qué es una salida como se mencionó en el capítulo anterior, y la claridad que brindan los *concepts* de C++, ya que a modo general P es un arreglo de `OutputIndex`, no importándole al generador ni otras clases la malla subyacente a pesar de que el proceso de inserción si lo sea. En este caso `OutputIndex = EdgeIndex`.

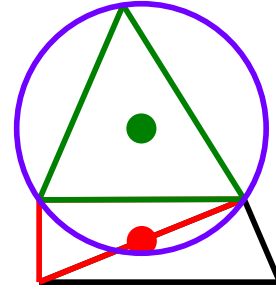
En la Figura 44 se muestran ilustraciones paso por paso del cómputo e inserción de una cavidad, en a) se pueden ver 3 triángulos que forman una malla, luego, en b) se escoge el triángulo en rojo como semilla para iniciar el recorrido *BFS* y se agregan los vecinos a la cola, en c), se revisa un vecino verificando que su circuncírculo contenga al circuncentro del triángulo rojo, dado que en este ejemplo si lo contiene, su arista compartida se marca para ser eliminada en d). Los pasos e) y f) siguen esta misma lógica con otro vecino, y en caso de que estos triángulos tuviesen otros vecinos, se hace la misma verificación siempre con el circuncentro del mismo triángulo semilla, deteniendo el recorrido.



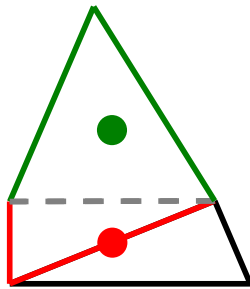
a) Malla original



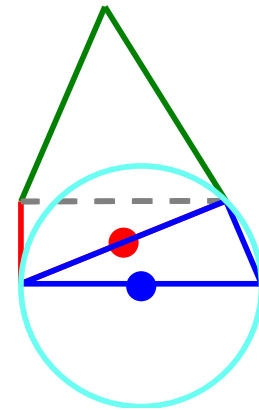
b) Circuncentro del triángulo semilla en rojo



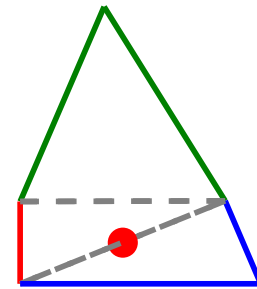
c) Circuncírculo del vecino conteniendo al punto rojo



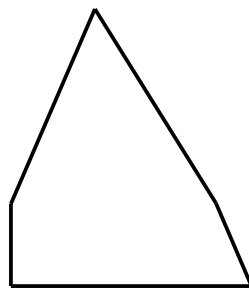
d) Marcado de arista compartida



e) Circuncírculo de otro vecino conteniendo al punto rojo



f) Marcado de arista con partida y fin del recorrido



g) Cavidad final tras reconexión de aristas

Figura 44: Pasos de la inserción de una cavidad

4.1.4. Postprocesado

Del mismo modo en que la primera etapa es altamente personalizable según el criterio de selección y comparador escogido, la etapa de postprocesado posee una gran variedad de

alternativas y posibilidad de extensión según el resultado deseado, como tener una malla que favorezca más o menos polígonos de cierta cantidad de lados, con mayor o menor área mínima o máxima, ángulos u otra cualidad. Durante el trabajo de memoria se desarrolló una etapa de postprocesado centrada en eliminar todos los triángulos restantes de la malla (*MergeTrianglesStrategy*), uniéndolos con alguno de sus vecinos según alguna política (*PolygonMergingPolicy*) de fusión de polígonos. Las políticas existentes que unen polígonos con alguno de sus vecinos al momento de escribir el documento son las siguientes:

- **EdgeLengthBasedMergingPolicy:** Une según la arista compartida con un vecino que tenga el mayor o menor largo según preferencia del usuario.
- **SizeBasedNeighborMergingPolicy:** Une según la cantidad de lados de los vecinos, escogiendo el vecino con más o menos lados según preferencia del usuario.
- **MaximizeConvexityMergingPolicy:** Intenta unir con un vecino de manera que el polígono resultante sea convexo, si no lo es, prueba con el vecino siguiente, si ninguna fusión resulta en un polígono convexo, no une los polígonos. Se considera que el polígono resultante es convexo si el signo del producto cruz entre 3 vértices consecutivos en una orientación en particular, ya sea horaria o antihoraria, tiene el mismo signo para todos los tripletes de vértices.
- **NullPolygonMergingPolicy:** Clase auxiliar para ejecuciones del programa donde no se desea hacer postprocesado.

La etapa de postprocesado hace uso de una estructura de datos similar a *union find*[50] para mantener un registro de quienes son los representantes válidos de los polígonos. La diferencia entre *union find* convencional y la estructura utilizada es que la operación de unión no es bilateral, es decir, unir A con B, significa algo distinto que unir B con A, puesto que el primero representa un índice de polígono cuyo representante será asignado, y el segundo, al índice del representante a asignar. Como no hay una manera concreta que defina unívocamente qué índice es cuál, esta lógica depende de como se llame al método de unión, definiéndose por convención que unir A con B significa que B es el nuevo representante de A. Dado que esta etapa también es altamente dependiente de detalles de la malla, en este caso, esta estructura guarda índices de aristas *half edge* como representantes. Una vez construida la estructura *union find*, se escoge el polígono a unir mediante la estrategia (*MergingStrategy*) con una política (*PolygonMergingPolicy*) particular.

El procedimiento para fusionar un polígono con uno de sus vecinos en una malla basada en *half edges* es el siguiente:

1. Se cuenta cuantas aristas tiene el polígono a unir, asegurándose de que efectivamente sean aristas compartidas con un vecino y no bordes de la malla.
2. Se recorren todas las aristas del polígono tratando de mantener el invariante de que ninguna arista compartida de polígono vecino sea la arista representante del vecino, si esta condición no se cumple, entonces se actualiza la arista representante en la estructura *union find* con otra del mismo polígono recorriendo su borde. Adicionalmente, se reemplaza el índice de salida en el arreglo de salida *P* de la etapa anterior.

3. Se delega a la política de fusión particular la decisión de elegir a qué vecino se une el polígono, y si es que esta fusión tiene éxito o no.
4. Si la fusión tiene éxito, se invalida el índice del polígono fusionado en P , y se actualiza la arista representante de todas las aristas que forman el nuevo polígono fusionado para que apunten a una arista válida en la estructura *union find*.
5. Una vez terminado el proceso, se eliminan los índices invalidados en P .

Es importante destacar que el invariante del paso 2 es esencial para mantener coherencia topológica en la malla, puesto que, de no cumplirse, habrá aristas inválidas en la salida que no realizan un ciclo completo a lo largo del borde de un polígono si se recorren con *next*.

Esta etapa se puede describir con el siguiente pseudocódigo:

Paso 5: Postprocesado
Entrada: Malla mutada M' , Conjunto de salidas P , Estrategia de unión S , Política de unión J
Salida: Malla final M' y arreglo de polígonos de salida P

```

1 U ← Union-Find de aristas con sus representantes de polígono
2 for cada índice de salida  $p_i \in P$  do
3   if  $S$  escoge a  $p_i$  then
4      $E_i \leftarrow$  Aristas de vecinos de  $p_i$ , inicialmente vacío
5      $N_i \leftarrow$  Aristas representantes de los vecinos en  $E_i$ 
6      $f \leftarrow$  Arista representante de  $p_i$ 
7      $h \leftarrow f$ 
8     do while  $h \neq f$ 
9       if  $\text{twin}(h) \notin$  borde de  $M'$  then
10         $E_i \leftarrow E_i \cup \{\text{Aristas compartidas por } p_i \text{ y } \text{twin}(h)\}$ 
11         $r_i \leftarrow$  Representante del vecino a través de  $h$ 
12         $o_i \leftarrow$  Copia de  $r_i$ 
13        if  $r_i = \text{twin}(h)$  then
14           $r_i \leftarrow M'.\text{next}(r_i)$  u otra arista
15        end if
16        if  $r_i \neq o_i$  then
17          Actualizar  $U$  con el nuevo representante  $r_i$ 
18           $P.\text{replace}(o_i, r_i)$ 
19        end if
20         $N_i \leftarrow N_i \cup \{r_i\}$ 
21      end if
22       $h \leftarrow M'.\text{next}(h)$ 
23    end while
24     $r_u \leftarrow J.\text{fusionar}(M', p_i, N_i, E_i)$ 
25    if  $r_u$  es un representante válido then
26       $i \leftarrow$  índice inválido de la malla, actualmente -1
27       $P.\text{replace}(p_i, i)$ 
28      Actualizar  $U$  con el representante  $r_u$  para sus aristas
29    end if
30  end if
31 end for
32 return  $M', P$ 

```

Figura 45: Algoritmo de fusión de polígonos

Al momento de realizar la fusión en la línea 24 se muta nuevamente la malla y es el momento en que se utiliza la clase `ConnectivityBackupT` interna a la malla particular que permite deshacer una unión de polígonos si la política no la determina apta. Por ejemplo, si `MaximizeConvexityMergingPolicy` determina que el polígono final es no convexo, utiliza la información de `ConnectivityBackupT` para reescribir los atributos `next` y `prev` de cada *half edge* involucrado para que vuelvan a su estado original.

4.2. Reescritura de Polylla

El algoritmo Polylla se movió a la clase `PolyllaGenerator` la cual extiende a `MeshGenerator` y puede ser usada en `PolygonalMesh` al igual que `DelaunayCavityGenerator`.

La lógica del algoritmo es exactamente la misma que había en Polylla-Mesh-DCEL[1], solo se hicieron cambios “estéticos” como renombrado de métodos, variables y uso de *alias* en lugar de tipos primitivos cuando se trabaja con índices que representan cosas distintas (véase Figura 32 y Figura 36).

La gran diferencia que tiene esta implementación, es que recibe el tipo de malla como parámetro *template*, desacoplando levemente el generador de detalles de la malla. Sin embargo, esta separación no puede hacerse por completo, ya que prácticamente todas las operaciones de Polylla dependen de la malla, pero esto queda encapsulado en una tercera clase, la clase `MeshHelper` (distinto *namespace* que la clase `MeshHelper` del otro generador), la cual tiene una especialización para *half edges* con el código original adaptado.

También se extraen algunos miembros de la clase `Polylla` original a otra clase llamada `PolyllaData`, la cual hereda de `MeshGeneratorData` para tener mayor facilidad a la hora de rastrear estadísticas y mayor flexibilidad para casos en que distintas mallas necesiten miembros diferentes proveyendo una especialización particular a su parámetro *template*.

La clase `PolyllaGenerator` se lee como una secuencia de los pasos descritos en la Sección 2.2.3, delegando el trabajo de realizar los pasos al `MeshHelper` que corresponda según el tipo de malla de manera explícita, lo que da un esquema claro para realizar una extensión a futuro que utilice un tipo de malla diferente.

Capítulo 5

Resultados

En este capítulo se incluyen los resultados experimentales, que incluyen tablas, gráficos y mallas de salida.

5.1. Parámetros utilizados y características de la máquina

Se utilizó la siguiente configuración de parámetros que produjo buenos valores experimentales:

- Criterio de selección: `NullSelectionCriterion`
- Comparador de triángulos: `EdgeLengthComparator` por arista más pequeña en orden ascendente.
- Estrategia de unión: `MergeTriangles`
- Política de unión: `SizeBasedMergingPolicy` según el vecino que tenga mayor tamaño (más aristas).

El código se probó con el compilador `g++` provisto por el entorno `mingw-64` y también por el compilador `clang` provisto por *Visual Studio*, ambos en Windows 11. Los resultados mostrados son aquellos producidos por el código compilado con `g++` en una máquina con las siguientes características relevantes:

- Sistema Operativo: Windows 11 25H2
- CPU: Intel Core i5-10400 @ 2.90GHz, 6 *Cores*, 12 *Threads*
- RAM: 16 GB DDR4 a 2666 MT/s
- Almacenamiento: SSD ADATA SU630 500GB

5.2. Muestra de mallas generadas

Las mallas triangulares utilizadas como base se generaron utilizando los scripts `10000x10000RandomPoints.py` y `datagenerator.sh` presentes en el repositorio de Polylla-Mesh-DCEL[1], el cual genera vértices en un cuadrado de 10000 por 10000 y luego hace que `Triangle[8]` genere una triangulación de Delaunay a partir de ellos. Se generaron 5 mallas de cada tamaño con semillas: 139, 68, 70, 14 y 43. Luego, de forma paralela se ejecutó

una instancia del programa con cada semilla para cada número de vértices. Considerando que el algoritmo como tal es completamente secuencial y que el procesador de la máquina utilizada posee 6 núcleos, cada ejecución no debería afectar a ninguna otra.

A continuación se muestran resultados de la aplicación del algoritmo basado en cavidades a una de estas mallas de 10 y 100 vértices, su malla Polylla respectiva con la implementación original y su malla basada en cavidades antes de postprocesar y después de postprocesar:

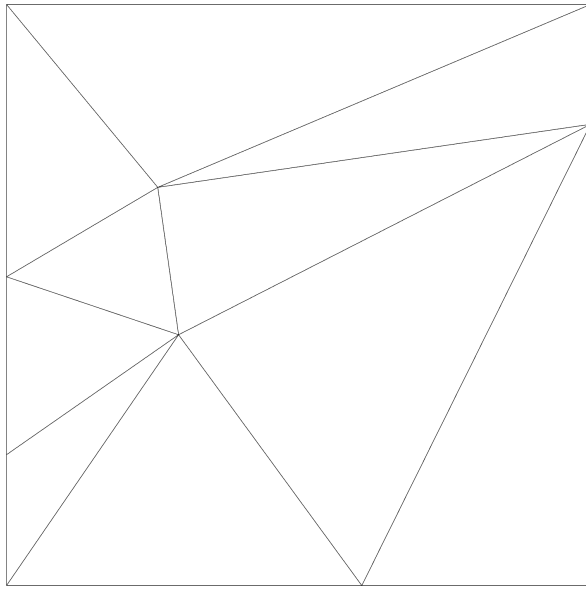


Figura 46: Malla poligonal generada por Triangle[8] de 10 vértices y 10 triángulos

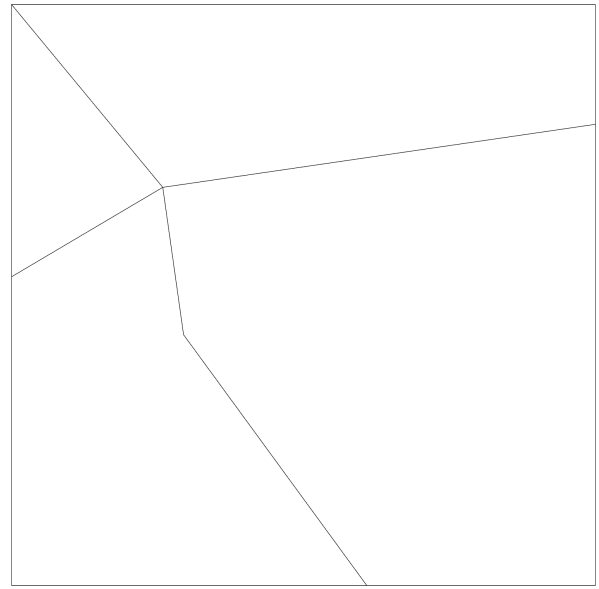


Figura 47: Malla poligonal generada por Polylla original a partir de la Figura 46 con 4 polígonos

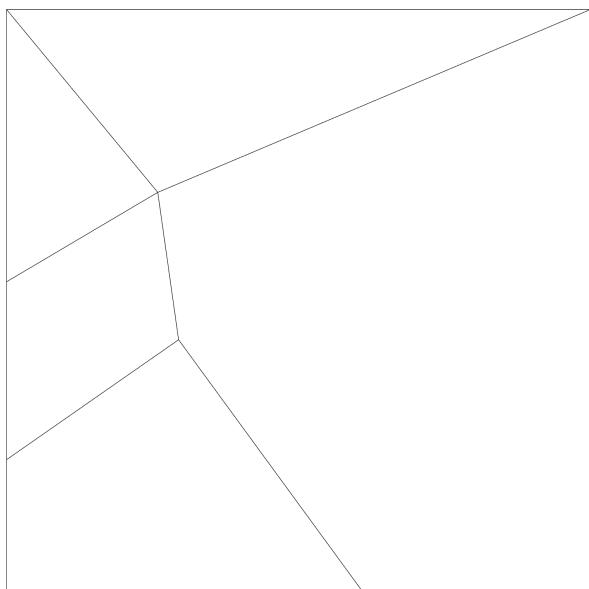


Figura 48: Malla poligonal generada desde cavidades a partir de la Figura 46 antes de postprocesar con 5 polígonos

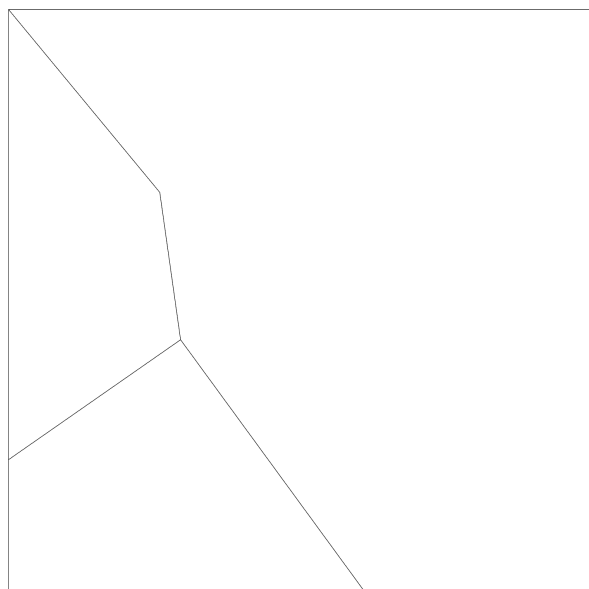


Figura 49: Malla de la Figura 48 después de postprocesar con 3 polígonos

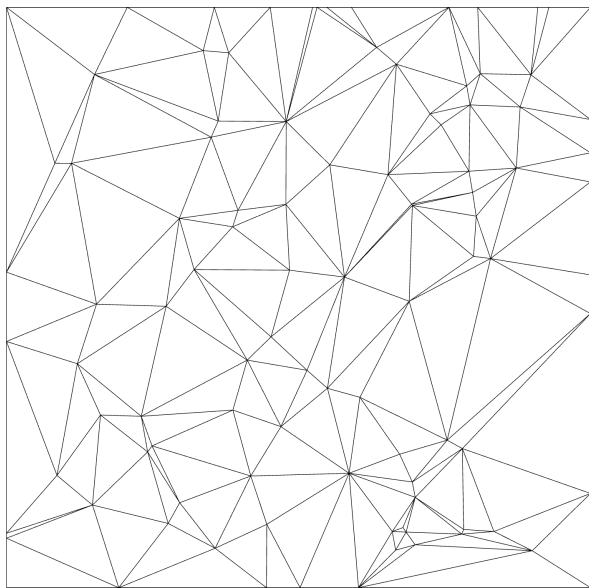


Figura 50: Malla poligonal generada por Triangle[8] de 100 vértices y 268 triángulos

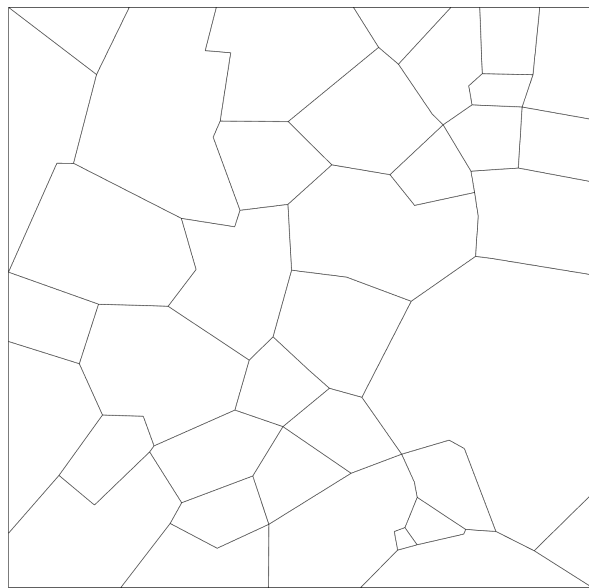


Figura 51: Malla poligonal generada por Polylla original a partir de la Figura 50 con 37 polígonos

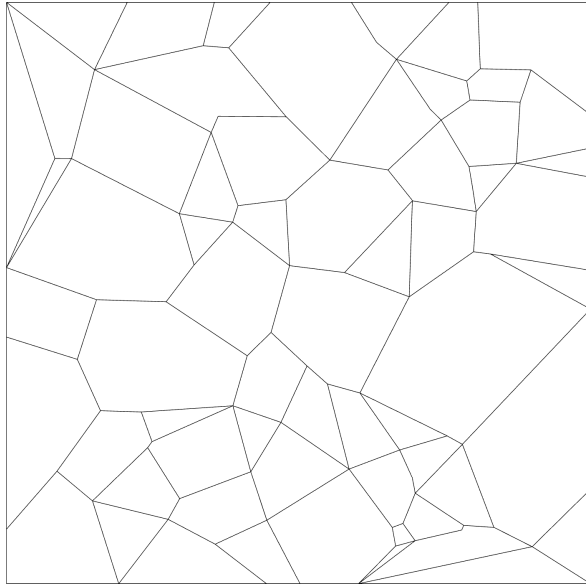


Figura 52: Malla poligonal generada desde cavidades a partir de la Figura 50 antes de postprocesar con 62 polígonos

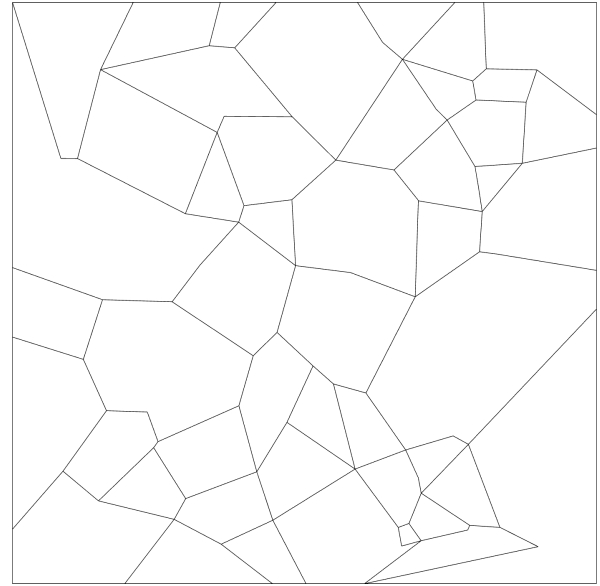


Figura 53: Malla de la Figura 52 después de postprocesar con 48 polígonos

También, en la Figura 54 y la Figura 55, se muestra la misma malla de la Figura 47 y la Figura 51 respectivamente, con la nueva implementación, las cuales resultan en mallas idénticas:

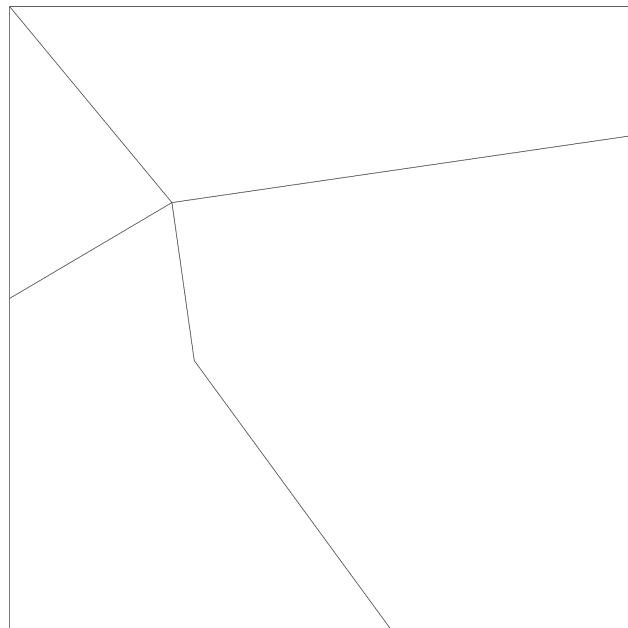


Figura 54: Malla de la Figura 47 con la nueva implementación de Polylla con 4 polígonos

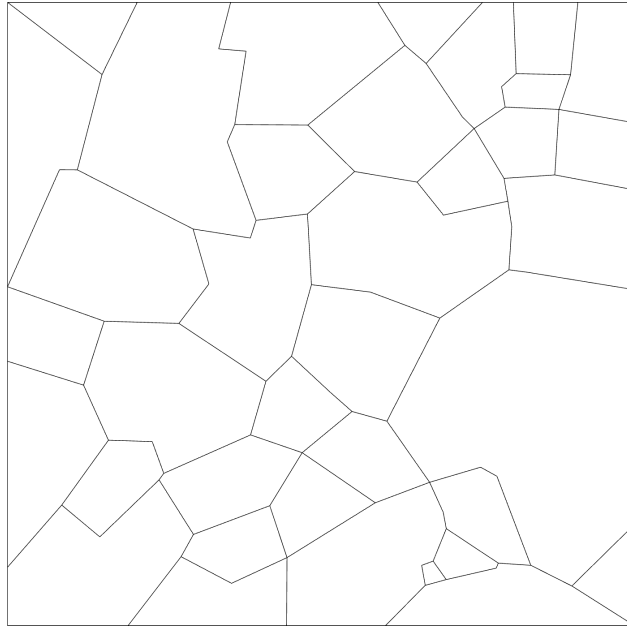


Figura 55: Malla de la Figura 51 con la nueva implementación de Polylla con 37 polígonos

Las mallas anteriores se generaron cargando los archivos `.off` resultantes de ejecutar Polylla y el generador basado en cavidades en el visualizador Camaron-Web[51]. La malla original también se cargó en Camaron-Web después de convertirla a `.off` utilizando el *script* `triangle_to_off.py` en el repositorio del proyecto.

5.3. Tablas comparativas de cualidades geométricas, tiempo y memoria

Las tablas siguientes muestran el promedio de estas 5 mallas por cada tamaño para distintas métricas:

Vertices	Malla original	Cavidades (antes de postproceso)	Cavidades (después de postproceso)	Polylla Origi- nal	Polylla Nuevo
10	10,0	4,8	3,2	3,6	3,6
10^2	173,2	68,6	51,6	35,8	35,8
10^3	1920,4	756,2	585,8	319,4	319,4
10^4	19748,0	7771,4	6020,6	3228,0	3228,0
10^5	199194,6	78029,6	60583,6	32280,8	32280,8
10^6	1997479,0	781730,0	607874,6	322388,2	322388,2

Tabla 1: Tabla comparativa de cantidad promedio de polígonos según cantidad de vértices para cada generador

Cantidad de vertices	Polylla Original	Generador de cavidades	Polylla nuevo
10	0,005	0,039	0,005
10^2	0,024	0,258	0,029
10^3	0,178	4,215	0,221
10^4	1,737	81,285	2,338
10^5	26,938	3155,964	32,112
10^6	299,942	586558,400	318,372

Tabla 2: Tabla comparativa de tiempo de ejecución total en milisegundos

Cantidad de vertices	Polylla Original	Generador de cavidades	Polylla nuevo
10	2	4	3
10^2	30	61	44
10^3	325	594	404
10^4	3294	5634	3675
10^5	33061	66042	46253
10^6	331880	610940	413287

Tabla 3: Tabla comparativa de uso de memoria total en Kilobytes, truncado a la unidad

Cantidad de vertices	Polylla Original	Generador de cavidades
10	61,1%	75%
10^2	41,3%	70,5%
10^3	39,69%	72,8%
10^4	39,44%	72,4%
10^5	39,44%	72,8%
10^6	39,42%	72,8%

Tabla 4: Tabla comparativa de porcentaje de polígonos convexos

	Ángulos Polylla Original		Ángulos Generador de cavidades	
Cantidad de vertices	Mínimo	Máximo	Mínimo	Máximo
10	43,68	206,97	35,25	207,96
10^2	21,11	278,76	11,70	290,46
10^3	14,69	290,22	6,53	305,70
10^4	6,16	303,04	1,66	335,42
10^5	< 0,01	311,10	0,30	340,92
10^6	< 0,01	330,59	0,26	350,07

Tabla 5: Tabla comparativa de ángulo mínimo y máximo en grados

Estos datos se obtuvieron a partir de múltiples ejecuciones de cada programa con la opción para escribir datos a formato `.json`, junto con el `script count_edges.py` en el repositorio del proyecto disponible en <https://github.com/Tchy258/Delaunay-cavity>.

También a continuación se muestra en detalle la distribución de valores de tiempo y memoria promedio para las mallas de generadas, tanto para el generador basado en cavidades como para ambas versiones de Polylla:

	Cantidad de vértices (Cavidad)					
Métrica de tiempo	10	100	1000	10000	100000	1000000
Ordenamiento de triángulos	0,008	0,057	1,217	20,441	458,995	8840,892
Cómputo de la cavidad	0,016	0,123	1,692	16,846	243,746	2655,512
Inserción de la cavidad	0,001	0,008	0,298	13,431	1015,383	407111,740
Postprocesado	0,011	0,061	0,917	29,119	1397,382	167435,400
Tiempo total	0,039	0,258	4,215	81,285	3155,964	586558,400

Tabla 6: Tabla de tiempos promedio en milisegundos detallados para el generador basado en cavidades según cantidad de vértices, truncado a 3 decimales

	Cantidad de vértices (Cavidad)					
Métrica de memoria	10	100	1000	10000	100000	1000000
Array de cavidades	0,96	16,63	184,36	1895,81	19122,68	191757,98
Array de visitados	0,01	0,17	1,92	19,75	199,19	1997,48
<i>Union find</i> de representantes	0,15	2,18	23,36	237,98	2393,55	23979,82
Aristas de entrada	1,54	24,58	196,61	1572,86	25165,82	201326,59
Aristas de salida	0,91	13,07	140,13	1427,86	14361,29	143878,94
Vértices de entrada	0,24	2,40	24,00	240,00	2400,00	24000,00
Vértices de salida	0,24	2,40	24,00	240,00	2400,00	24000,00
Memoria total	4,05	61,42	594,37	5634,25	66042,54	610940,82

Tabla 7: Tabla de uso promedio de memoria detallado en KB para el generador basado en cavidades según cantidad de vértices

	Cantidad de vértices (Polylla nuevo)					
Métrica de tiempo	10	100	1000	10000	100000	1000000
Etiquetado de aristas máximas	< 0,001	0,006	0,055	0,664	10,606	116,328
Etiquetado de aristas frontera	< 0,001	0,004	0,046	0,426	5,835	53,702
Etiquetado de aristas semilla	0,001	0,006	0,035	0,372	5,113	48,793
Recorrido buscando polígonos no simples	0,002	0,010	0,070	0,771	9,259	86,163
Reparación de polígonos no simples	< 0,001	0,001	0,012	0,104	1,296	13,385
Recorrido y reparación	0,002	0,012	0,083	0,875	10,556	99,548
Tiempo total	0,005	0,029	0,221	2,338	32,112	318,372

Tabla 8: Tabla de tiempos promedio en milisegundos detallados para Polylla nuevo según cantidad de vértices, truncado a 3 decimales

	Cantidad de vértices (Polylla nuevo)					
Métrica de memoria	10	100	1000	10000	100000	1000000
Aristas máximas	0,04	0,54	5,84	59,49	598,39	5994,96
Aristas frontera	0,04	0,54	5,84	59,49	598,39	5994,96
Aristas semilla	0,02	0,23	2,05	16,38	131,07	2097,15
Aristas barrera	0,04	0,54	5,84	59,49	598,39	5994,96
Lista de triángulos	0,00	0,01	0,02	0,04	0,04	0,06
Aristas de entrada	1,54	24,58	196,61	1572,86	25165,82	201326,59
Aristas de salida	0,91	13,07	140,13	1427,86	14361,29	143878,94
Vértices de entrada	0,24	2,40	24,00	240,00	2400,00	24000,00
Vértices de salida	0,24	2,40	24,00	240,00	2400,00	24000,00
Memoria total	3,06	44,31	404,33	3675,62	46253,39	413287,61

Tabla 9: Tabla de uso promedio de memoria detallados en KB para Polylla nuevo según cantidad de vértices

	Cantidad de vértices (Polylla original)					
Métrica de tiempo	10	100	1000	10000	100000	1000000
Etiquetado de aristas máximas	< 0,001	0,003	0,039	0,426	7,277	96,912
Etiquetado de aristas frontera	< 0,001	0,004	0,042	0,412	5,960	62,505
Etiquetado de aristas semilla	0,003	0,008	0,036	0,317	5,110	50,080
Recorrido buscando polígonos no simples	< 0,001	0,006	0,050	0,502	7,478	79,074
Reparación de polígonos no simples	< 0,001	0,001	0,009	0,078	1,111	11,370
Recorrido y reparación	< 0,001	0,007	0,060	0,581	8,589	90,445
Tiempo total	0,005	0,024	0,178	1,737	26,938	299,942

Tabla 10: Tabla de tiempos promedio en milisegundos detallados para Polylla original según cantidad de vértices, truncado a 3 decimales

	Cantidad de vértices (Polylla original)					
Métrica de memoria	10	100	1000	10000	100000	1000000
Aristas máximas	0,04	0,54	5,84	59,49	598,39	5994,96
Aristas frontera	0,04	0,54	5,84	59,49	598,39	5994,96
Aristas semilla	0,02	0,23	2,05	16,38	131,07	2097,15
Aristas barrera	0,04	0,54	5,84	59,49	598,39	5994,96
Lista de triángulos	0,00	0,01	0,02	0,04	0,04	0,06
Malla de entrada	0,76	10,89	116,78	1189,88	11967,74	119899,12
Malla de salida	0,76	10,89	116,78	1189,88	11967,74	119899,12
Vértices de entrada	0,48	4,80	48,00	480,00	4800,00	48000,00
Vértices de salida	0,26	2,42	24,02	240,02	2400,02	24000,02
Memoria total	2,40	30,87	325,16	3294,69	33061,78	331880,34

Tabla 11: Tabla de uso promedio de memoria detallados en KB para Polylla original según cantidad de vértices

5.4. Distribución de polígonos según cantidad de aristas

A continuación se presentan múltiples gráficos mostrando la distribución promedio de cantidad de polígonos según su cantidad de aristas en las mismas ejecuciones anteriores considerando el postprocesado en el caso del generador basado en cavidades:

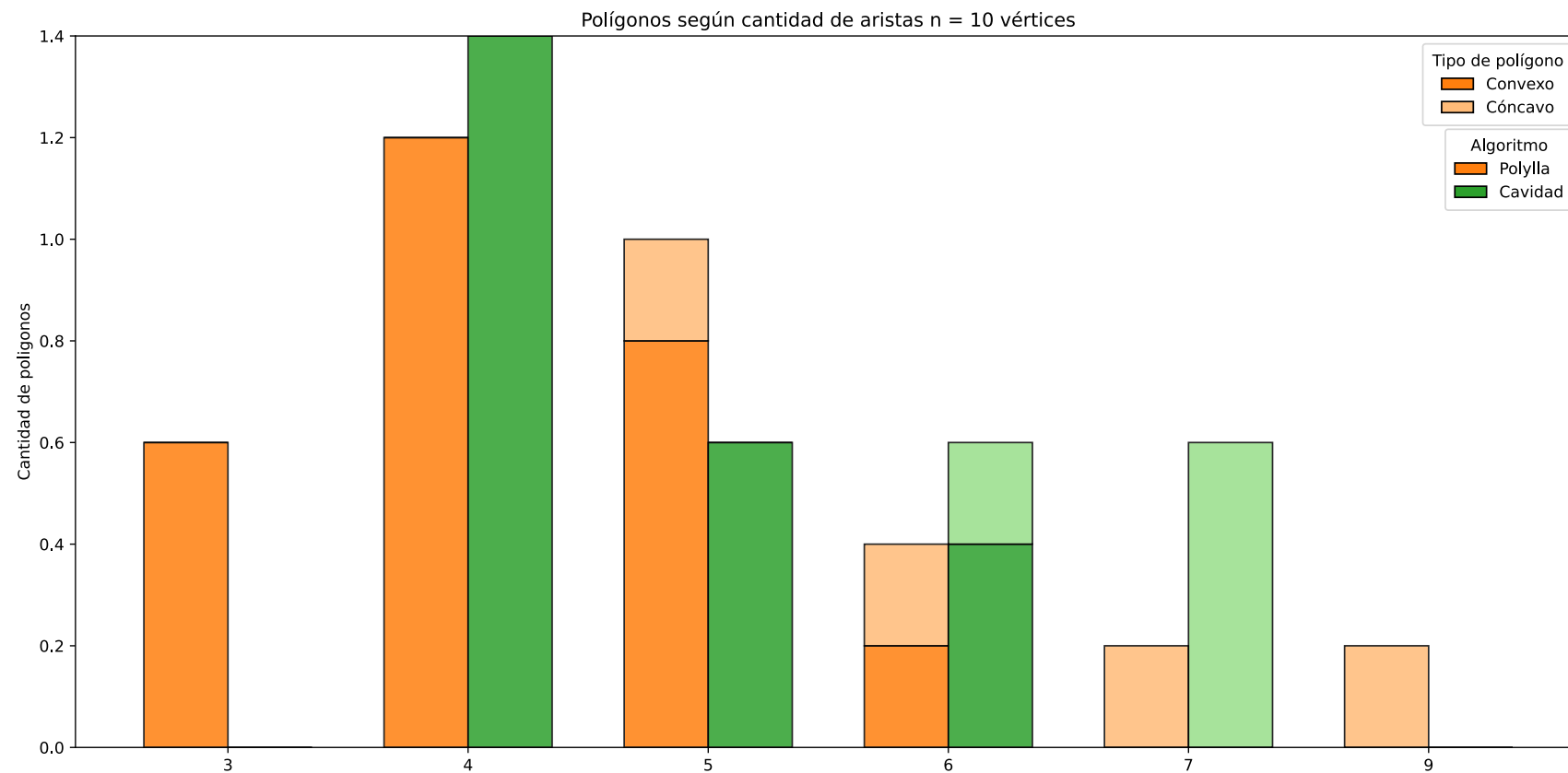


Figura 56: Distribución promedio de polígonos según cantidad de aristas con 10 vértices

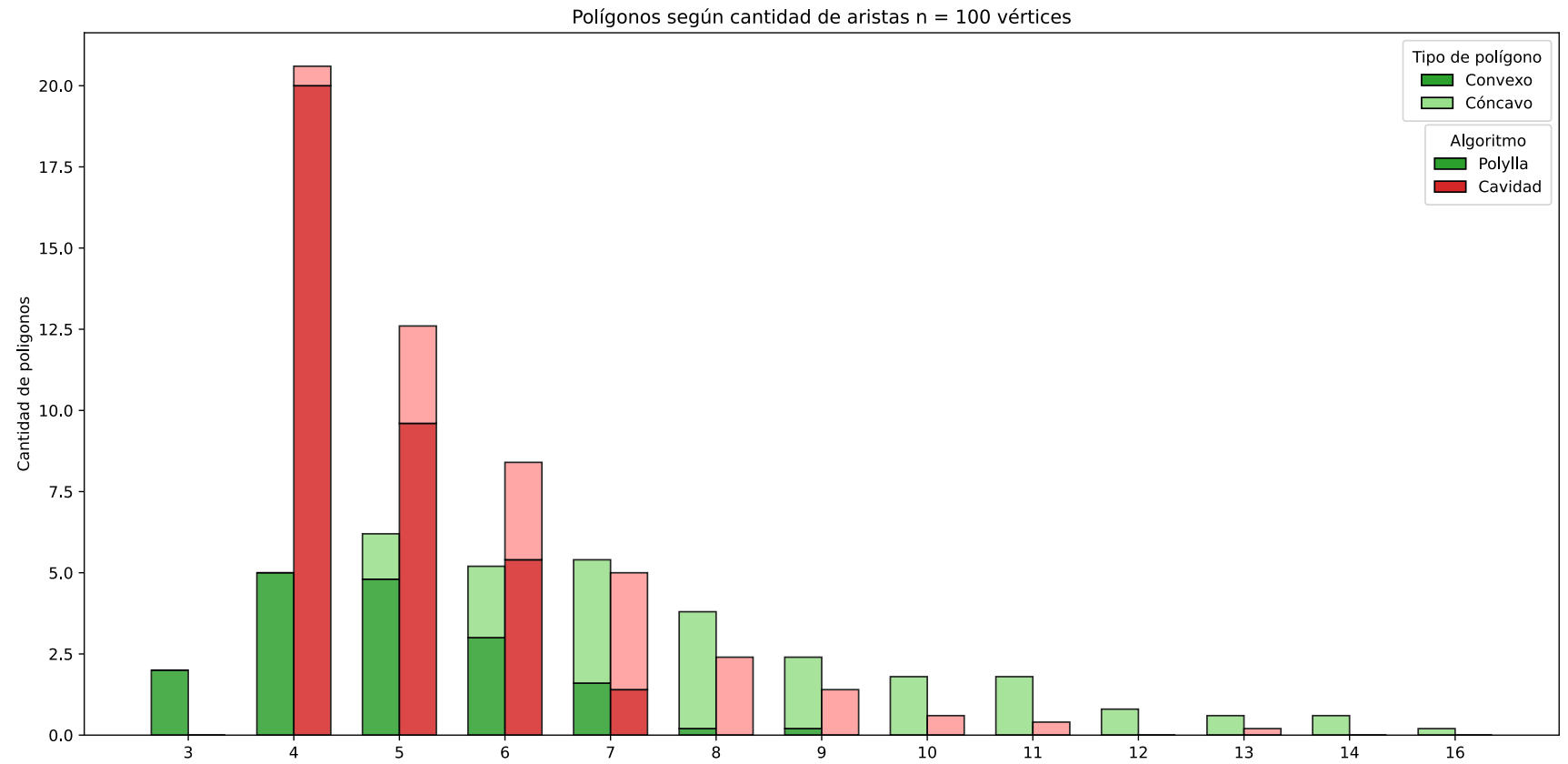


Figura 57: Distribución promedio de polígonos según cantidad de aristas con 100 vértices

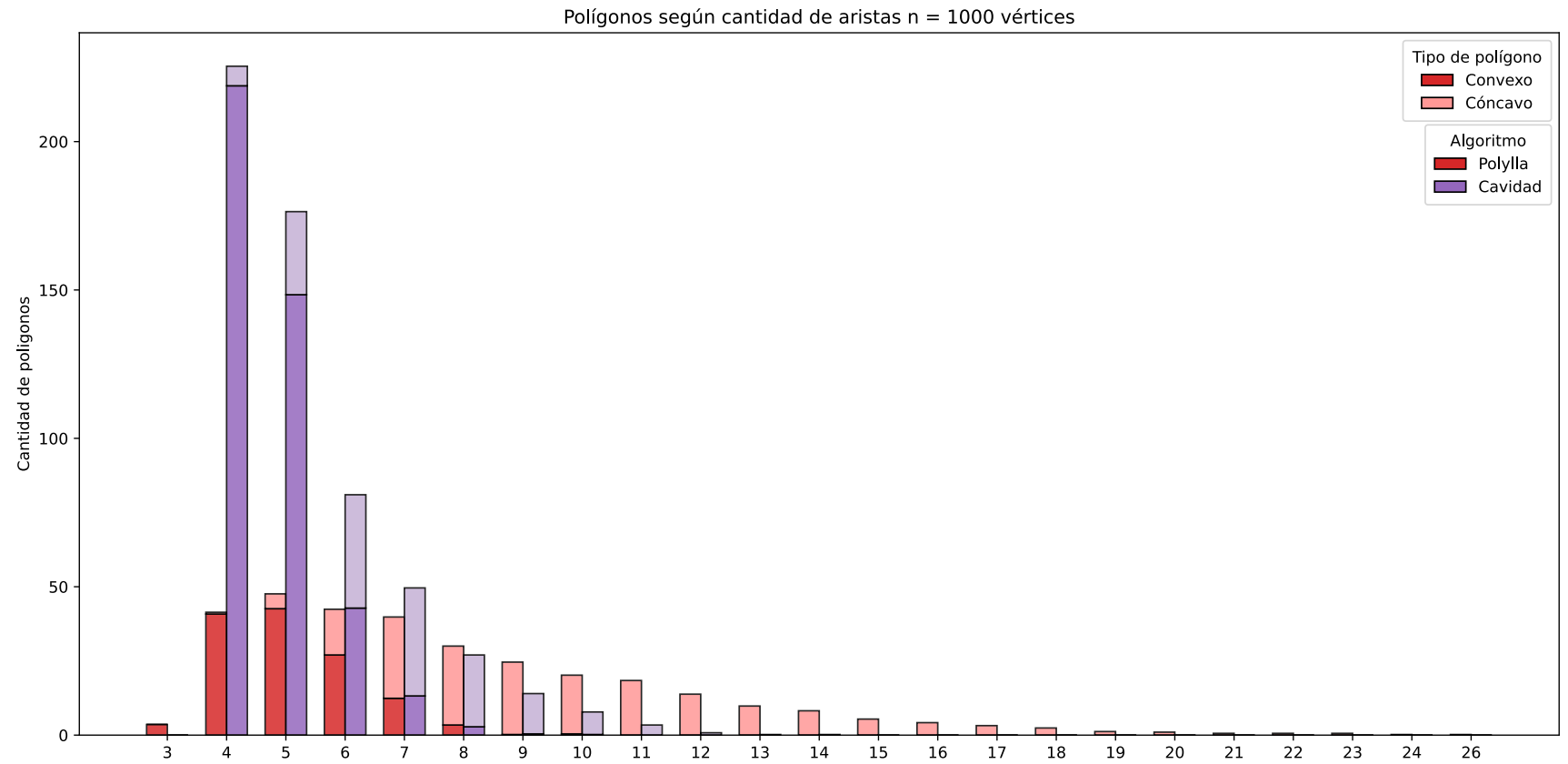


Figura 58: Distribución promedio de polígonos según cantidad de aristas con 1000 vértices

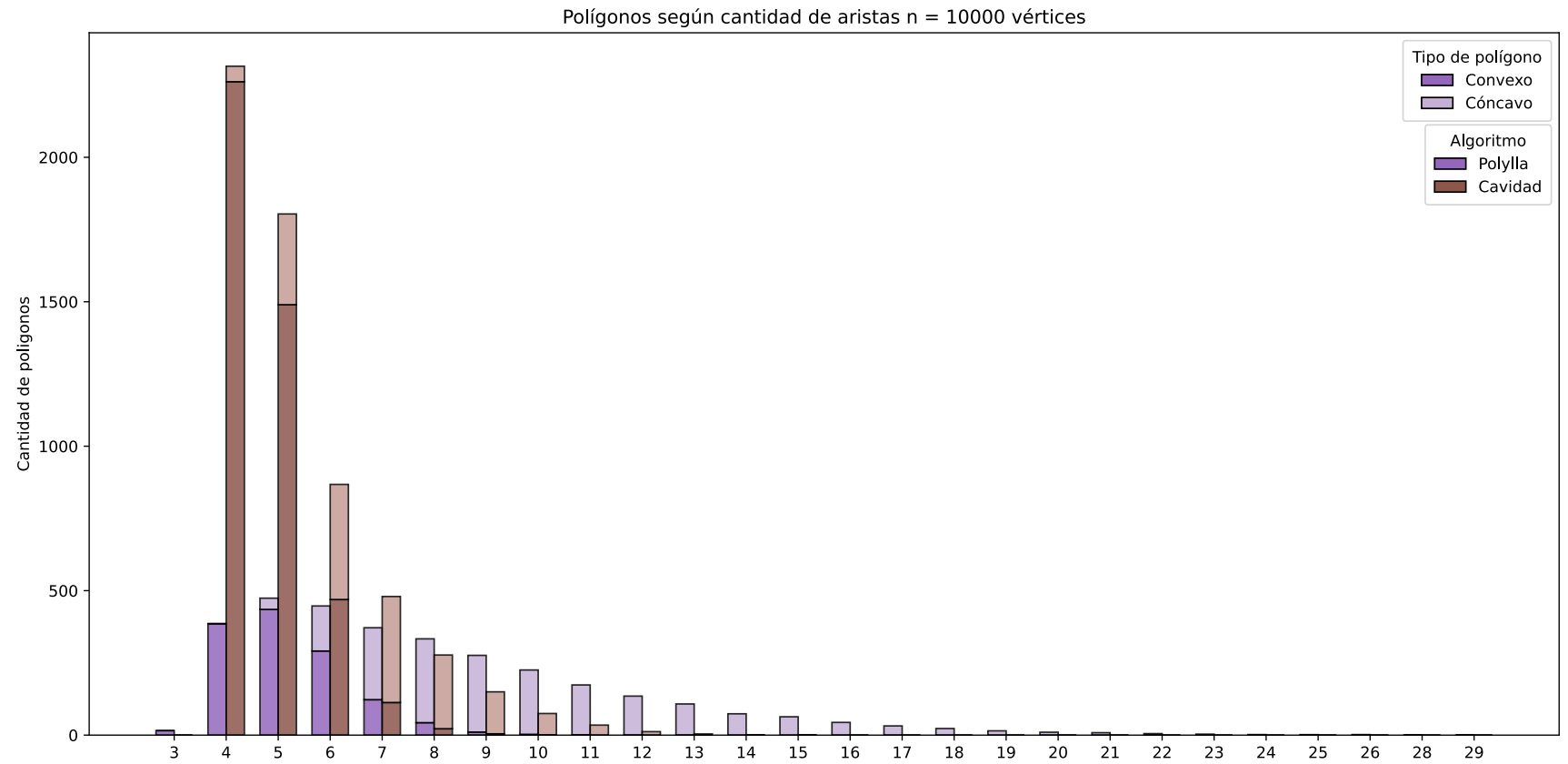


Figura 59: Distribución promedio de polígonos según cantidad de aristas con 10000 vértices

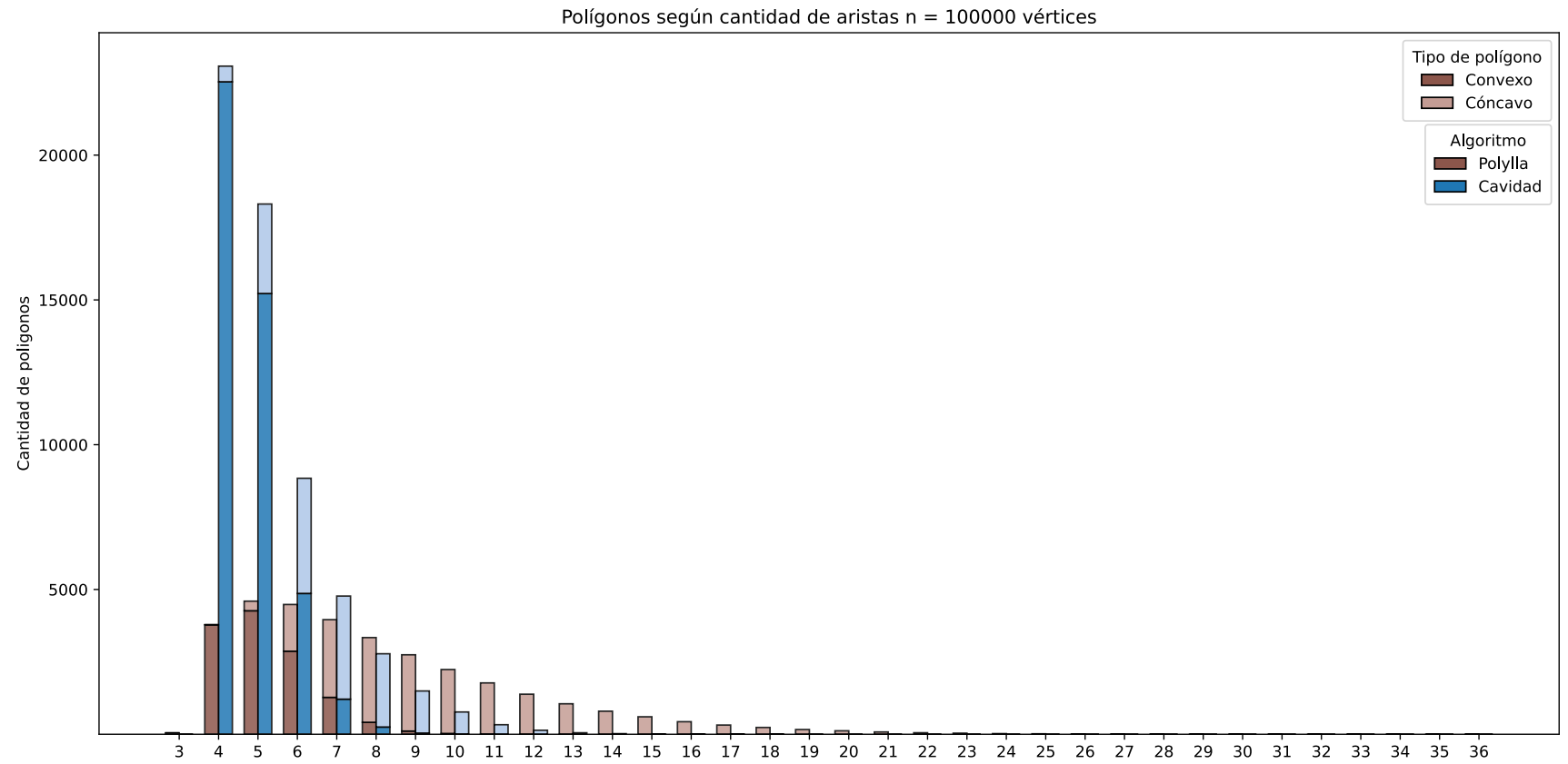


Figura 60: Distribución promedio de polígonos según cantidad de aristas con 100000 vértices

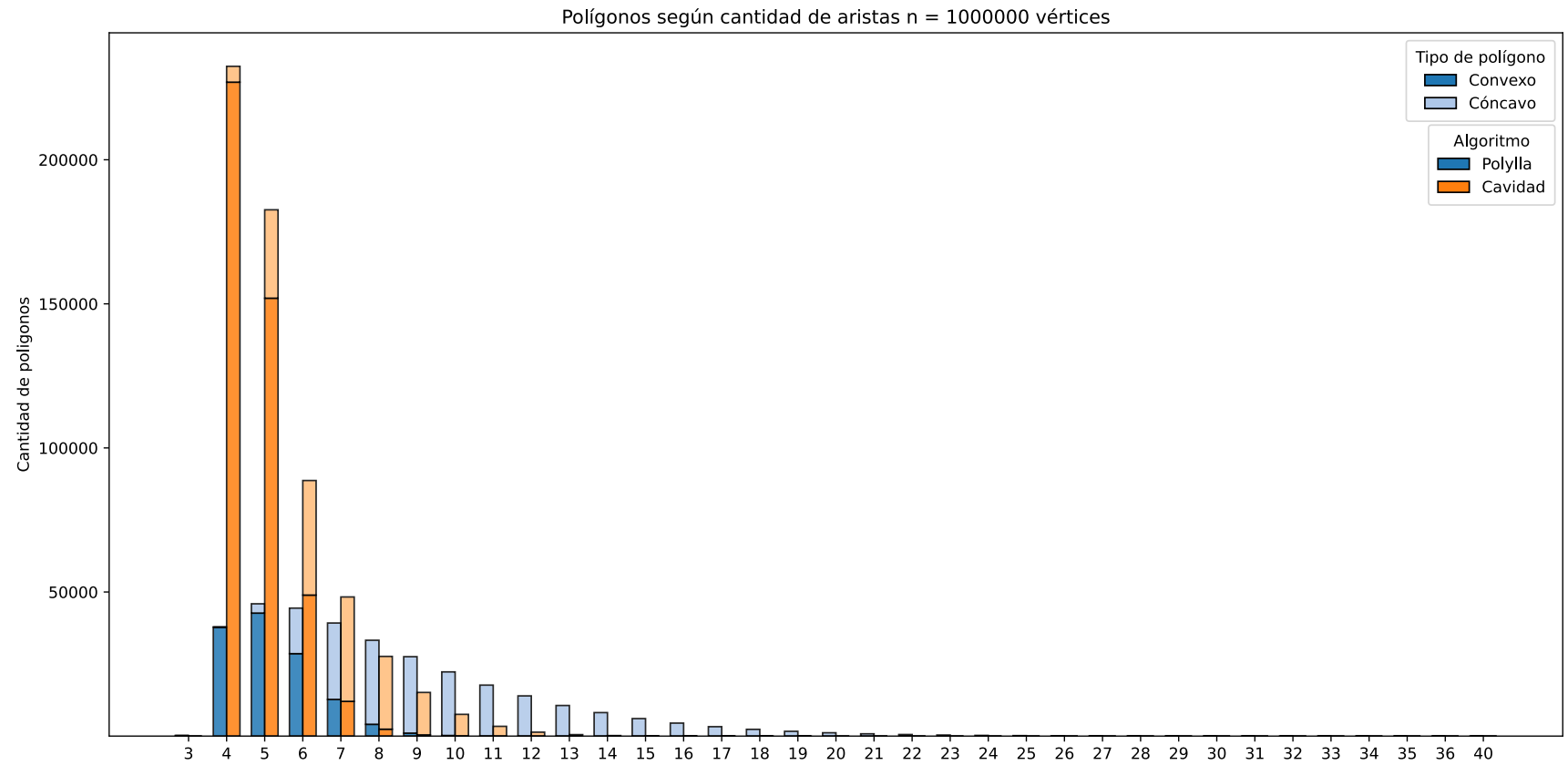


Figura 61: Distribución promedio de polígonos según cantidad de aristas con 1000000 de vértices

5.5. Análisis y discusión

Los resultados obtenidos permiten comparar el comportamiento del algoritmo de generación de mallas basado en cavidades con Polylla, considerando métricas de eficiencia, calidad geométrica y características estructurales de las mallas generadas. En esta sección se analizan dichas métricas y se discuten sus implicancias en el contexto de la generación de mallas poligonales arbitrarias.

5.5.1. Eficiencia en tiempo y memoria

En términos de tiempo de ejecución y uso de memoria, los resultados muestran de forma consistente que el algoritmo basado en cavidades presenta un mayor costo computacional que Polylla, tanto en su implementación original como en su reescritura. Esta diferencia se vuelve más significativa a medida que aumenta el número de vértices de la malla de entrada, lo que sugiere una mayor complejidad asociada a las etapas de cómputo de cavidades, manejo de estructuras auxiliares y postprocesado.

Este comportamiento es esperable dado que el algoritmo propuesto realiza operaciones adicionales respecto a Polylla, tales como el cálculo explícito de circuncentros, la identificación de conjuntos de triángulos que forman cavidades y la posterior inserción de estas en la malla. Además, la flexibilidad del diseño basado en criterios de selección, comparadores y políticas de unión configurables con *templates*; prioriza la extensibilidad y claridad del código por sobre la optimización extrema del rendimiento.

La reescritura de Polylla dentro del mismo marco de diseño permitió confirmar que parte de la brecha de rendimiento se debe a la lógica propia del algoritmo de cavidades y no únicamente a decisiones de implementación, ya que, aunque la diferencia de rendimiento entre la implementación original de Polylla y la reescritura no es despreciable (Tabla 2), sobre todo en uso de memoria, si es pequeña en comparación a la brecha que tiene Polylla con el algoritmo basado en cavidades.

5.5.2. Calidad geométrica de las mallas

Desde el punto de vista de la calidad geométrica, el algoritmo basado en cavidades demuestra ser capaz de generar mallas poligonales válidas y con un alto porcentaje de polígonos convexos. Si bien Polylla presenta, en promedio, mejores valores en métricas como una cantidad de polígonos mucho menor y menores ángulos máximos, las mallas generadas por cavidades mantienen una calidad adecuada para su uso en métodos numéricos que admiten geometría poligonal arbitraria. También tiende a generar ángulos mínimos mayores, lo que brinda mejor estabilidad a programas que procesen estas mallas, puesto que es menos probable que consideren 3 vértices como colineales, producto de errores de precisión (aproximación del ángulo a 0).

Un aspecto relevante es la distribución del número de aristas por polígono. Los resultados indican que el algoritmo basado en cavidades tiende a concentrar la mayor parte de los polígonos en un rango acotado, aproximadamente entre 4 y 7 (y en general hasta 10) aristas,

como se ve en la Figura 56 hasta la Figura 61. Esta convergencia sugiere un mayor control estructural sobre la forma de los elementos generados, lo que puede resultar beneficioso para métodos como el Virtual Element Method (VEM), donde se busca un balance entre simplicidad geométrica y flexibilidad topológica.

En contraste, Polylla tiende a generar polígonos con una mayor cantidad de aristas, lo que puede ser ventajoso en ciertos contextos, pero también puede introducir problemas al intentar utilizar estas mallas para el VEM.

Es importante destacar también que tanto Polylla como el generador basado en cavidades mantienen la misma cantidad de vértices de entrada en la salida.

5.5.3. Impacto del postprocesado

El paso de postprocesado incorporado en el algoritmo basado en cavidades tiene un impacto significativo en la reducción del número de polígonos y en la mejora de la estructura final de la malla.

Sin embargo, este beneficio también tiene un costo asociado en tiempo de ejecución, ya que, como se ve en la Tabla 6, este paso toma el segundo mayor tiempo, solo superado por la inserción inicial de las cavidades, reforzando la idea de que existe un compromiso explícito entre calidad estructural y eficiencia computacional dentro del enfoque propuesto.

5.5.4. Discusión

En conjunto, los resultados permiten afirmar que el algoritmo basado en cavidades no se equipara con Polylla en términos de rendimiento, siendo bastante más lento y utilizando más memoria que este, pero sí ofrece una alternativa viable y flexible para la generación de mallas poligonales de alta convexidad y geometría controlada al tener una concentración de polígonos con una cantidad de aristas entre 4 y 7. Su diseño modular y extensible lo posiciona como una base adecuada para futuras mejoras, optimizaciones y extensiones a otras representaciones de malla.

Las limitaciones observadas en eficiencia, junto con la ausencia de pruebas directas utilizandolas para el VEM debido a restricciones técnicas, abren líneas claras de trabajo futuro orientadas a optimizar la implementación, fortalecer la validación mediante tests unitarios y evaluar empíricamente el desempeño de las mallas generadas en aplicaciones numéricas concretas.

Capítulo 6

Conclusión

A partir de los resultados obtenidos es posible concluir que el algoritmo basado en cavidades, si bien no tiene mejor eficiencia en cuanto a tiempo o memoria comparado a Polylla, provee una alternativa viable para generar mallas de polígonos arbitrarios con un muy alto nivel de convexidad que converge al rango entre las 4 y 7 aristas aproximadamente. También la solución elaborada es bastante modular, permitiendo extensiones futuras de manera correcta al hacer uso de *templates* y *concepts*.

Dado que este trabajo de memoria fue realizado en un semestre, tiene muchos aspectos a mejorar, en particular, es altamente necesario diseñar una forma general de escribir pruebas (o *tests*) unitarias que se adapten bien a la variedad de configuraciones posibles probando invariantes sólidas que se cumplan transversalmente e idealmente sin tener que repetir tests múltiples veces. También es de suma importancia buscar maneras más eficientes de implementar el algoritmo para acercarse más al rendimiento de Polylla sin comprometer la legibilidad o flexibilidad del código. Por otra parte, quedó pendiente el probar las mallas generadas por el algoritmo para el VEM, puesto que el repositorio[52] utilizado para mostrar el uso de mallas hechas con Polylla en el VEM mencionado en su artículo[3], utiliza métodos que a día de hoy están deprecados en versiones modernas de las librerías requeridas, además el entorno en el que se ejecutaron estas pruebas es difícil de replicar en Windows debido a que las versiones antiguas de las librerías requieren que ciertos componentes, en particular cabeceras específicas de C, estén presentes en el sistema anfitrión para ser instaladas correctamente. Finalmente, es necesario idear una forma más sencilla de generar los archivos ejecutables, ya que el método actual que depende de un *script* de *Python* se puede hacer difícil de mantener en el tiempo y además, estos archivos generados tienen nombres demasiado largos, lo cual es un problema para el sistema operativo Windows sin antes habilitar la capacidad de tener rutas de archivo de tamaño superior a 260.

Bibliografía

- [1] S. Salinas y R. Carrasco, *Polylla-Mesh-DCEL*. [En línea]. Disponible en: <https://github.com/ssalinasfe/Polylla-Mesh-DCEL/>
- [2] H. Si, «Detri2 is a C++ program for generating (weighted) Delaunay triangulations for (weighted) point sets in 2d.». [En línea]. Disponible en: <https://www.wias-berlin.de/people/si/detri2.html>
- [3] S. Salinas-Fernández, N. Hitschfeld-Kahler, A. Ortiz-Bernardin, y H. Si, «POLYLLA: polygonal meshing algorithm based on terminal-edge regions», *Engineering with Computers*, vol. 38, n.º 5, pp. 4545-4567, 2022, doi: 10.1007/s00366-022-01643-4.
- [4] Nü es, «A Delaunay triangulation and its Voronoi diagram». [En línea]. Disponible en: https://commons.wikimedia.org/wiki/File:Delaunay_Voronoi.svg
- [5] Accountalive, «Showing next, prev and twin of a halfedge in a DCEL». [En línea]. Disponible en: <https://commons.wikimedia.org/wiki/File:Dcel-halfedge-connectivity.svg>
- [6] CS184/284A Course Staff, «An Introduction to the Half-Edge Data Structure». [En línea]. Disponible en: <https://cs184.eecs.berkeley.edu/sp24/docs/half-edge-intro>
- [7] J. R. Shewchuk, «Triangle: Engineering a 2D quality mesh generator and Delaunay triangulator», en *Applied Computational Geometry Towards Geometric Engineering*, M. C. Lin y D. Manocha, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 203-222.
- [8] J. R. Shewchuk, «Triangle: A Two-Dimensional Quality Mesh Generator and Delaunay Triangulator». [En línea]. Disponible en: <https://www.cs.cmu.edu/~quake/triangle.html>
- [9] B. Delaunay, «Sur la sphère vide. A la mémoire de Georges Voronoï», *Известия Российской академии наук. Серия математическая*, n.º 6, pp. 793-800, 1934.
- [10] L. Veiga, F. Brezzi, A. Cangiani, G. Manzini, L. Marini, y A. Russo, «Basic Principles of Virtual Element Methods», *Mathematical Models and Methods in Applied Sciences*, vol. 23, pp. 199-214, 2012, doi: 10.1142/S0218202512500492.
- [11] V. Jagota, A. P. S. Sethi, y K. Kumar, «Finite Element Method: An Overview», *Walailak Journal of Science and Technology (WJST)*, vol. 10, n.º 1, pp. 1-8, 2013, [En línea]. Disponible en: <https://wjst.wu.ac.th/index.php/wjst/article/view/499>

- [12] A. Hrennikoff, «Solution of Problems of Elasticity by the Framework Method», *Journal of Applied Mechanics*, vol. 8, n.º 4, pp. A169-A175, dic. 1941, doi: 10.1115/1.4009129.
- [13] H. Brönnimann, «Designing and Implementing a General Purpose Halfedge Data Structure», en *Algorithm Engineering*, G. S. Brodal, D. Frigioni, y A. Marchetti-Spaccamela, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 51-66.
- [14] K. J. Weiler, *Topological structures for geometric modeling (Boundary representation, manifold, radial edge structure)*. Rensselaer Polytechnic Institute, 1986.
- [15] «Constraints and concepts». [En línea]. Disponible en: <https://en.cppreference.com/w/cpp/language/constraints.html>
- [16] A. Sutton y B. Stroustrup, «Design of Concept Libraries for C++», 2011, pp. 97-118. doi: 10.1007/978-3-642-28830-2_6.
- [17] «Templates — C++ language». [En línea]. Disponible en: <https://en.cppreference.com/w/cpp/language/templates.html>
- [18] B. Stroustrup, *The C++ Programming Language*, 4.^a ed. Boston, MA, USA: Addison-Wesley, 2013.
- [19] Oracle Inc., «What Is an Interface?». [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/java/concepts/interface.html>
- [20] J. Ruppert, «A Delaunay Refinement Algorithm for Quality 2-Dimensional Mesh Generation», *Journal of Algorithms*, vol. 18, n.º 3, pp. 548-585, 1995, doi: <https://doi.org/10.1006/jagm.1995.1021>.
- [21] C. Lawson, «Software for C1 Surface Interpolation», *Mathematical Software*. Academic Press, pp. 161-194, 1977. doi: <https://doi.org/10.1016/B978-0-12-587260-7.50011-X>.
- [22] M.-C. Rivara, «New longest-edge algorithms for the refinement and/or improvement of unstructured triangulations», *International Journal for Numerical Methods in Engineering*, vol. 40, n.º 18, pp. 3313-3324, 1997, doi: [https://doi.org/10.1002/\(SICI\)1097-0207\(19970930\)40:18%3C3313::AID-NME214%3E3.0.CO;2-%23](https://doi.org/10.1002/(SICI)1097-0207(19970930)40:18%3C3313::AID-NME214%3E3.0.CO;2-%23).
- [23] G. Voronoi, «Nouvelles applications des paramètres continus à la théorie des formes quadratiques. Premier mémoire. Sur quelques propriétés des formes quadratiques

- positives parfaites.», *Journal für die reine und angewandte Mathematik (Crelles Journal)*, vol. 1908, n.º 133, pp. 97-102, 1908, doi: doi:10.1515/crll.1908.133.97.
- [24] The CGAL Project, «CGAL: Computational Geometry Algorithms Library». [En línea]. Disponible en: <https://www.cgal.org/>
 - [25] E. Gamma, R. Helm, R. Johnson, y J. Vlissides, «Strategy», en *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994, pp. 315-322.
 - [26] K. Driesen y U. Hölzle, «The Direct Cost of Virtual Function Calls in C++», en *ACM SIGPLAN Notices*, ACM, 1996, pp. 137-148. doi: 10.1145/236337.236386.
 - [27] G. Padlewski, M. Pszeniczny, y N. R. Smith, «Modeling the Invariance of Virtual Pointers in LLVM», *arXiv preprint arXiv:2003.04228*, 2020, [En línea]. Disponible en: <https://arxiv.org/abs/2003.04228>
 - [28] CppReference contributors, «Using-declarations». [En línea]. Disponible en: https://en.cppreference.com/w/cpp/language/using_declaration
 - [29] J. R. Shewchuk, «Lecture Notes on Geometric Robustness», *Department of Electrical Engineering and Computer Sciences, University of California at Berkeley*, pp. 19-20, abr. 2013.
 - [30] GeeksforGeeks, «Problem with std::vector<bool> in C++». [En línea]. Disponible en: <https://www.geeksforgeeks.org/cpp/problem-with-std-vector-bool-in-cpp/>
 - [31] A. Allain y LearnCpp, «std::vector<bool>». [En línea]. Disponible en: <https://www.learncpp.com/cpp-tutorial/stdvector-bool/>
 - [32] M. Austern, «N1847: vector<bool>: More Problems, Better Solutions», 2005. [En línea]. Disponible en: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2005/n1847.pdf>
 - [33] AUTOSAR Consortium, «AUTOSAR C++14 Rule A18-1-2: Avoid using std::vector<bool>». [En línea]. Disponible en: <https://www.mathworks.com/help/bugfinder/ref/autosarc14rulea1812.html>
 - [34] MISRA Consortium, «MISRA C++:2023 Rule 26.3.1: Specialization std::vector<bool> shall not be used». [En línea]. Disponible en: <https://www.mathworks.com/help/bugfinder/ref/misracpp2023rule26.3.1.html>

- [35] S. Meyers, *Effective STL: 50 Specific Ways to Improve Your Use of the Standard Template Library*. Addison-Wesley, 2001.
- [36] N. M. Josuttis, *The C++ Standard Library: A Tutorial and Reference*, 2nd ed. Addison-Wesley, 2012.
- [37] «std::unordered_map Class Template Reference — libstdc++». [En línea]. Disponible en: <https://gcc.gnu.org/onlinedocs/gcc-12.4.0/libstdc++/api/a08628.html>
- [38] E. Niebler, «constexpr if», jun. 2016. [En línea]. Disponible en: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0292r2.pdf>
- [39] M. Matsumoto y T. Nishimura, «Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator», *ACM Transactions on Modeling and Computer Simulation*, vol. 8, n.º 1, pp. 3-30, 1998, doi: 10.1145/272991.272995.
- [40] «libstdc++ stable_sort — source code in stl_algo.h». [En línea]. Disponible en: https://gcc.gnu.org/onlinedocs/libstdc++/latest-doxygen/a00626_source.html
- [41] A. Stepanov y M. Lee, «SGI STL: stable_sort — original reference implementation». [En línea]. Disponible en: https://www.sgi.com/tech/stl/stable_sort.html
- [42] «libc++ stable_sort — source code in __algorithm/stable_sort.h». [En línea]. Disponible en: https://codebrowser.dev/llvm/libcxx/include/__algorithm/stable_sort.h.html
- [43] M. Corporation, «MSVC STL: algorithm — stable_sort implementation». [En línea]. Disponible en: <https://github.com/microsoft/STL/blob/main/stl/inc/algorithm>
- [44] «std::stable_sort — C++ reference». [En línea]. Disponible en: https://en.cppreference.com/w/cpp/algorithm/stable_sort
- [45] «libstdc++ stable_partition — source code in stl_algo.h». [En línea]. Disponible en: https://gcc.gnu.org/onlinedocs/libstdc++/libstdc++-html-USERS-4.0/stl__algo_8h.html
- [46] G. D'Angelo, «libstdc++: add constexpr stable_partition (gcc commit r15-8970)». [En línea]. Disponible en: <https://gcc.gnu.org/pipermail/libstdc++-cvs/2025q1/042783.html>

- [47] «libc++ stable_partition — source code in __algorithm/stable_partition.h». [En línea]. Disponible en: https://codebrowser.dev/llvm/libcxx/include/__algorithm/stable_partition.h.html
- [48] «std::stable_partition — C++ reference». [En línea]. Disponible en: https://en.cppreference.com/w/cpp/algorithm/stable_partition
- [49] E. F. Moore, «The Shortest Path Through a Maze», en *Proc. International Symposium on the Theory of Switching*, 1959, pp. 285-292.
- [50] R. E. Tarjan, «Efficiency of a Good But Not Linear Set Union Algorithm», *Journal of the ACM*, vol. 22, n.º 2, pp. 215-225, 1975, doi: 10.1145/321879.321884.
- [51] D. Zamora-Quiroz y D. Muñoz, «Cameron-Web: A mesh visualizer focused on inspecting and evaluating the quality of meshes». [En línea]. Disponible en: <https://github.com/diegozq/cameron-web>
- [52] S. Salinas y A. Chetan, *VEM-for-polylla-testing*. [En línea]. Disponible en: <https://github.com/ssalinasfe/VEM-for-polylla-testing>
- [53] H. Schreiner y Otros, «CLI11: A Command-Line Parser for C++11 and Beyond (v2.6.1)». [En línea]. Disponible en: <https://github.com/CLIUtils/CLI11>

Anexo A

Repositorio del proyecto

El repositorio del proyecto con todo el código se encuentra disponible en <https://github.com/Tchy258/Delaunay-cavity>, posee un archivo *README.md* en inglés con las instrucciones para compilar y ejecutar los algoritmos.

Anexo B

Formatos de archivo para mallas geométricas

2.1. Formato `.off`

El formato `.off` (Object File Format) es un formato de texto utilizado para representar geometría poligonal en tres dimensiones, especialmente mallas de superficies.

Su estructura general es:

- Una cabecera con la palabra `OFF`.
- Una línea con tres enteros: número de vértices, número de caras y número de aristas.
- Una lista de vértices, donde cada línea contiene las coordenadas (x , y , z) de un vértice.
- Una lista de caras, donde cada línea comienza con el número de vértices de la cara, seguido de los índices de dichos vértices.

Este formato es común en visualización y procesamiento geométrico, y no incluye información topológica avanzada más allá de las caras.

Para representar mallas geométricas en dos dimensiones con este formato, basta con mantener fija la tercera coordenada de cada vértice, en este trabajo, se mantiene con valor 0 siempre.

2.2. Formatos `.node`, `.ele` y `.neigh`

Estos tres archivos suelen utilizarse conjuntamente para describir mallas, especialmente las generadas por el software Triangle[8]

2.2.1. Formato `.node`

El archivo `.node` contiene la información de los nodos (vértices) de la malla.

Incluye:

- Una cabecera con el número total de nodos, la dimensión (2D o 3D), el número de atributos por nodo y el número de marcadores de frontera.
- Una lista de nodos, donde cada línea especifica:
 - El índice del nodo
 - Sus coordenadas espaciales
 - Opcionalmente, atributos adicionales
 - Opcionalmente, un marcador de frontera

El marcador de frontera indica si el vértice es parte del “borde” de la malla descrita.

2.2.2. Formato **.ele**

El archivo **.ele** describe los elementos de la malla (triángulos en 2D o tetraedros en 3D).

Incluye:

- Una cabecera con el número de elementos, el número de nodos por elemento y el número de atributos.
- Una lista de elementos, donde cada línea contiene:
 - El índice del elemento
 - Los índices de los nodos que lo componen según el orden en que fueron descritos en el archivo **.node** respectivo
 - Opcionalmente, atributos del elemento

2.2.3. Formato **.neigh**

El archivo **.neigh** es opcional y almacena información de vecindad entre elementos.

Incluye:

- Una cabecera con el número de elementos y el número de vecinos por elemento.
- Una lista donde cada línea indica:
 - El índice del elemento
 - Los índices de los elementos vecinos a través de cada cara o arista
 - Un valor negativo suele indicar ausencia de vecino (frontera)

Este archivo permite reconstruir la conectividad topológica de la malla.

2.3. Formato **.ale**

El formato **.ale** se utiliza para describir mallas y datos asociados en simulaciones numéricas.

A modo general, su contenido puede incluir:

- Definición de nodos y elementos de la malla.
- Información temporal sobre el movimiento de la malla.
- Variables físicas asociadas a nodos o elementos (por ejemplo, velocidad o desplazamiento).

- Parámetros de control para la actualización de la malla durante la simulación.

A diferencia de los formatos anteriores, `.ale` no está completamente estandarizado y su estructura exacta depende del software que lo genere o utilice. En este proyecto, dado que los archivos `.ale` se utilizan para simulaciones con el VEM[10] tras convertirlos al formato `.mat` utilizado, estos contienen lo siguiente por cada línea:

- El string `Custom` que describe el tipo de dominio.
- La cantidad total de vértices
- Las coordenadas (x, y) de cada vértice separadas por saltos de línea.
- La cantidad de elementos (polígonos) de la malla.
- Un listado de caras donde cada una incluye:
 - La cantidad de vértices que componen la cara
 - El índice de cada vértice según el orden en que fueron descritos anteriormente, similar a un archivo `.ele`.
- Índices de los vértices que forman el borde de la malla (*Dirichlet boundary*).
- Un 0 (*Neumann boundary*)
- Las coordenadas que definen el *Bounding Box* de la malla, es decir, el valor mínimo y máximo de coordenada x y coordenada y de la malla en ese orden, que describen un rectángulo (o caja) que encierra a todos los otros vértices.

2.4. Formato `.mat` de MATLAB

El formato `.mat` es un formato binario desarrollado por **MATLAB** para almacenar variables del espacio de trabajo de forma persistente.

Sus características principales son:

- Puede contener múltiples variables en un solo archivo.
- Cada variable conserva su nombre, tipo y dimensiones.
- Soporta distintos tipos de datos, como:
 - Escalares y arreglos numéricos
 - Vectores y matrices
 - Arreglos multidimensionales
 - Estructuras (`struct`)
 - Celdas (`cell`)
 - Cadenas de texto
 - Datos lógicos y complejos

Internamente, existen distintas versiones del formato:

- Versiones antiguas (v4, v6, v7) basadas en almacenamiento binario propio.
- La versión v7.3, que utiliza el estándar HDF5, permitiendo manejar archivos de gran tamaño y acceder parcialmente a los datos.

El formato `.mat` es ampliamente utilizado para:

- Intercambio de datos entre scripts y funciones de MATLAB.
- Almacenamiento de resultados de simulaciones numéricas, en este proyecto, para el VEM.
- Compartir datos con otros entornos que soportan lectura de archivos `.mat`, como Python, Julia u Octave.

A diferencia de los formatos de malla, `.mat` no impone una estructura geométrica específica, sino que actúa como un contenedor general de datos.

Anexo C

CLI11

En el código del proyecto se hace uso de la biblioteca CLI11[53], la cual brinda una *API* fácil de usar para procesar argumentos desde la terminal e incluso leer configuraciones desde un archivo con distintos formatos de manera sencilla.

Anexo D

PlantUML y Clang-UML

El diagrama de clases completo del Anexo E mostrado parcialmente a lo largo del Capítulo 3 fue generado utilizando *Clang-UML*, un programa escrito en C++ que permite generar diagramas de proyectos escritos en C++, disponible en: <https://github.com/bkryza/clang-uml>.

La salida de *Clang-UML*, en esta ocasión, es un archivo de texto en *PlantUML*, un lenguaje de marcado para declarar diagramas de clases: <https://plantuml.com/es/>

El significado de cada uno de los símbolos se puede ver en detalle en el siguiente enlace: <https://plantuml.com/es/class-diagram>

A modo de resumen, los íconos y distintos formatos de texto tienen el siguiente significado:

Icono para atributo	Icono para método	Visibilidad
		private
		protected
		public

Tabla 12: Iconografía de PlantUML

Los atributos o métodos subrayados son estáticos, mientras que los que están escritos en *cursiva*, son abstractos.

Anexo E

Diagrama completo de clases

En el siguiente enlace se encuentra un diagrama de clases completo de la solución en formato *svg* para ser visualizado en un computador con el nivel de ampliación que se desee: https://github.com/Tchy258/Delaunay-cavity/blob/main/diagrams/delaunay_cavity.svg

Dado su enorme tamaño no es adecuado para mostrarse completo en el informe.

También se provee el diagrama completo de la implementación original de Polylla previamente mostrado por partes en el capítulo 3



Figura 62: Diagrama UML completo de la implementación de Polylla original